卓越工程综合训练

Comprehensive Practice for Outstanding Engineer Education Plan

课程编号：100076108



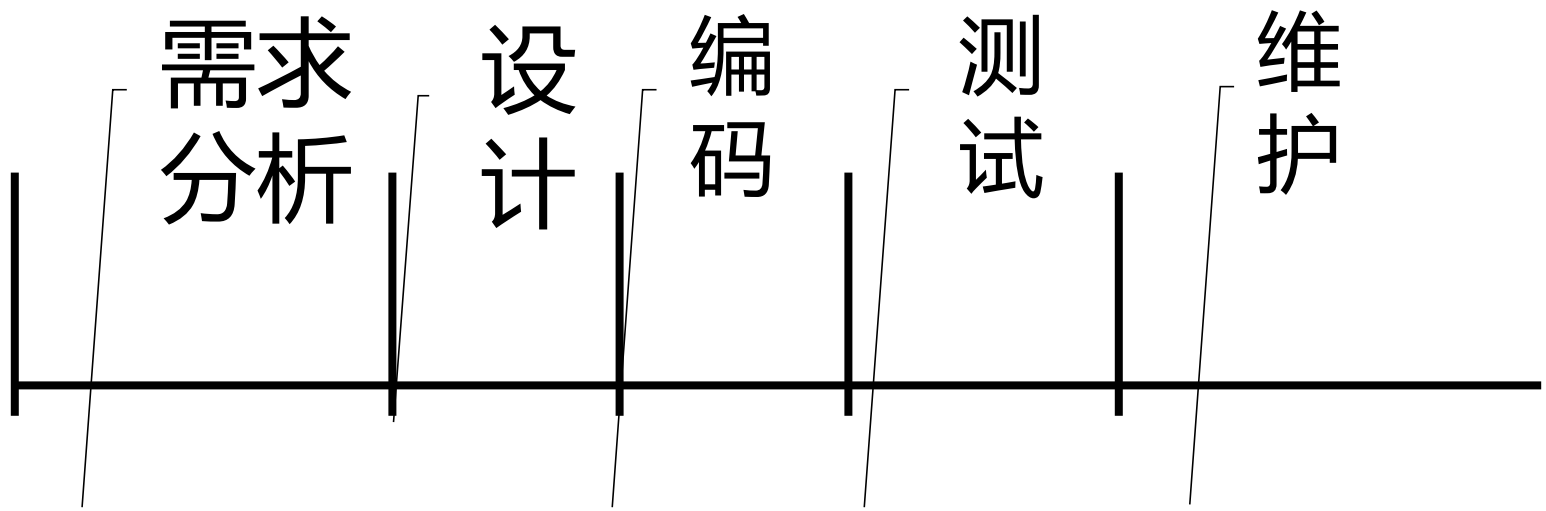
闫波

北京理工大学



软件工程

软件工程



- 软件需求
  - （正在构建的）系统必须符合的条件或具备的功能和能力。【用户需解决某一问题或达到某一目标所需的软件功能；系统或系统构件为了满足合同、规约标准或其他正式实行的文档而必须满足或具备的软件功能】
  - 需求分析的任务是借助于当前系统的物理模型导出目标系统的逻辑模型，解决目标系统“做什么”的问题。

软件工程-软件需求分析与设计

- 站在用户的角度上思考问题
- 业务 业务 还是业务

软件工程-软件需求分析与设计

- 大学教务管理系统



说明：1、2012级学生初始密码为本人身份证号的后6位（字母均为大写），请登录后及时更改密码。

2、对于2011年前入校的教师：用户名为：t+工号，如工号为：12345，则用户名为：t12345；
对于2011年（含）后入校的教师，用户名为：工号的后6位，如工号为：6120110001，则用户名为：110001。

目前系统无法支持搜狗浏览器，请用IE浏览器进行访问





软件工程-软件需求分析与设计





软件需求-存在的问题?

软件工程-软件需求分析与设计

- 对大学教务管理系统--排课、学生选课的理解?
- 多个教师上同一门课？单双周？实验？卓越计划学生？教室的安排（人数 位置）？教室调整？教师多门课时间冲突？热门课选不上？小学期？上课时间调整？放假调课？休学、退学？短期出国？学生选课的冲突？集中选课的系统压力？高并发的处理？……
- Curriculum-course-class-lessen?

软件工程-软件需求分析与设计

- 站在用户的角度上思考问题
- 业务-业务-业务



软件工程-软件需求分析与设计

- 快贷系统的合规性检查
  - 前端还是后端?
  - 业务的完整性保证?
  - 如何平衡?

软件工程-软件需求分析与设计



案例-理解需求

- 大学教务管理系统-学生基本信息管理
  - 学生基本信息有什么?
  - （学号 姓名 性别 所在学院 专业 班级 出生日期 籍贯 电话 邮箱 入学年份 培养类别 宿舍 校区 银行卡 照片…… ） 够不够?
  - 谁使用? 怎么使用? 大类招生 书院制?
  - 学生 教师 教务人员……

软件工程-软件需求分析与设计

- 大学教务管理系统-学生基本信息管理
  - 教师要求能看到学生的情况 全部学生?
  - 学生能更新自己的信息 更新哪些?
  - 教务人员能看到和修改学生信息 哪些能改
  - 导入 导出?
  - 其他?

软件工程-软件需求分析与设计

- 大学教务管理系统-学生基本信息管理
  - 增删改查（CRUD）？
  - 新学生 个人信息修改？ 上传照片？查询条件？ 删除？ 批量修改?
  - 批量导入（来源） 批量导出 统计分析?
  - 与其他模块 其他系统的接口?

软件工程-软件需求分析与设计

- 大学教务管理系统-学生基本信息管理
  - 新增（教务人员？）
  - 批量导入 批量增加（从哪里来？格式要求？）
  - 个人信息修改（谁能改？能改哪些信息？考虑权限）
  - 查询（不同的角色查询权限？哪些角色？导出吗？）
  - 删除（真正删除吗？）
  - 其他功能？ 非功能性需求？如何通过模型表示？

- 大学教务管理系统-学生基本信息管理
  - 学生类
    - 属性：学号 姓名 性别 出生日期 院系 ……
    - 行为：新增 修改 删除 查询 导入 导出 统计分析
    - 行为：选课、退课、成绩核查、出国……
    - 一个类全搞定?
    - O-R映射

案例-架构设计

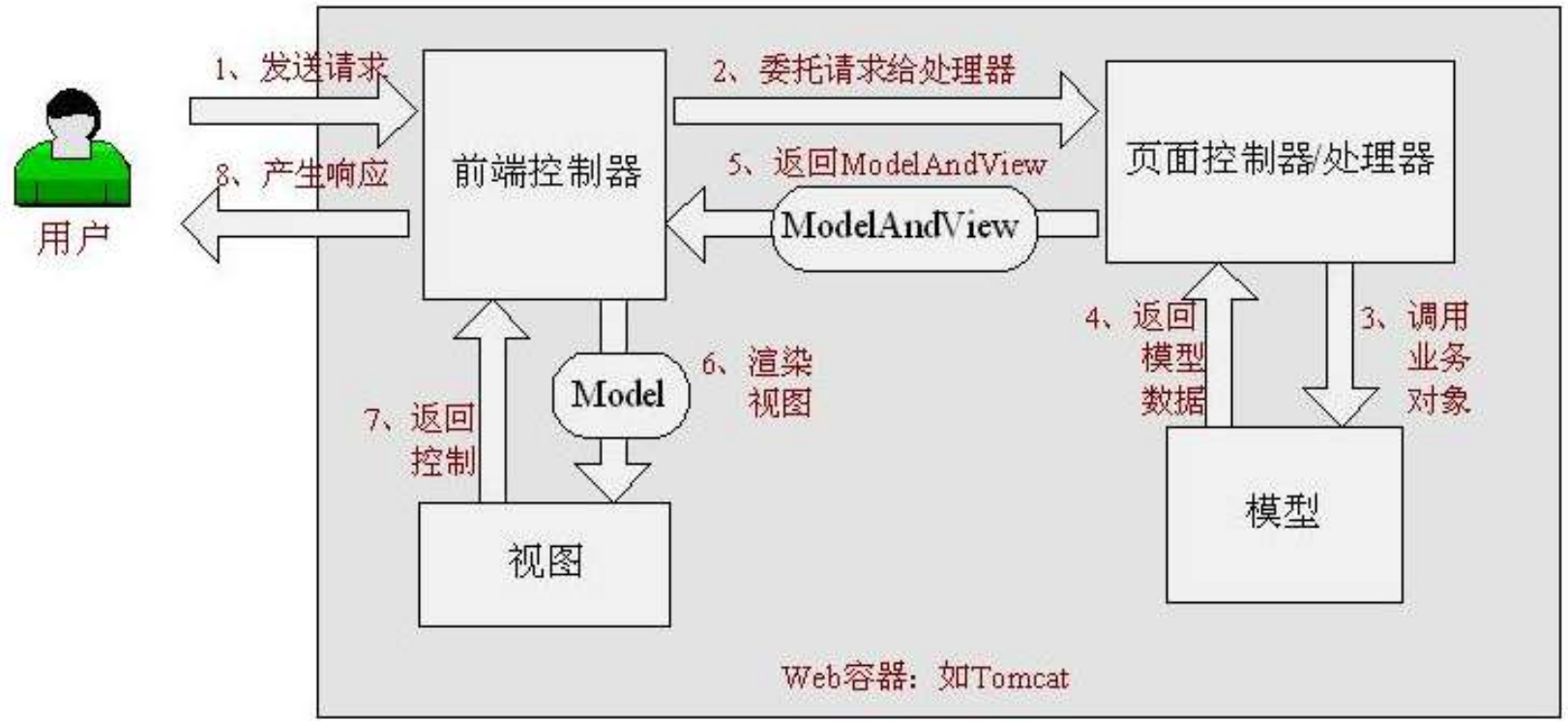
- 大学教务管理系统-学生基本信息管理
  - 开发语言？应用服务器？数据库？UML?
  - 系统架构？框架？页面风格?
  - 充分考虑接口?
  - 设计模式?
  - 非功能性的考虑？（安全 高并发 与其他系统交互 门户登录 异构性 校外访问 兼容性……）

软件工程-软件需求分析与设计

- 大学教务管理系统-学生基本信息管理
  - 系统架构
  - 框架？

    Hibernat

    Ehcache,
  - 类关系图
  - 数据库表



大学教务管理系统-学生基本信息管理

- Student数据表
- 学号,姓名,性别,学院，专业，籍贯，联系方式，……
- StuNo, Name,Sex,College,Prof,Nationalit,……
- 数据类型？长度？照片怎么存?
- 主外键关系，自增ID是否有必要?
- 学生表 学院表 书院表 专业表 地区表（分级） 性别表 培养类别表 校区表 宿舍表 照片表 银行卡表 ……

软件工程-软件需求分析与设计

- 大学教务管理系统-学生基本信息管理
  - Student类
  - Property:StuNo, Name, Sex, College, Prof, ……
  - Method：newStu(id,name,…); updStu();delStu()…
  - 类之间的关系?
  - 方法的细化；公共方法的提取；与他人的接口定义；……
  - 类关系图？如何保证高内聚松耦合?

软件工程-软件需求分析与设计

- 大学教务管理系统-学生基本信息管理
  - 需求文档
  - 设计文档（概设 详设）
  - 原型展示
  - 模型？格式？

软件工程-软件需求分析与设计

➢软件需求

用清晰、简洁

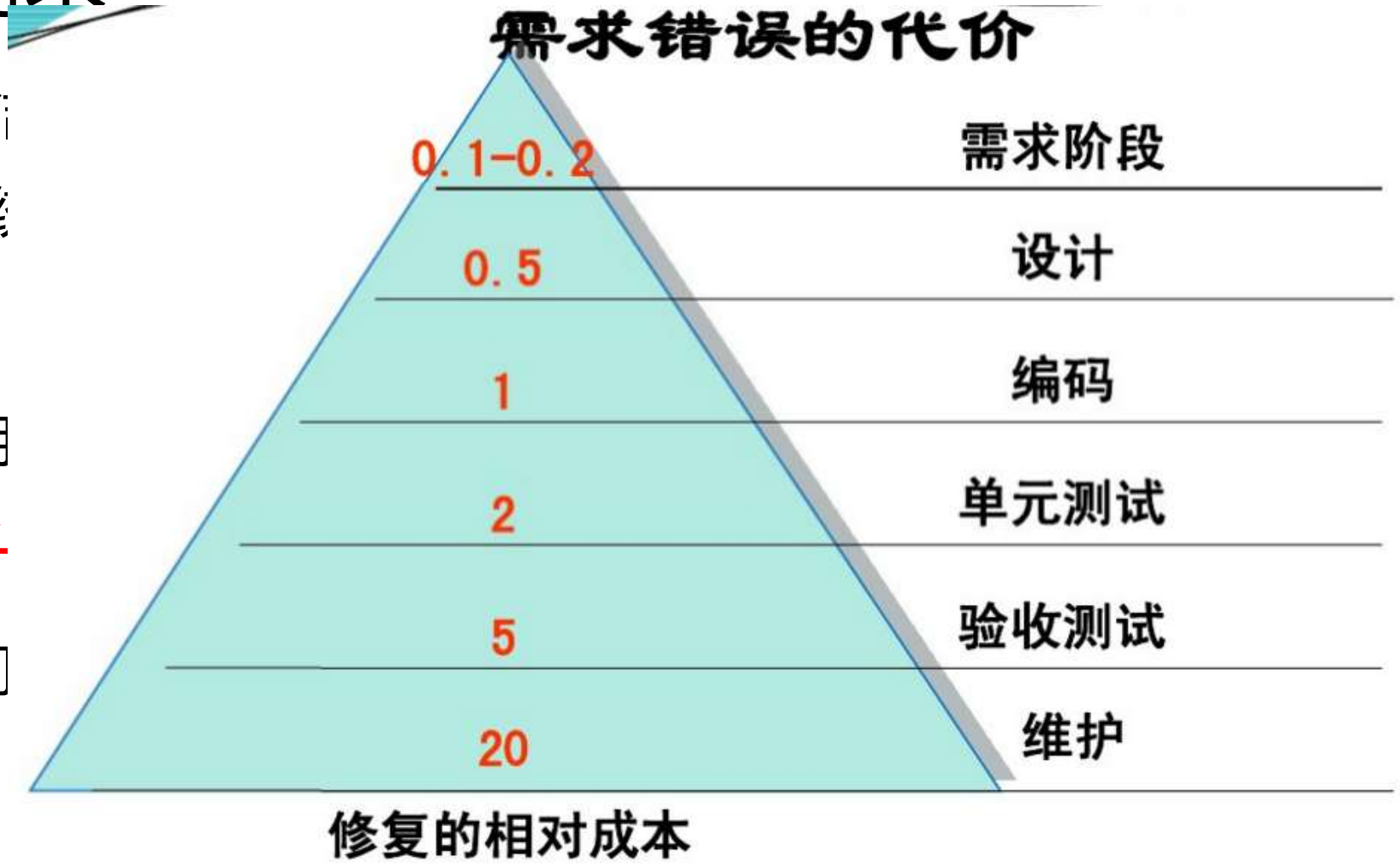
须足够使系统

要点：

1)无法说明

2)确定做什

3)确定的问



软件工程-软件需求分析与设计

- 需求工程：用经过证实的有效原理、方法，通过合适的工具和记号，系统地描述待开发产品或系统的行为特征及相关约束。
- 需求工程过程：包括：需求获取、需求分析、编写软件需求规格说明（Software Requirement Specification，SRS）、一致性验证。
- 需求与模型：模型是描述现实问题的一种手段，它能抽象和简化地反映实际产品或系统的外部特征和本质。因此，它是需求工程的主要方法。

软件工程-软件需求分析与设计

目标性功能需求

某系统需求这样写道：

根据详细调研和需求分析，系统功能必须满足：

1）编制计划、工程拨款管理，……工程必复、进度统计；

2）工程项目管理

3）计划拨款、征费收缴信息及其他收拨款信息查询统计；

4）路产信息，违章建筑信息，工程材料、进度信息管理；

5）养征信息管理，收费站信息管理；

6）养护信息管理，桥梁维护预警；

……

目标性需求不具体，缺少必要的条件限定和执行过程。

无二义性需求

通常给出的需求，都会给出必要的条件限定和执行描述，但对于它们的理解，并不一定只有一种。

例如：

下面的问题具有二义性：

“发现不友好的并且带有未知任务的或者有可能在5分钟内飞入禁区的飞机时鸣响警报。……”

软件工程-软件需求分析与设计

- 第一种解释：

不友好的、带有未知任务的飞机要鸣响警报；或者5分钟内飞入禁区的飞机要鸣响警报（无论是否友好？无论是否带有未知任务？）。

- 第二种解释：

不友好的飞机，如果带有未知任务或者5分钟内会飞入禁区，则要鸣响警报（友好的飞机无论怎样都不鸣响警报？）

- 第三种解释：……

无二义性需求-----陈述的问题有且仅有一种解释

软件工程-软件需求分析与设计

➢二义性缺陷的原因

- 对问题的理解不深入
- 表述不具体和严谨
- 自然语言本身的丰富语义表现力

➢解决二义性的途径----模型

- 系统模型可以抽象表示问题，使问题简单清晰
- 模型可以提高对问题的准确性理解
- 容易沟通并达成一致的理解
- 容易检查宏观上的完整性

随着问题的复杂，模型也会变得复杂。

软件工程-软件需求分析与设计

非功能性需求

反映软件系统质量和特性的额外要求：

过程需求：交付需求

实现方法需求

标准需求

外部需求：法规需求

费用需求

互操作性需求 ……

产品需求：可用性

性能需求

可靠性需求

可移植性需求

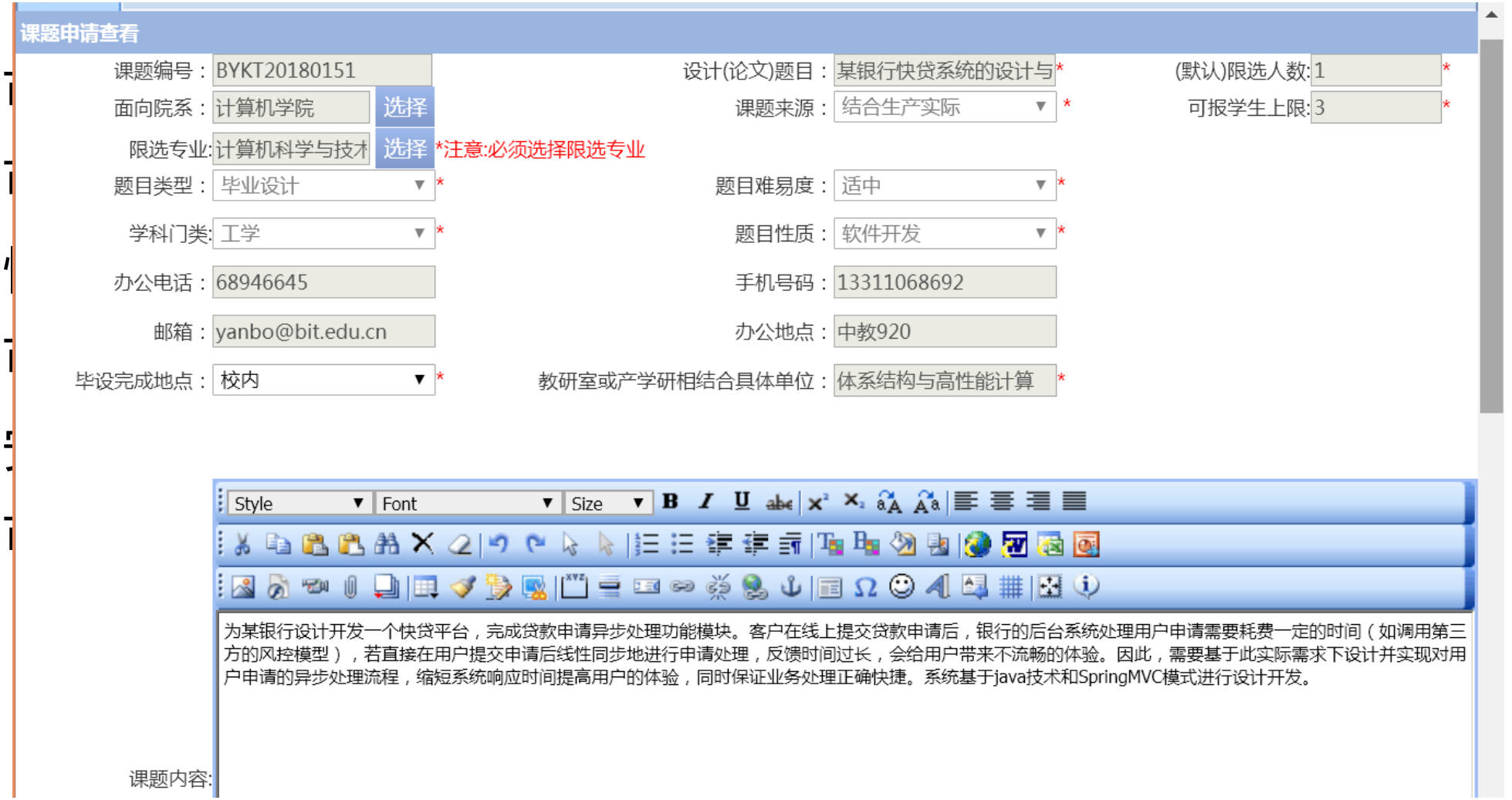
可重用性需求

安全性需求

……

软件工程-软件需求分析与设计

非功能性需求针对的问题



软件工程-软件需求分析与设计

需求工程产品

整个软件需求过程

(Software Requ

SRS不同于用户需

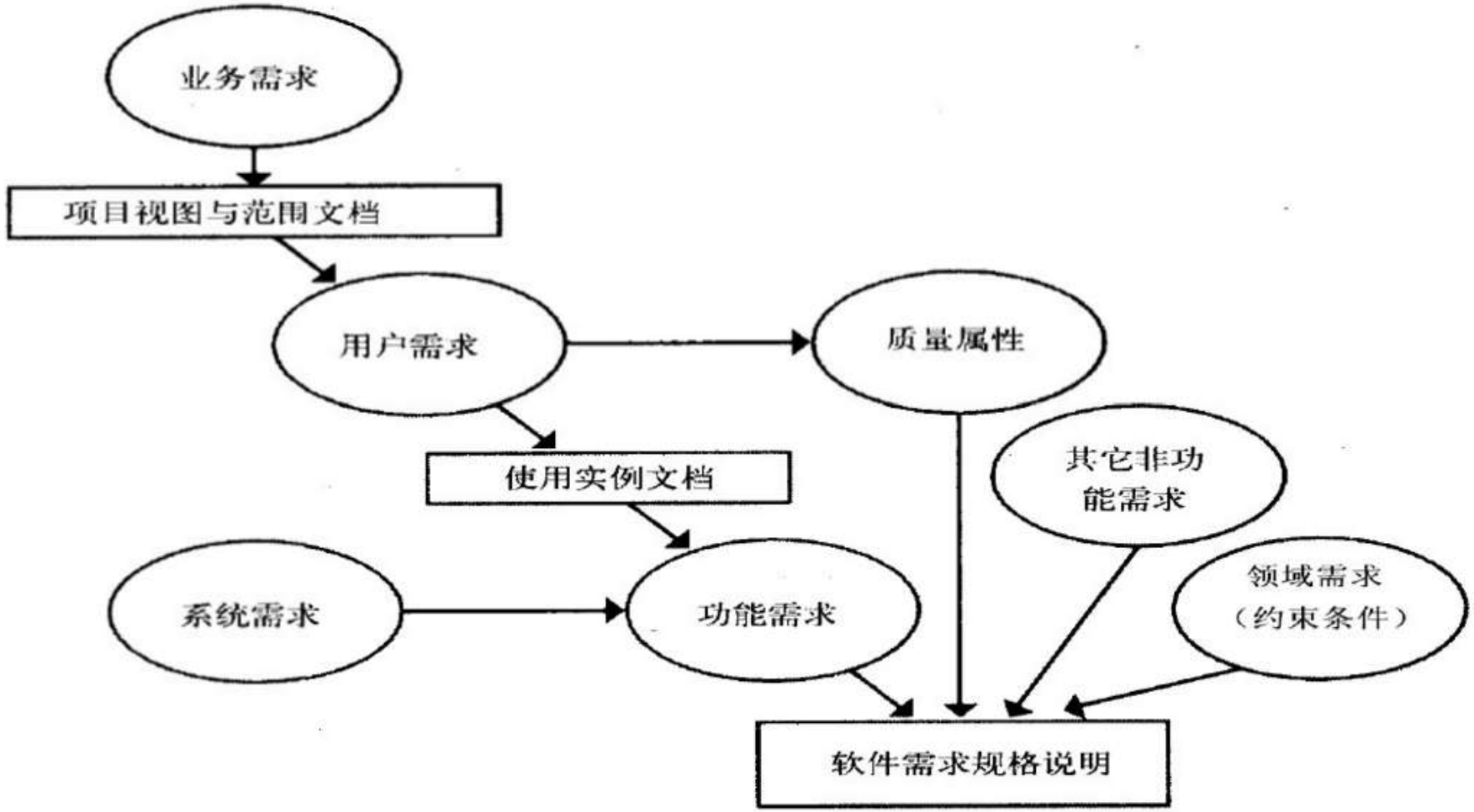
1）具有精确含义

2）便于交流，可

3）表达力强，有

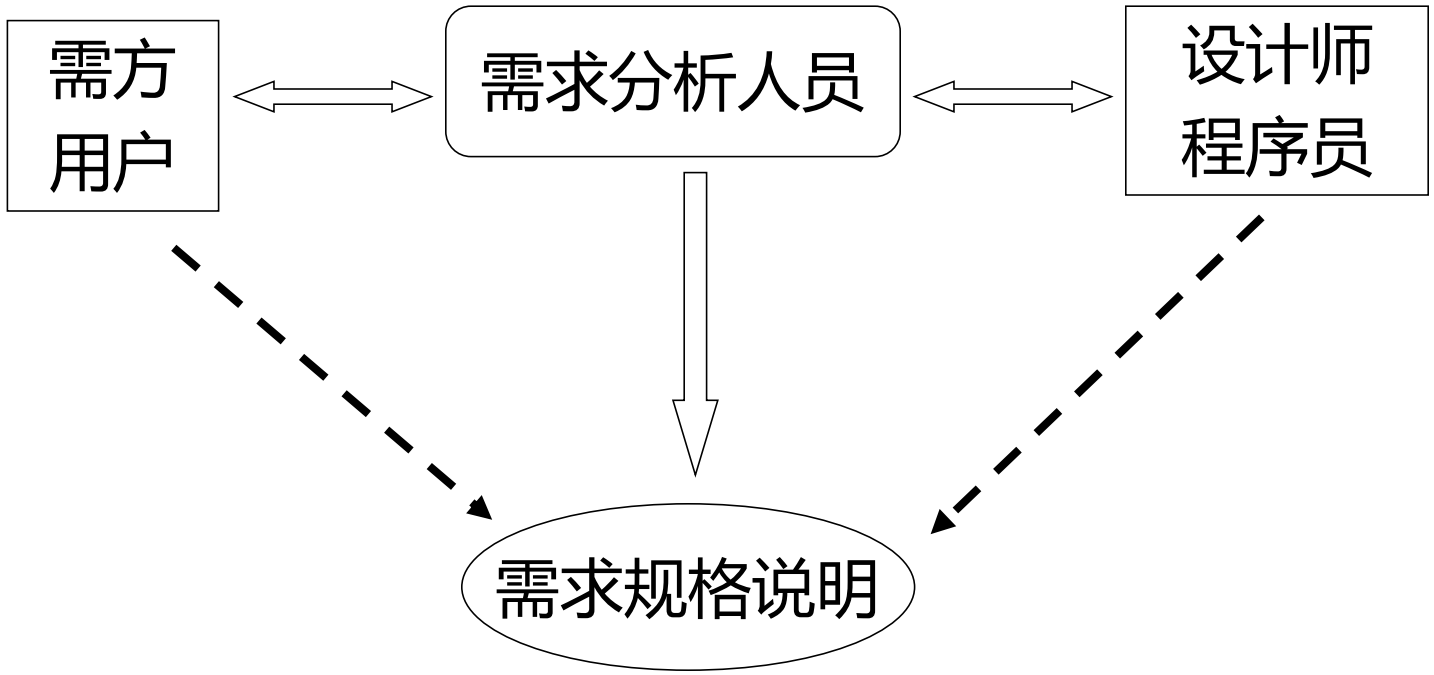
4）适宜于演进，

5）有工具支持，



软件工程-软件需求分析与设计

- 需求工程所涉及的角色



软件工程-软件需求分析与设计

- 需求工程过程

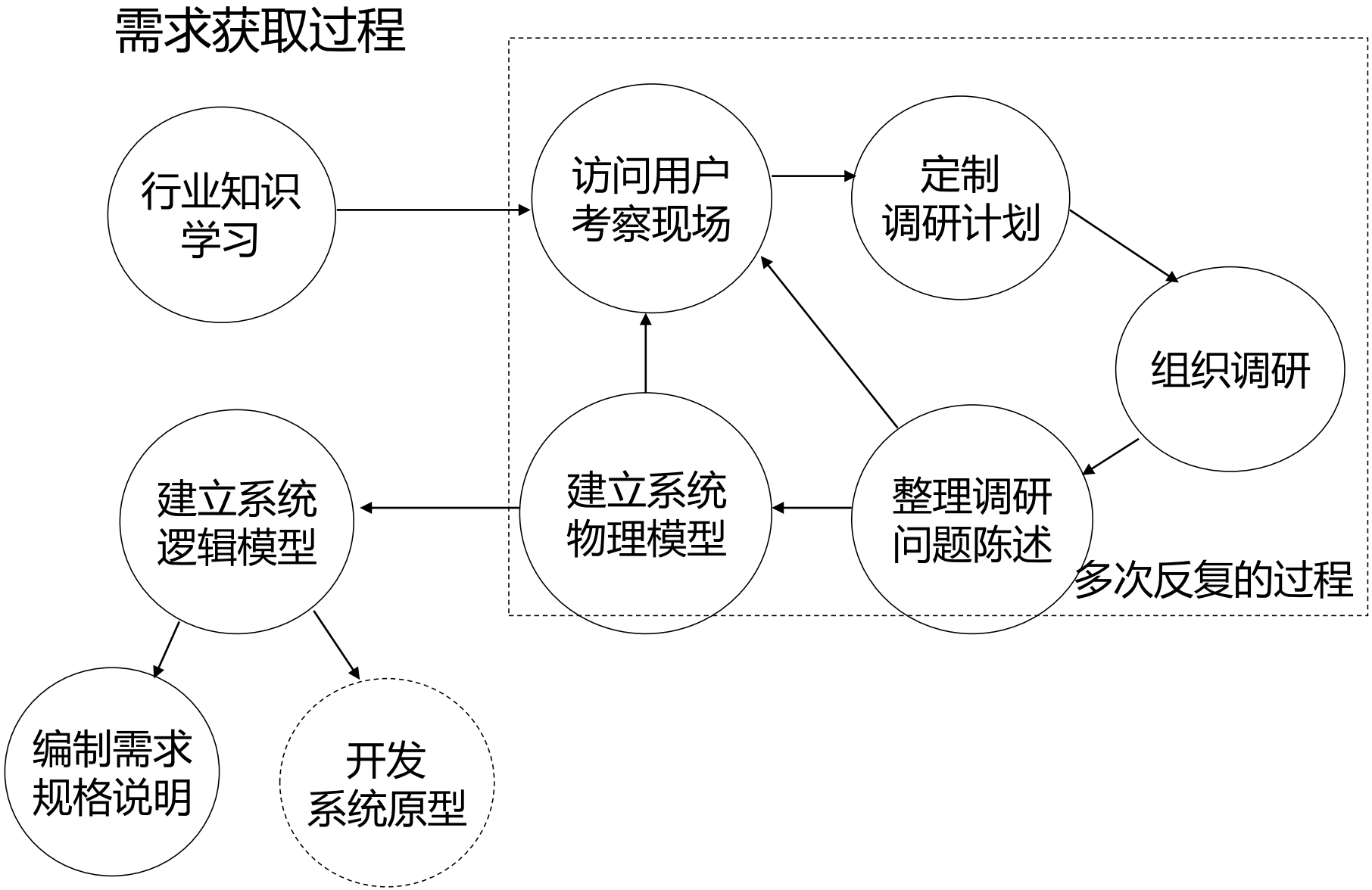
  包括：需求获取

  需求分析

  编写软件需求规格说明（SRS）

  需求验证

软件工程-软件需求分析与设计



软件工程-软件需求分析与设计

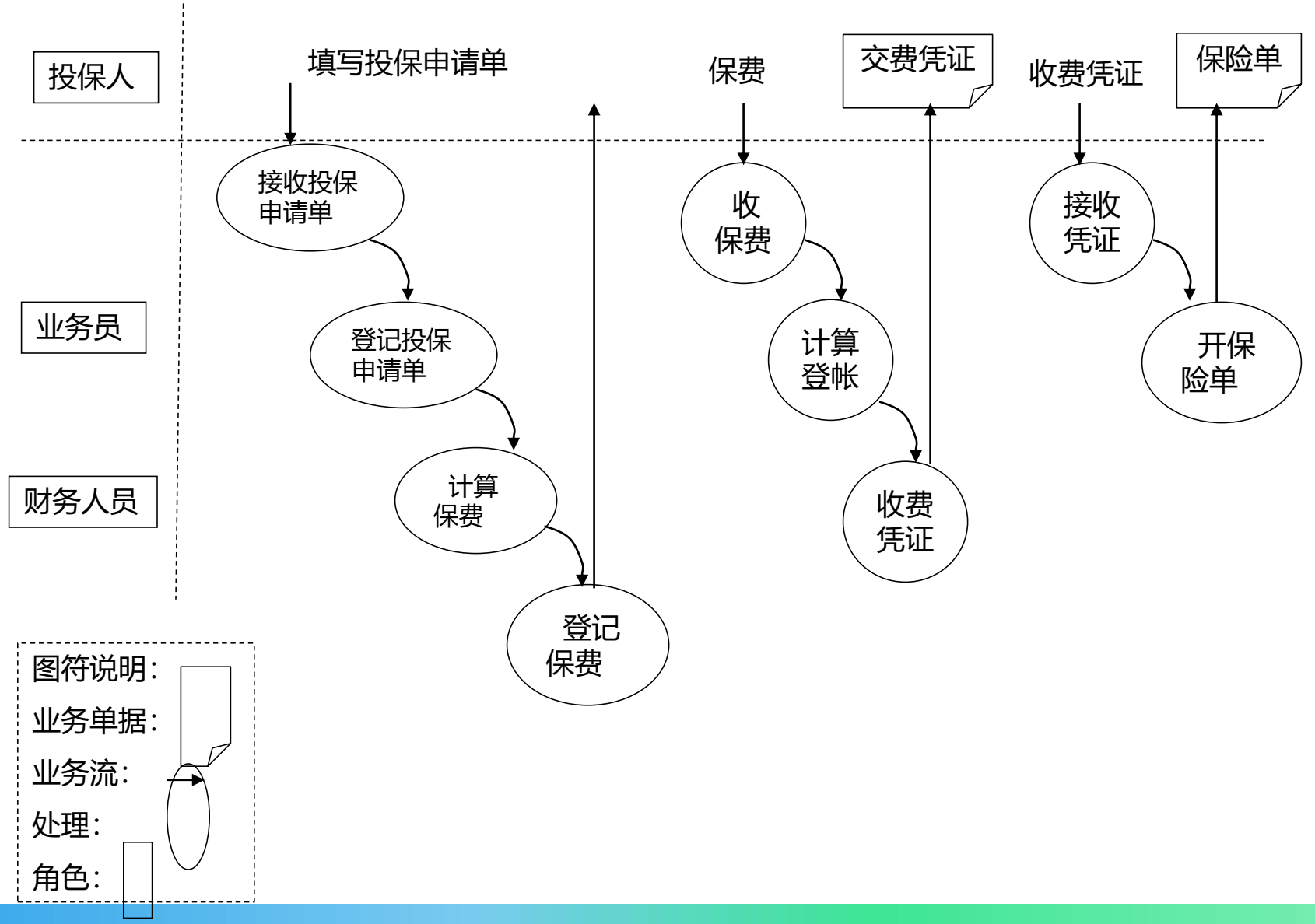
物理模型导出逻辑模型

- 业务陈述：

保险公司柜台业务，保险业务人员受理投保人的投保申请（包括：保险人，保险受益人、保险种类、保险年限等信息）；业务人员需要录入申请，根据投保申请计算保费；投保人若同意计算得到的保费，业务人员则将保费登记在投保申请单上；投保人持投保申请到交费处办理交费业务；待交保费业务完成，需持交费票据，再由业务员办理保险单。保险单将作为保险公司与投保人理赔时的法律合同依据。

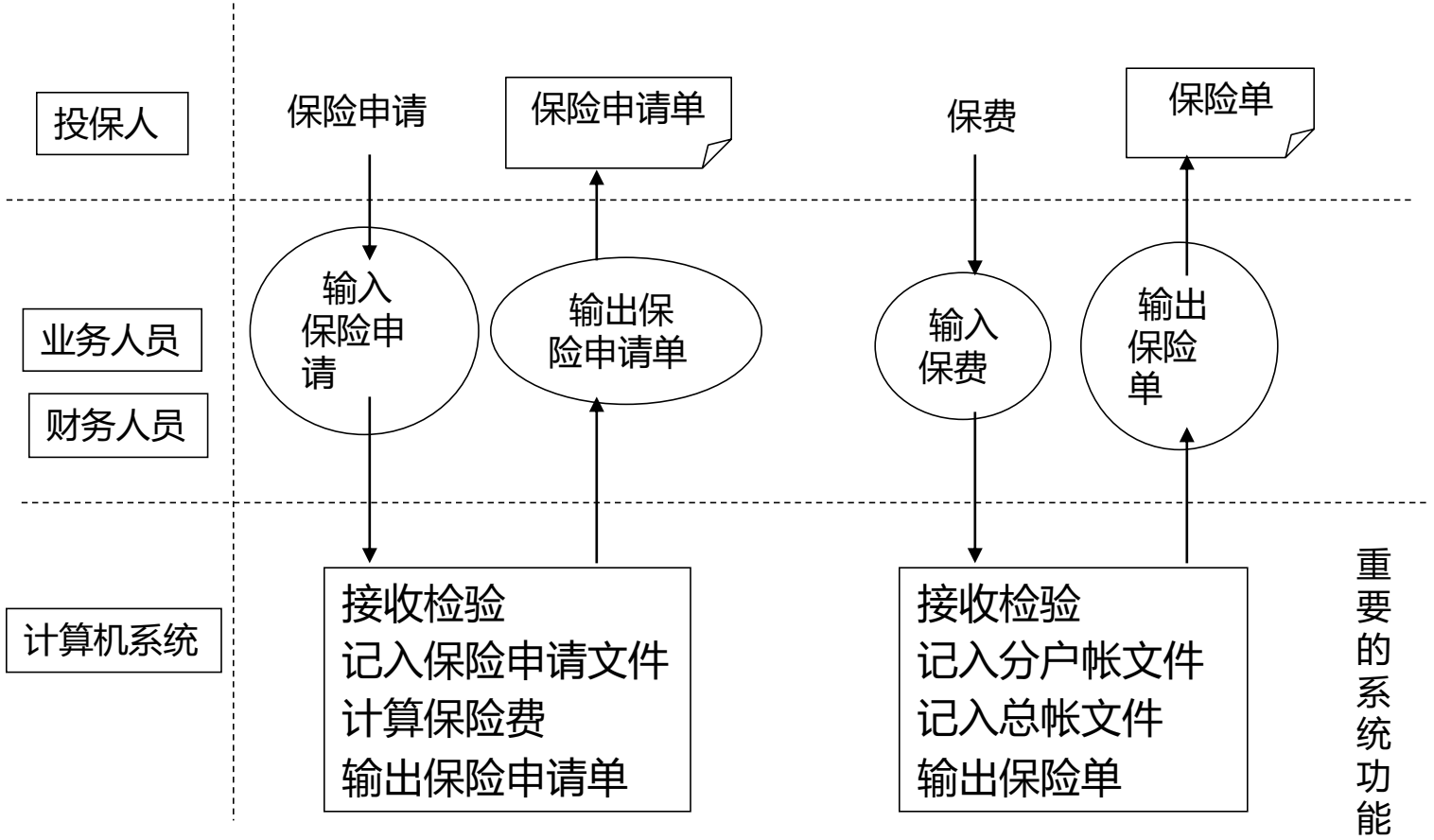
软件工程-软件需求分析与设计

保险公司柜台业务过程物理模型：



软件工程-软件需求分析与设计

分析保险公司柜台业务逻辑过程，得到系统的主要功能：



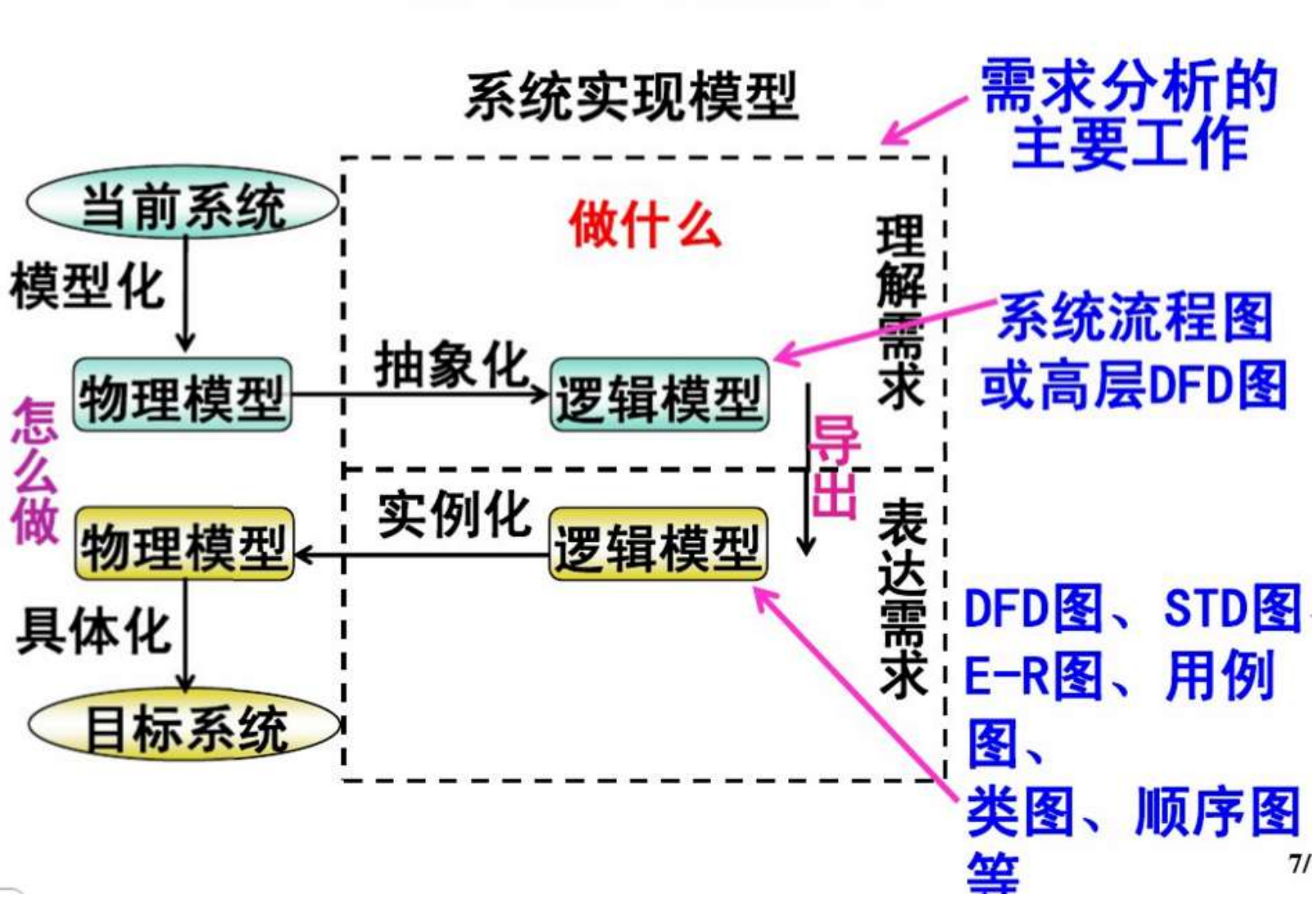
逻辑模型

可根据系统的主要功能画出系统逻辑模型。

软件工程-软件需求分析与设计

- 完成需求
  “软件需
  主要的两
  - 结构化
  - 面向对

面向对象
的重点。



编写需求规格说明

IEEE97年提出的需求规格说明的8条原则：

1）从实现中分离功能，即描述“做什么”而不是“怎么做”；

2）有面向处理的规格说明，以描述系统级的动态行为；

3）以软件为元素，描述系统中各元素的关系；

4）对系统的运行环境说明，以保持系统接口描述的一致；

5）以用户能接受和理解的形式描述；

6）规格说明必须可操作；

7）可容忍不完备性和可修改性；

8）说明应局部化和松散耦合，以适应信息变化的易修改性。

需求验证

评审需求规格说明的质量一般具有如下性质：

1）正确性
2）无二义性
3）完整性
4）可验证性
5）一致性
6）非计算机人员理解性
7）可修改性
8）可跟踪性

软件工程-软件需求分析与设计

- 原型是制造业普遍使用的产品开发方法；
- 原型能表现出目标产品的功能和行为特性；
- 软件原型与一般产品的原型区别：

1）对于整个开发过程而言，相对时间短；

2）对于整体软件费用而言，相对费用低；

3）一个原型只对应软件产品的一个版本，而不是一个原型下对应多个型号。

软件工程-软件需求分析与设计

●软件原型的类型

1）全功能展示

2）人机交互模拟

3）程序实现框架

4）探索性开发技术研究

5）作为开发演进的基础

●软件原型方法的过程

1）功能选择

包括最终产品的部分功能，为探索系统需求和实现技术。

包括可选择的多个方案，为通过原型演示，进行方案比较。

2）构造

着眼于评估，而不是长期使用。

性能问题先不与考虑（不包括主要为解决性能问题的原型）。

3）评估

评估是原型化方法中决定性的工作；

评估要有人力物力的保证；

评估一定有用户的参与；

4）使用

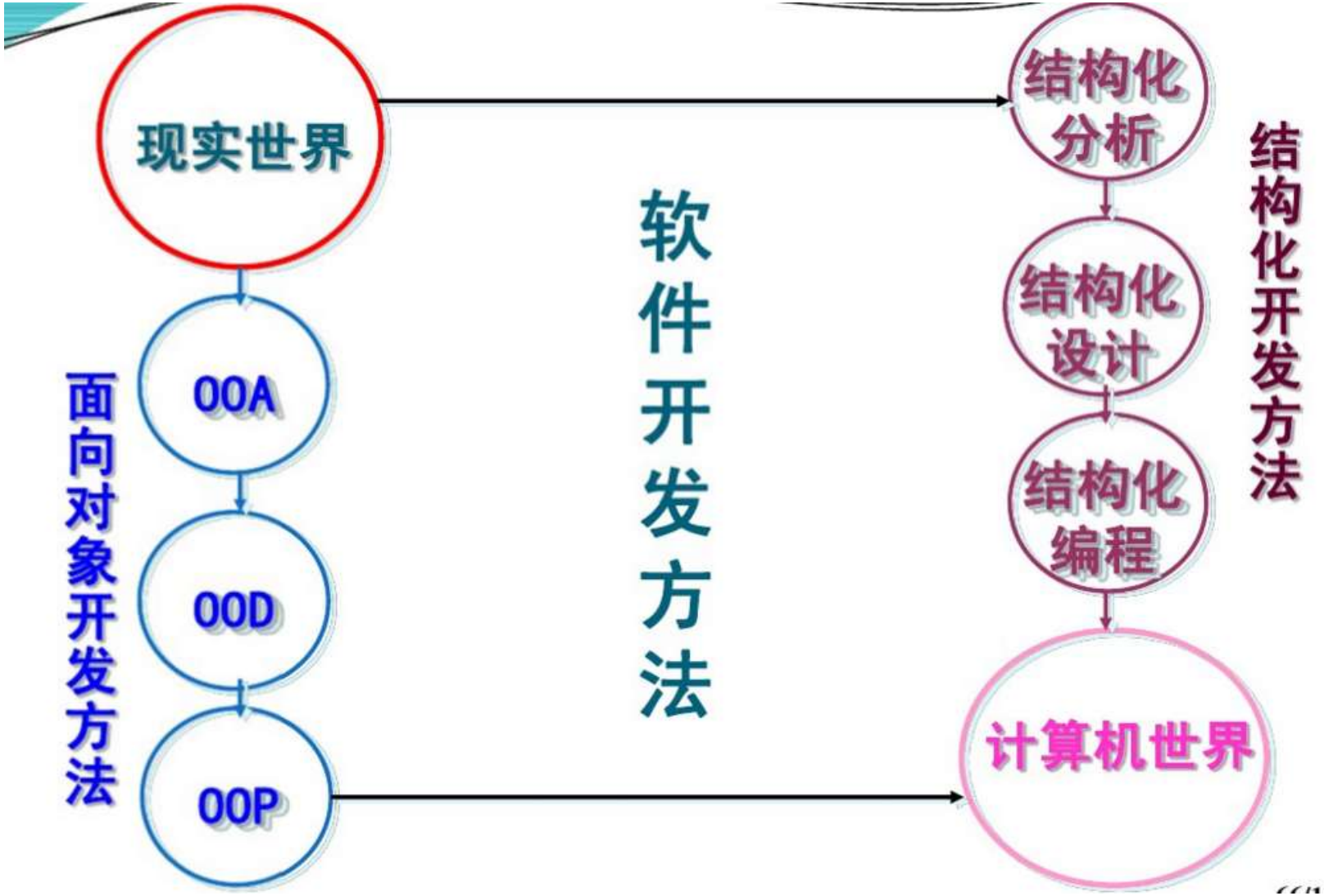
几种对原型可能的使用：

用于未来系统的某一部分；

用于后期开发实现技术的使用；

用于未来系统的熟悉环境性培训；完全不使用；

软件工程-软件需求分析与设计



结构化分析与建模

(Structured Analysis)

软件工程-软件需求分析与设计

➢ 结构化方法的主要思想： (Structured Analysis)

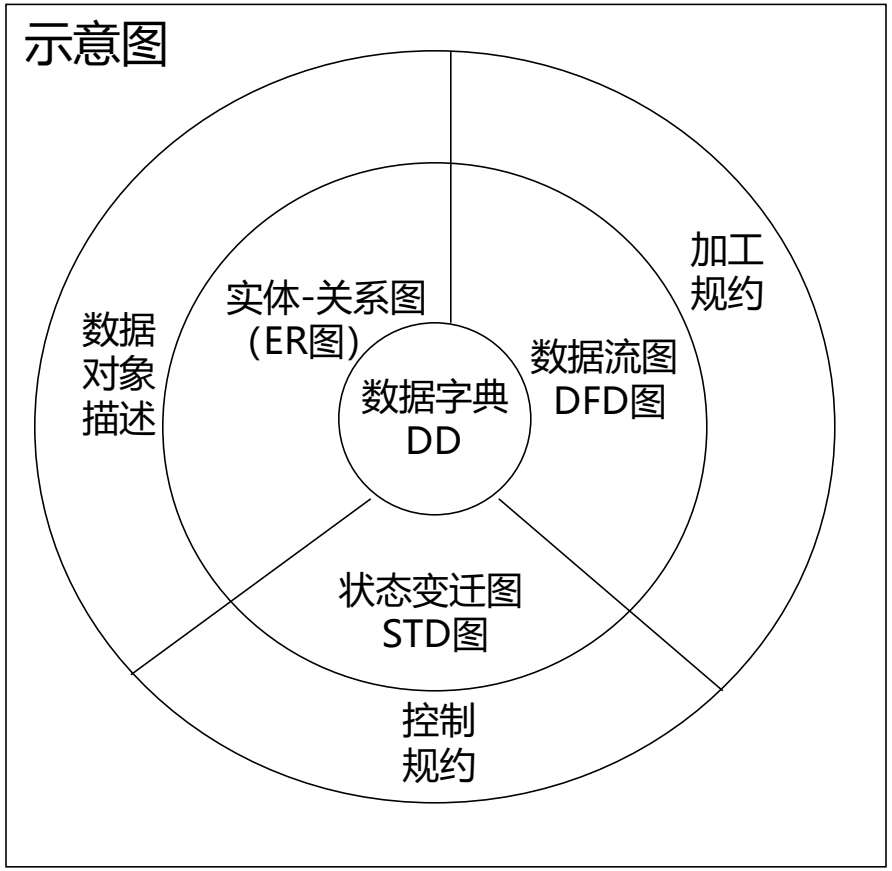
- SA是面向数据流的分析方法，在20世纪90年代广发被应用。最初由Douglas Ross提出，DeMareco推广，Ward和Mellor以及后来的Hatley和Pirbhai扩充。
- SA方法适
- 主要思想层分解。

- **结构化分析方法**是一种传统的系统建模技术，其过程是创建描述信息内容和数据流模型，依据功能和行为对系统进行划分，并描述过必须建立的系统要素。
- **结构化分析**：使用数据流程图、数据字典、结构化英语、判定表和判定树等工具，来建立一种新的、称为结构化说明书的目标文档－需求规格说明书。
- 结构化体现在将软件系统抽象为一系列的逻辑加工单元，各单元之间以**数据流**发生关联。
- 该方法的要点是：面对数据流的分解和抽象；把复杂问题自顶向下逐层分解

软件工程-软件需求分析与设计

➢结构化方法基本内容包括:

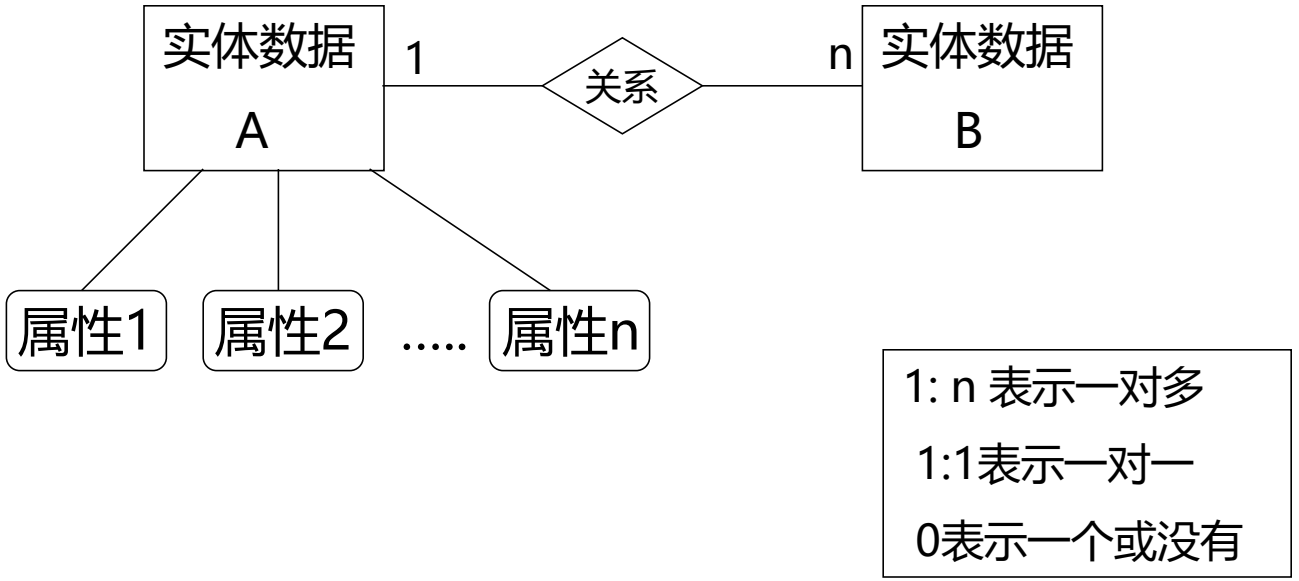
- 数据建模
- 功能建模
- 行为建模



软件工程-软件需求分析与设计

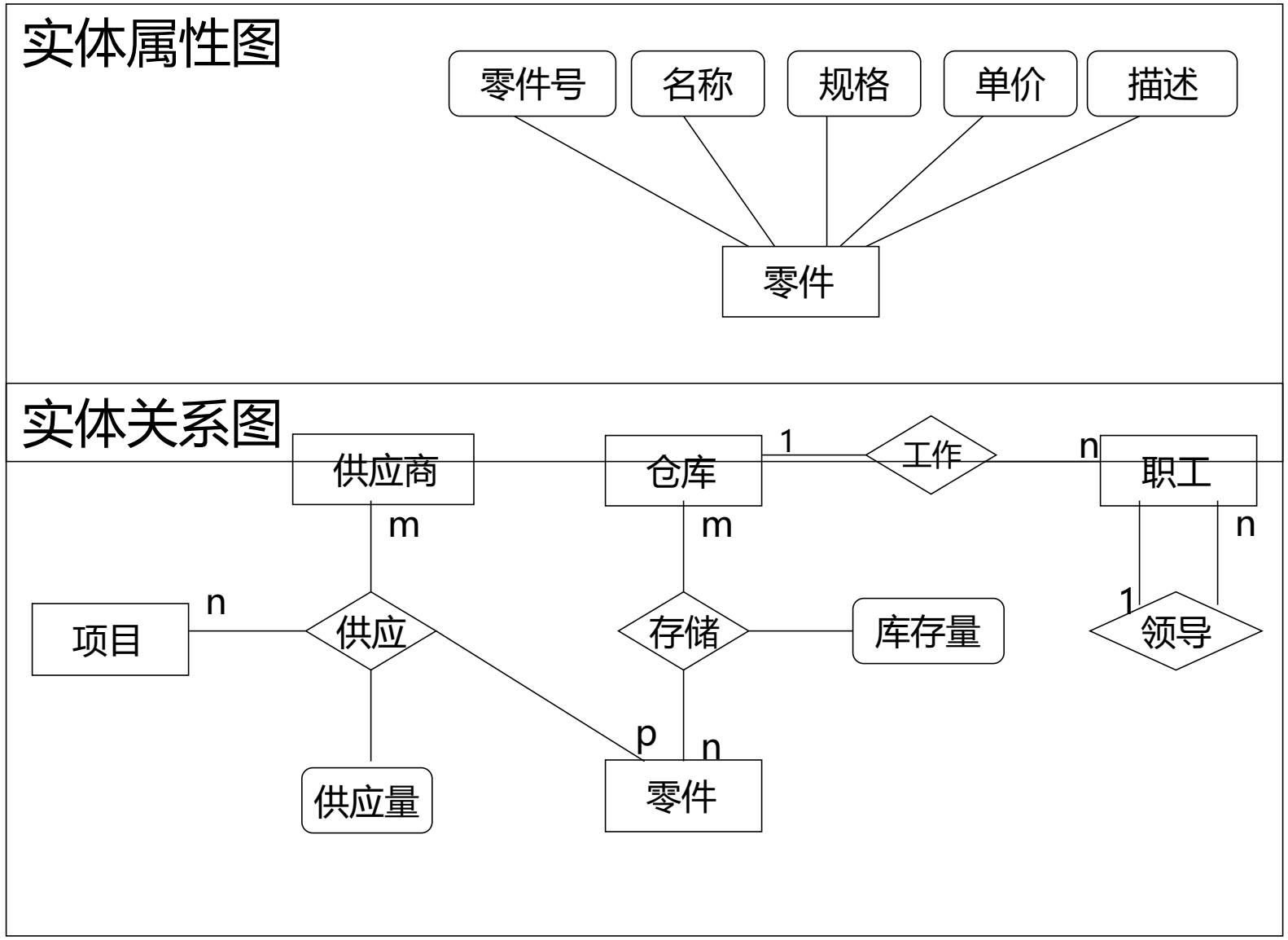
数据建模E-R图

- 实体 –关系图 （Entity Relation Diagram）
- 实体：系统需要记录及处理的数据集（DB中的表）
- 关系：实体间在系统中需要的必然联系（DB中的主键和外键）



软件工程-软件需求分析与设计

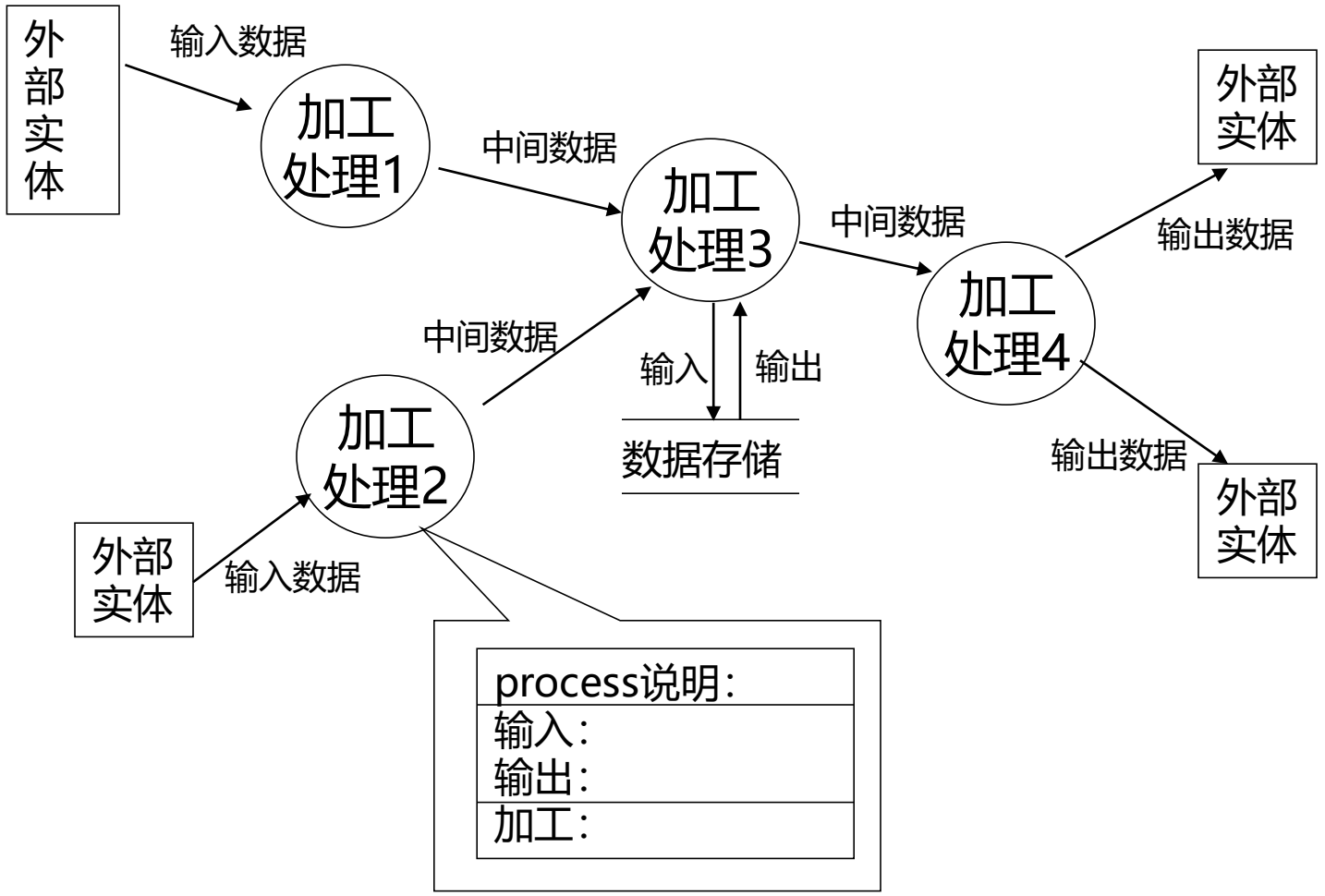
实体 –关系图示例



软件工程-软件需求分析与设计

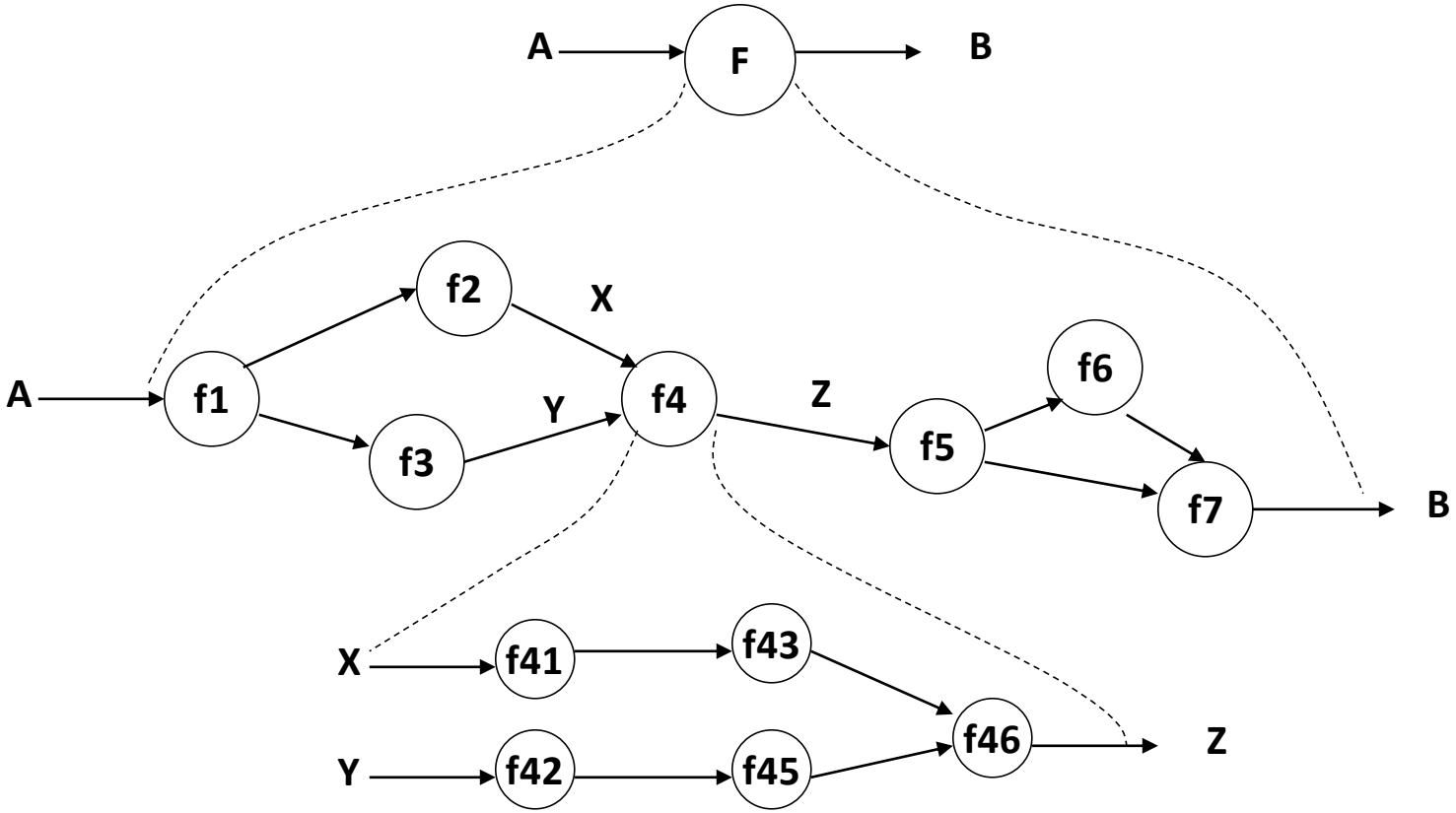
功能建模

数据流图(Data Flow Diagram DFD)



软件工程-软件需求分析与设计

➢数据流结构化



软件工程-软件需求分析与设计

➢数据字典

- 作用：对数据流图中的数据流、文件、数据项和加工的约束性描述
- 数据流定义：列出数据流组成的数据项

数据项集合={数据项1+数据项2+数据项3...}即：文件

数据项=数据子项1=数据子项2+数据子项3...

数据子项=[范围]|[限值]|[类型]|[精度]...

- 加工说明：

简单处理逻辑、激发条件、优先级、执行频率、最大数据量、峰值等

面向对象分析与建模

软件工程-软件需求分析与设计

➢思想方法

- 从现实世界中客观存在的事物出发，直接以问题域中的事物为中心，思考和认识问题。
- 将事物的本质特征和系统责任，抽象表示为系统的对象，作为系统的基本构成单位，建立软件系统。
- 强调运用日常逻辑思维经常采用的思想方法和原则，例如：抽象、分类、继承、聚合、封装和关联，并以易懂方式表达。

类和对象的概念

- 类使用抽象的方式，抽象是人们分析问题常用的方法，抽象即忽略事物非本质的东西，而突出事物本质的特征和行为；
- 在外部世界中，相同属性和操作的对象属于一个类，对象是类的一个实例；在机器空间中，类是一个可复用的模板，对象是复用出来的独立可执行程序块；
- 抽象包括数据抽象和过程抽象，数据抽象是根据施加在数据上的操作来定义的数据类型；过程抽象是将确定的处理功能定义为独立的过程实体；
- 对象包括实体对象和无形对象；

封装和消息的概念

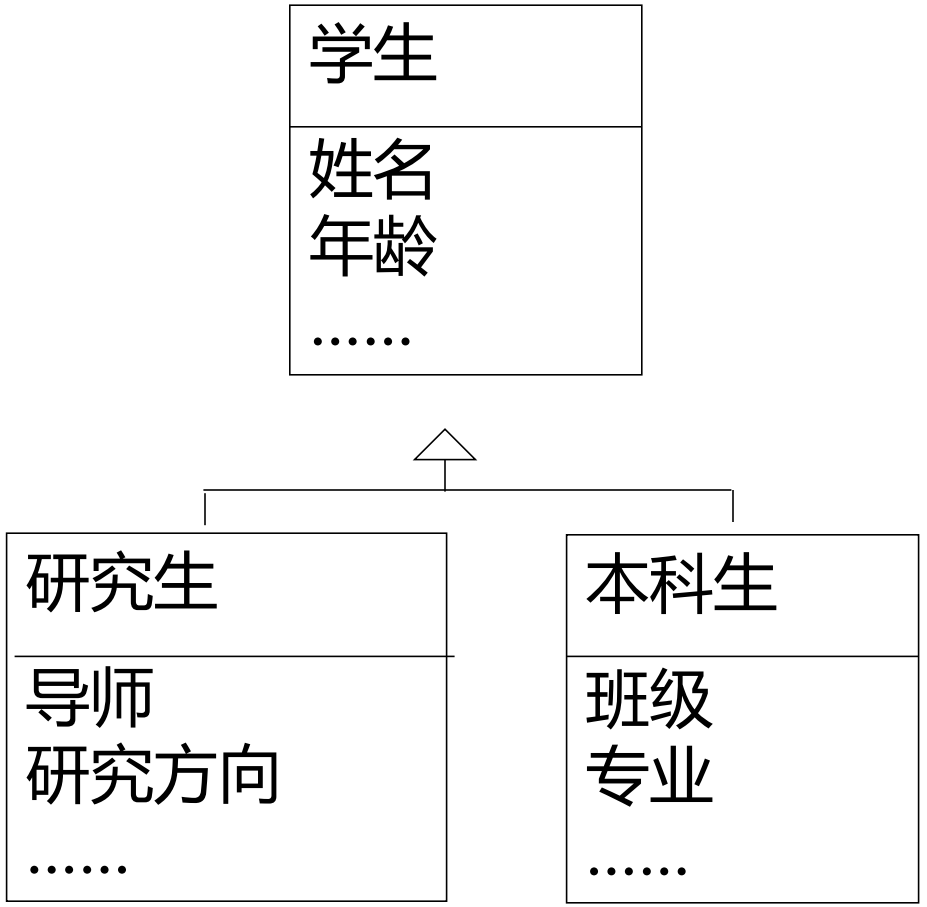
- 把类的内部属性和一些操作隐藏起来，只将公共的操作对外可见。避免外界错误和内部修改带来的影响；
- 对象只通过消息来请求其他的对象执行自身的操作；
- 消息必须直接发给指定的对象，消息中包括请求执行操作的必要信息；
- 一个对象是消息的接收执行者，也可以是消息的请求发送者。

继承的概念

- 类可以有子类，子类继承父类的全部属性和操作，并允许添加自己的属性和操作；
- 继承是泛化和特殊的关系,父类具有某事物的一般共性,子类描述的事物在父类的基础上并比父类更特殊；
- 继承可以有多重性，允许多于一个父类；
- 继承允许多层，各层之间具有传递性；
- 继承的重要作用在于源代码的复用。

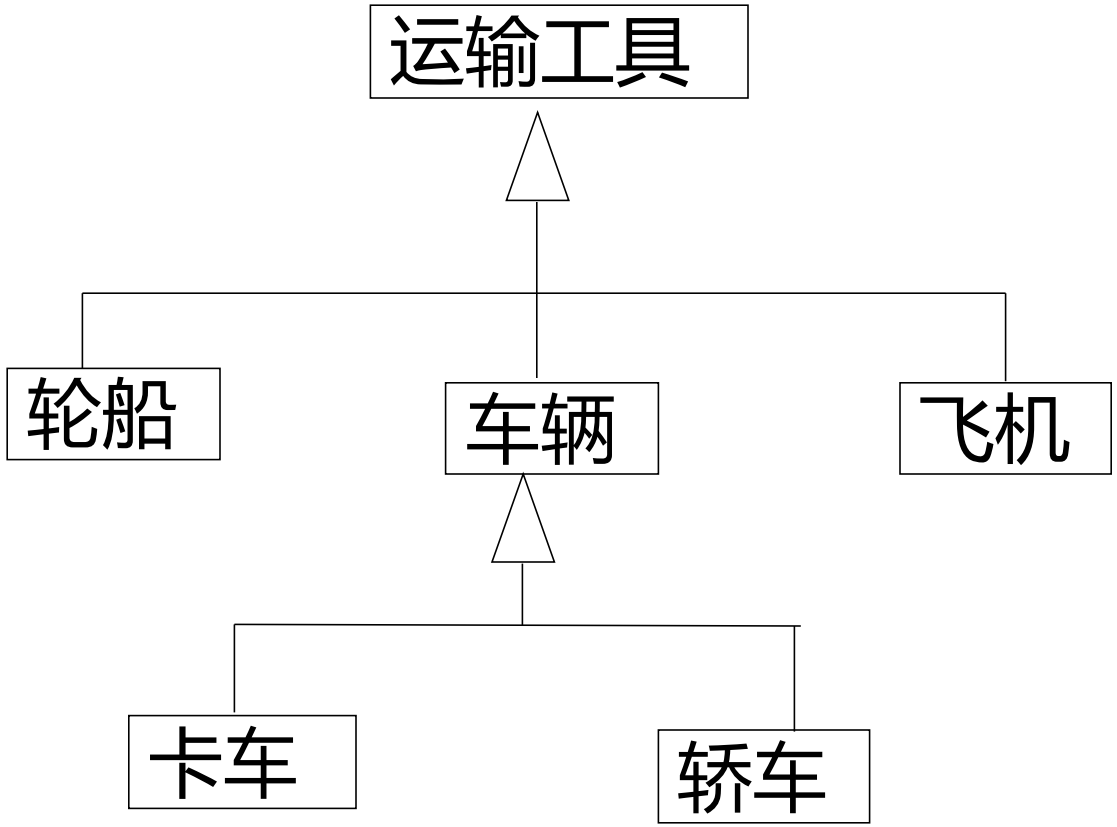
软件工程-软件需求分析与设计

继承示例



软件工程-软件需求分析与设计

多层继承示例



➢多态性

- 在继承类结构中允许定义同名操作，同一个消息的响应可以执行不同的行为，即同一操作的多种形态。
- 多态性更好地体现了操作语义的一致性，实现接口封装独立性和信息隐蔽的原则。

软件工程-软件需求分析与设计



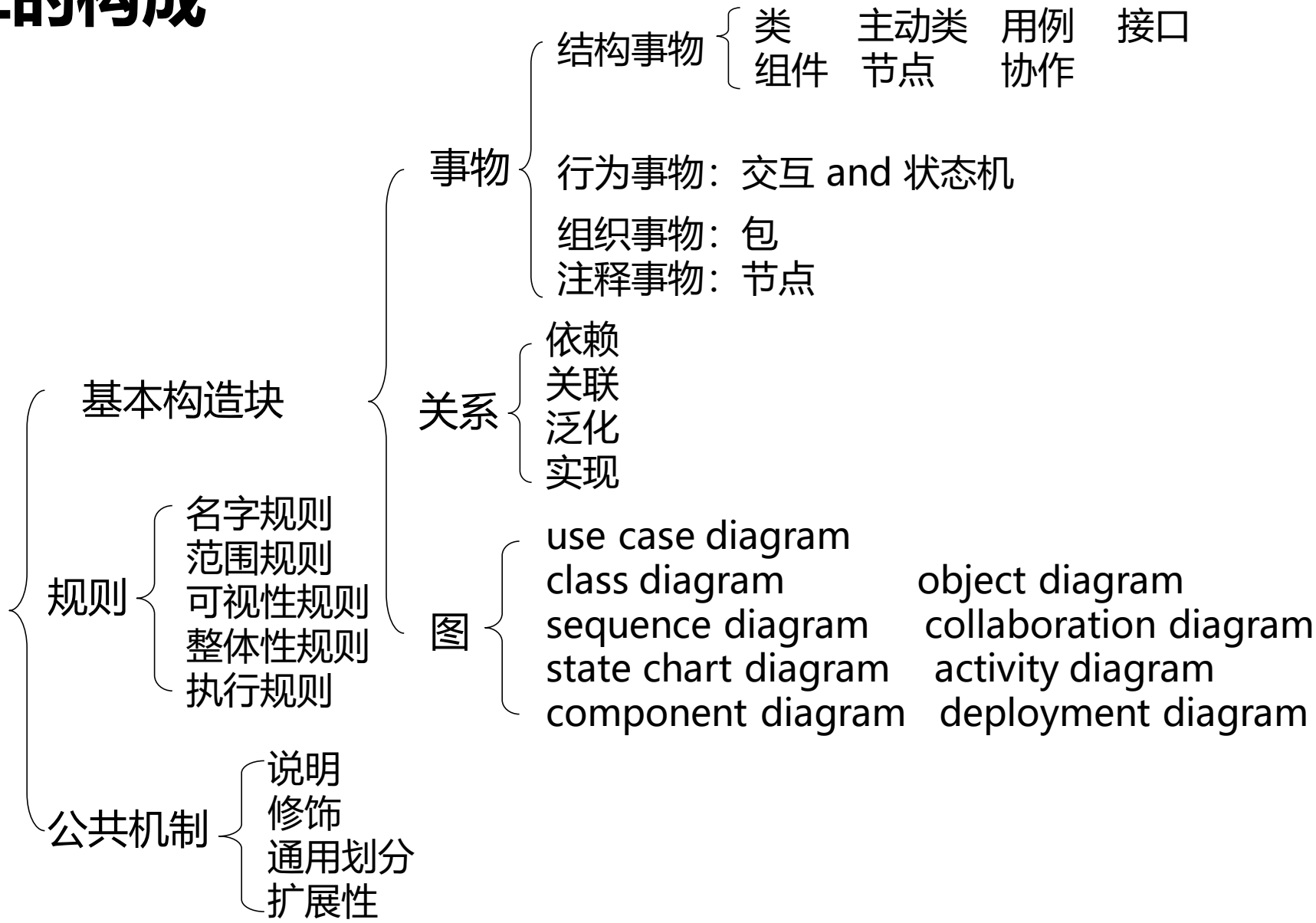
统一建模语言UML

- **UML（Unified Modeling Language）**定义：

- UML是对软件密集型系统中的制品进行可视化、详述、构造和文档化的语言。[Booch，The Unified Modeling Language User Guide]

其中：制品（artifact）是指软件开发过程中产生的各种产物，如：模型、测试用例、源代码等等。

UML的构成


基本构造块
事物
结构事物
类
主动类
用例
接口
组件
节点
协作
行为事物：交互 and 状态机
组织事物：包
注释事物：节点
关系
依赖
关联
泛化
实现
图
use case diagram
class diagram
object diagram
sequence diagram
collaboration diagram
state chart diagram
activity diagram
component diagram
deployment diagram
规则
名字规则
范围规则
可视性规则
整体性规则
执行规则
公共机制
说明
修饰
通用划分
扩展性

UML模型

- 目标：用抽象表示的图表和文字，简化现实问题描述以理解所建造的软件系统 。

需求模型 use case diagram	基本模型 class diagram object diagram	行为模型 Sequence diagram Collaboration diagram state chart diagram
工作流模型 Activity diagram	组织模型 Packet diagram	实现模型 Component diagram Deployment diagram

软件工程-软件需求分析与设计

- **用例图（use Case diagram ）**

描述使用系统功能的角色和系统相关的功能，是需求建模的重要工具。

- **活动图（activity diagram ）**

用来描述任务流程或算法过程，可用来分析系统并发事务流程。

注意：

用例图和活动图的建立，并不包含面向对象思想。

软件工程-软件需求分析与设计

● 类图（class diagram ）

类图是系统模型的基础，描述系统的静态结构。

- 对象层：描述系统实体以及承载的系统责任
- 特征层：描述实体抽象的特征
- 关系层：实体类的固有关系

● 对象图（object diagram ）

描述类图的实例关系，为表示系统特定时刻的对象情况。注：在UML中使用较少

● 包图（package diagram ）

描述系统的组织模型，为控制表示的复杂性。

软件工程-软件需求分析与设计

- **时序图（sequence diagram ）**

描述按时间顺序排列的对象交互。

- **协作图（collaboration diagram ）**

表示交互对象的行为组织结构。

- **状态转换图（state chart diagram ）**

描述对象在生命周期内，响应事件的状态转换过程，以及响应事件后所做的反应。

软件工程-软件需求分析与设计

- **部署图（deployment diagram ）**

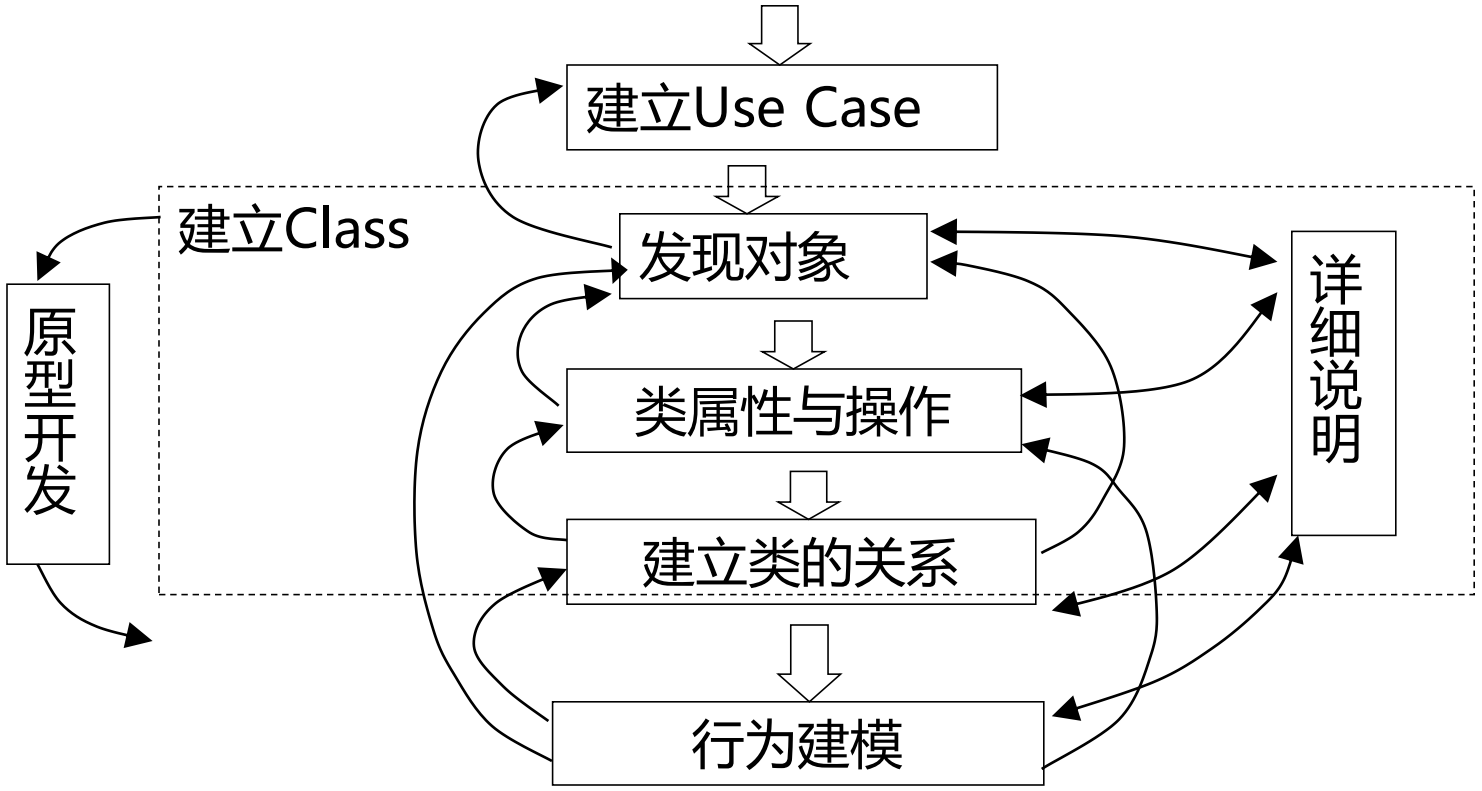
用来描述系统中计算结点的拓扑结构，一个系统只有一个部署图，可用来分析分布式系统。

- **构件图（component diagram ）**

描述一组构件以及相互间的关系，是系统实现的物理建模。

UML建模过程

- 面向对象的建模过程，目前没有统一标准，各种建模方法的建模过程存在着较大差别，但基本原则是统一的。强调过程中各个步骤的相对独立，不要求按固定顺序。



软件工程-软件需求分析与设计

建模过程的建议

1）首先进行需求确定；

2）建立类图的过程可随时切换到其他活动；

3）行为建模与类图交错进行；

4）详细说明分散在各活动中；

5）对详细说明要审查和补充；

5）原型可以反复地进行；

6）规模较小系统可以省略包图；

软件工程-软件需求分析与设计

●UML事物

- UML包含4种事物：构件事物 行为事物 分组事物 注释事物；
- 构件事物： UML模型的静态部分，描述概念或物理元素
- 包括以下几种：

类：具有相同属性相同操作 相同关系相同语义的对象的描述

接口：描述元素的外部可见行为，即服务集合的定义说明

协作：描述了一组事物间的相互作用的集合

用例：代表一个系统或系统的一部分行为，是一组动作序列的集合

构件：系统中物理存在，可替换的部件

节点：运行时存在的物理元素

- 另外，参与者、信号应用、文档库、页表等都是上述基本事物的变体。

软件工程-软件需求分析与设计

●UML事物

- 行为事物：UML模型图的动态部分，描述跨越空间和时间的行为。

  交互：实现某功能的一组构件事物之间的消息的集合，涉及消息、动作序列、链接。

  状态机：描述事物或交互在生命周期内响应事件所经历的状态序列。

- 分组事物： UML模型的组织部分，描述事物的组织结构。

  包：把元素组织成组的机制。

- 注释事物： UML模型的解释部分，用来对模型中的元素进行说明，解释。

  注解：对元素进行约束或解释的简单符号。

软件工程-软件需求分析与设计

●UML关系

- 依赖。　依赖(dependency)是两个事物之间的语义关系，其中一个事物(独立事物)发生变化，会影响到另一个事物(依赖事物)的语义。如人使用计算机。
- 关联。　关联(association)是一种结构关系，它指明一个事物的对象与另一个事物的对象间的联系。
- 泛化。　泛化(generalization)是一种特殊/一般的关系。也可以看作是常说的继承关系。
- 实现。　实现(realization)是类元之间的语义关系，其中的一个类元指定了由另一个类元保证执行的契约。

软件工程-软件需求分析与设计

类	是对一组具有相同属性、相同操作、相同关系和相同语义的对象的描述	NewClass
对象		对象名：类名　：类名　类名
接口	是描述了一个类或构件的一个服务的操作集	Interface
协作	定义了一个交互，它是由一组共同工作以提供某种协作行为的角色和其他元素构成的一个群体	
用例	是对一组动作序列的描述	usecase
主动类	对象至少拥有一个进程或线程的类	class suspend() flush()
构件	是系统中物理的、可替代的部件	componet
参与者	在系统外部与系统直接交互的人或事物	actor

节点	是在运行时存在的物理元素	NewProcessor
交互	它由在特定语境中共同完成一定任务的一组对象间交换的消息组成	
状态机	它描述了一个对象或一个交互在生命期内响应事件所经历的状态序列	state
包	把元素组织成组的机制	NewPackage
注释事物	是UML模型的解释部分	
依赖	一条可能有方向的虚线	
关联	一条实线，可能有方向	
泛化	一条带有空心箭头的实线	
实现	一条带有空心箭头的虚线	

Use Case 用例图

需求建模

作用：

Use Case 用于对系统的功能及与系统进行交互的外部事物建模。

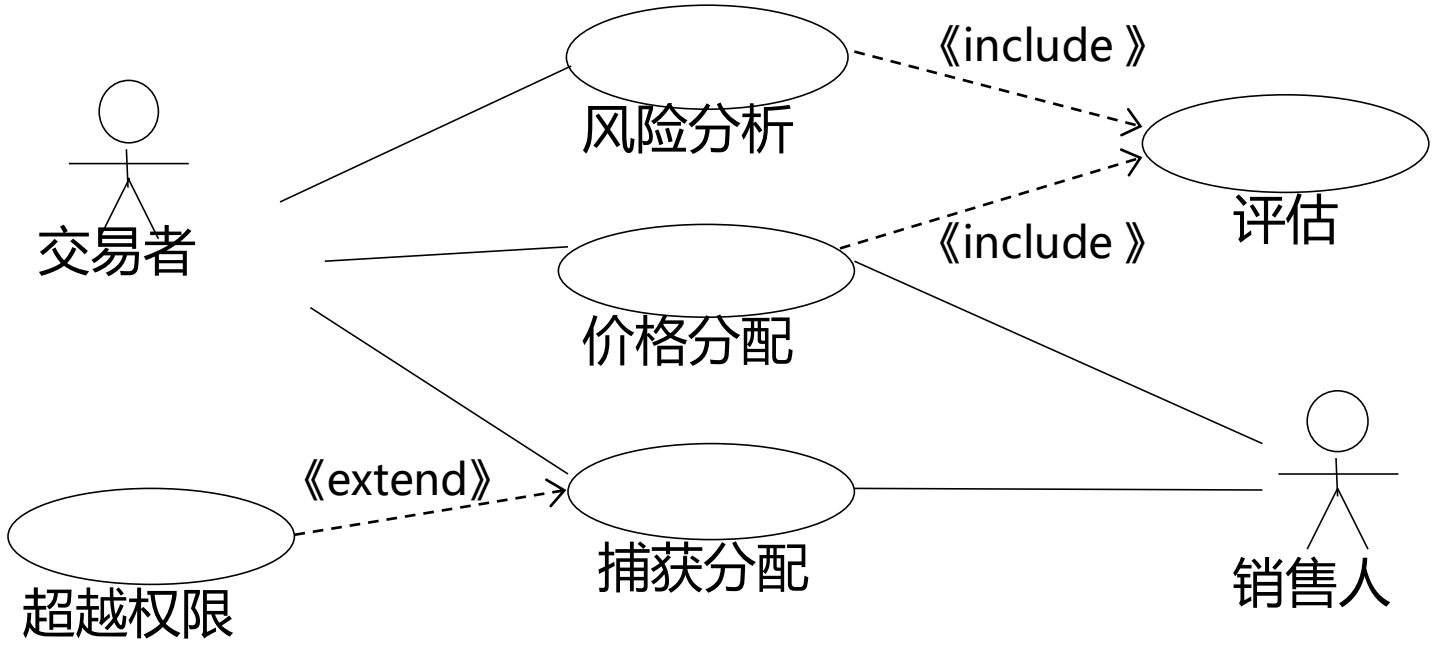
目的：

通过寻找与系统交互的外部事物，说明他们与系统如何交互，可以使用户和开发者，对系统的理解达成共识。

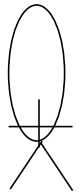
软件工程-软件需求分析与设计



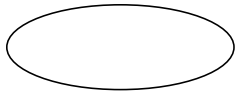
Use Case示例



use case 图元素



参与者：使用系统相关功能的角色



用例：与参与者有交互的一个功能

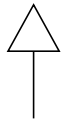
参与者与用例的关系：表示双向，没有箭头



用例之间的关系：表示包含其中必须的功能



用例之间的关系：表示扩展可选的功能



参与者之间或用例之间的关系：表示继承

- **目标**

从系统边界入手，对未知空间描述。

系统边界---系统与外界事物的分界；

未知空间---待建造的未来系统；

建立use case的切入点

切入点---与未来系统进行交互的的事物；

包括：人员、设备、系统，以及操作功能

- **用例（use case）**

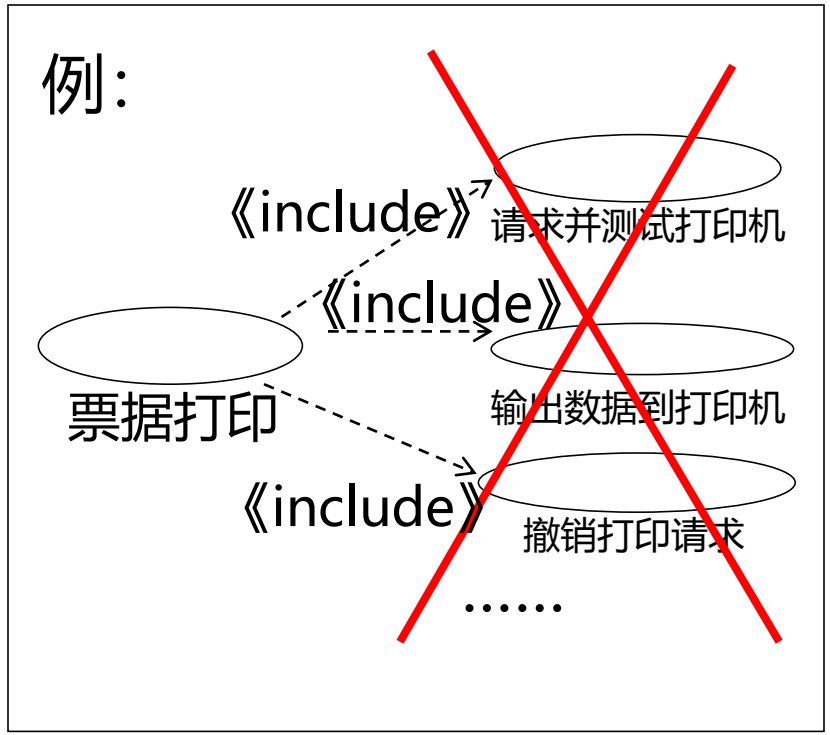
一个用例描述系统的一项功能，该项功能可被描述为参与者可视的一组操作，其中的每个操作表示参与者与系统的一个交互过程。

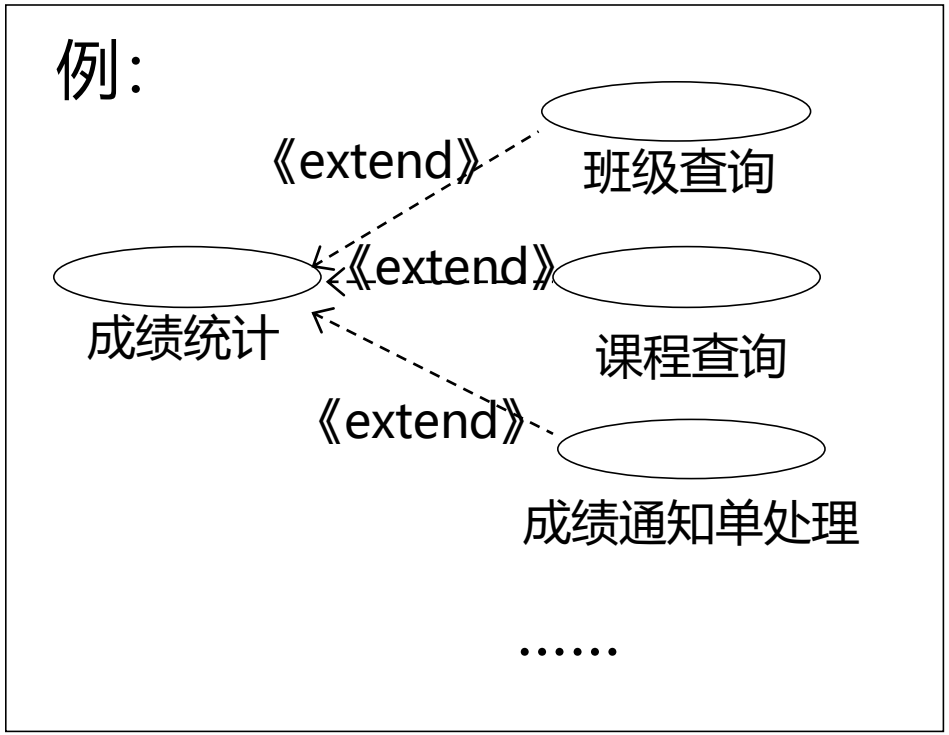
用例要点：

- 用例描述系统外部可见的功能需求；
- 只描述做什么，不描述怎么做；
- 多数是由参与者发起的动作；
- 也允许系统发起的动作，例如：异常情况处理；

软件工程-软件需求分析与设计

- 用例不描述功能实现的各项处理
- 用例仅描述外部可视的交互操作

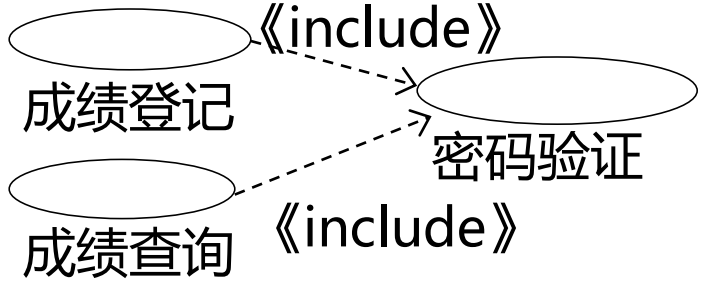




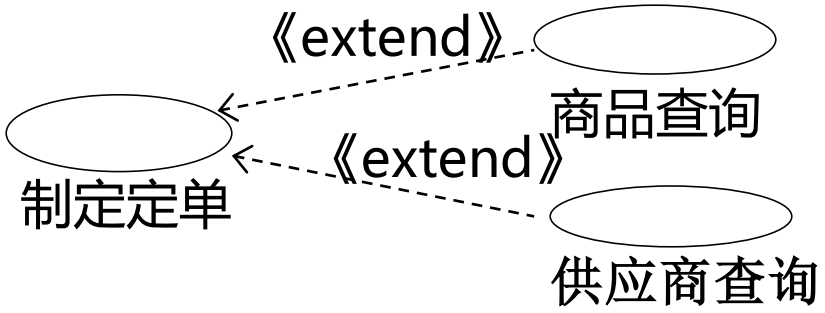
软件工程-软件需求分析与设计

- 用例关系
  - 包含关系《include》：描述用例间具有的公用行为
  - 扩展关系《extend》： 描述用例间可选的独立行为
  - 泛化关系 generalization： 用例之间的继承关系

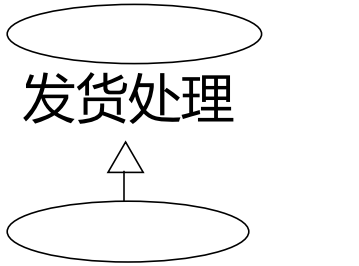
例：



例：



例：

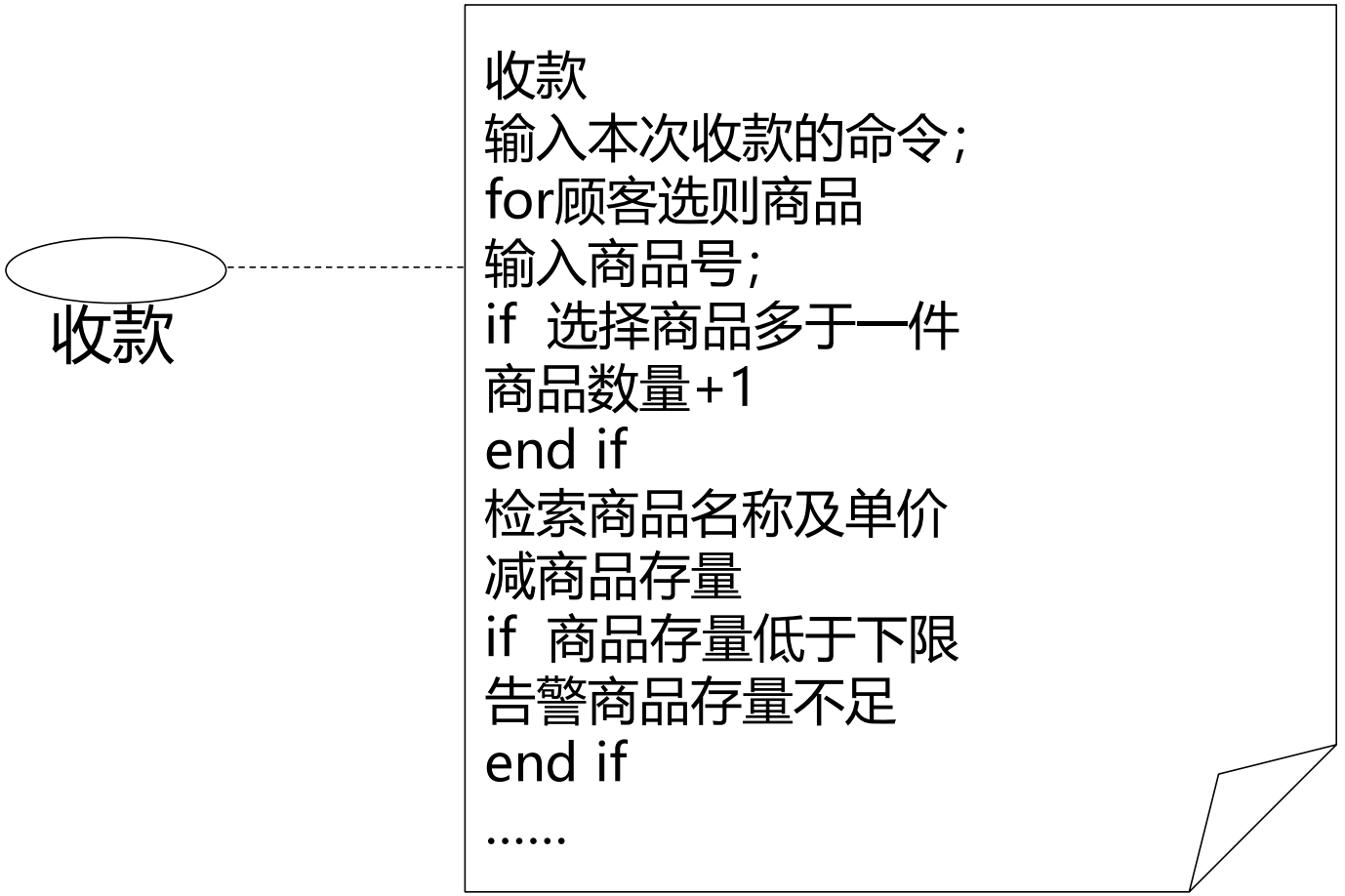


软件工程-软件需求分析与设计

用例说明

对有必要说明算法的用例，可以给出详细的说明。

例：



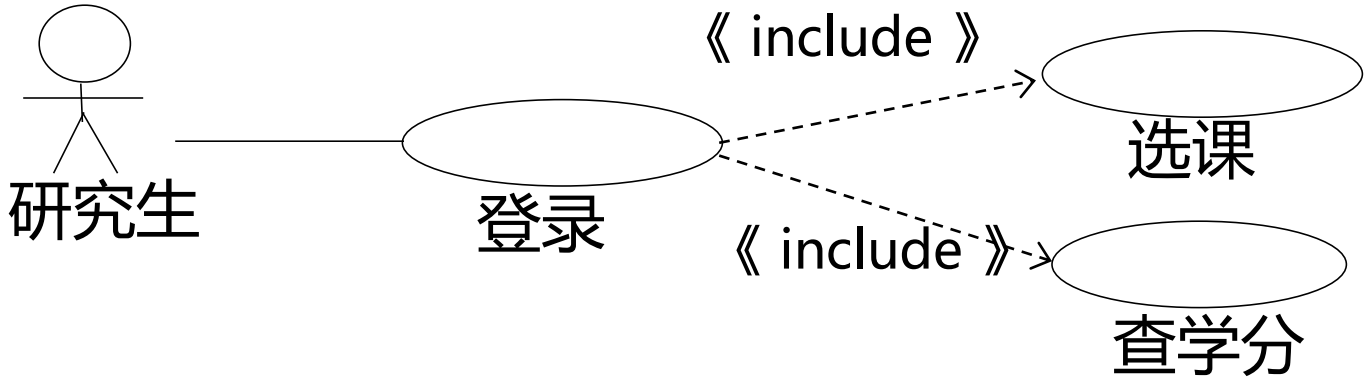
软件工程-软件需求分析与设计

- **用例表示举例：**

- 研究生教务系统：对登录、选课，以及查学分功能用例描述的四种表示，说明了四种不同的工作方式。

表示一：

说明研究生在登录后，有两个功能是被反复使用的。两个功能作为登录主程序从属的，并且都是必须要执行的功能。



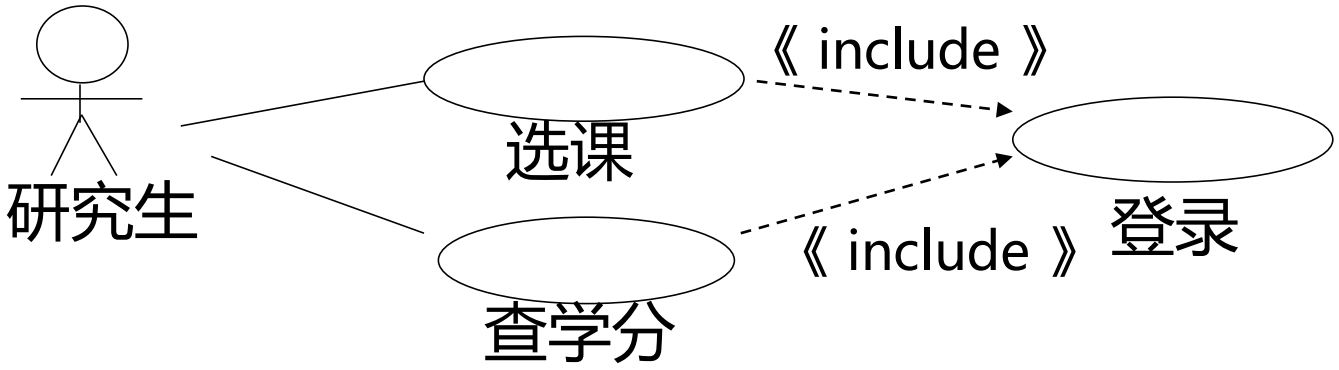
从处理逻辑上看有问题。

软件工程-软件需求分析与设计

表示二：

研究生可选择的两个功能，都包含有登录事务的需要。

每个功能的进入都必须登录。



当有这种严格的限定时，才需要这样的考虑。

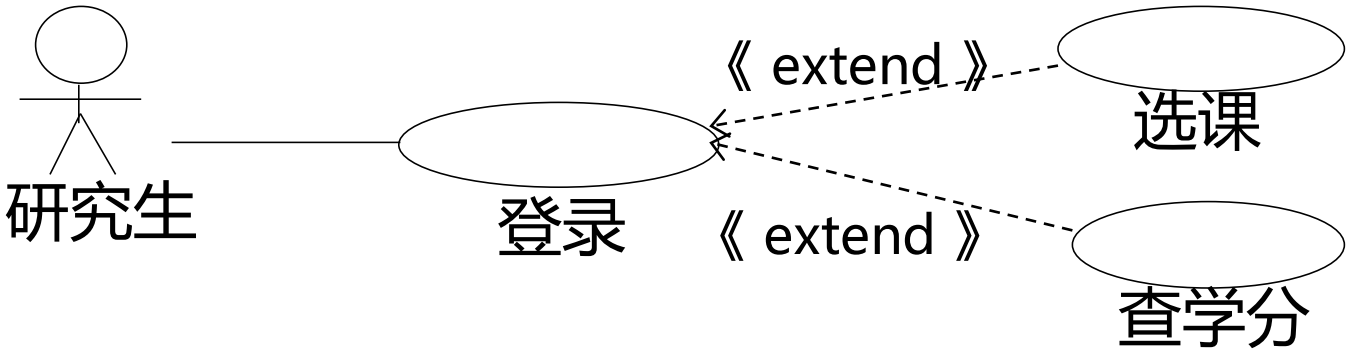
软件工程-软件需求分析与设计

表示三：

研究生在登录后，选择“选课”或“查学分”

在登录中根据必要的条件，选择是哪个功能。

条件：
if 选择“选课” then 选课
If 选择“查学分” then 查学分

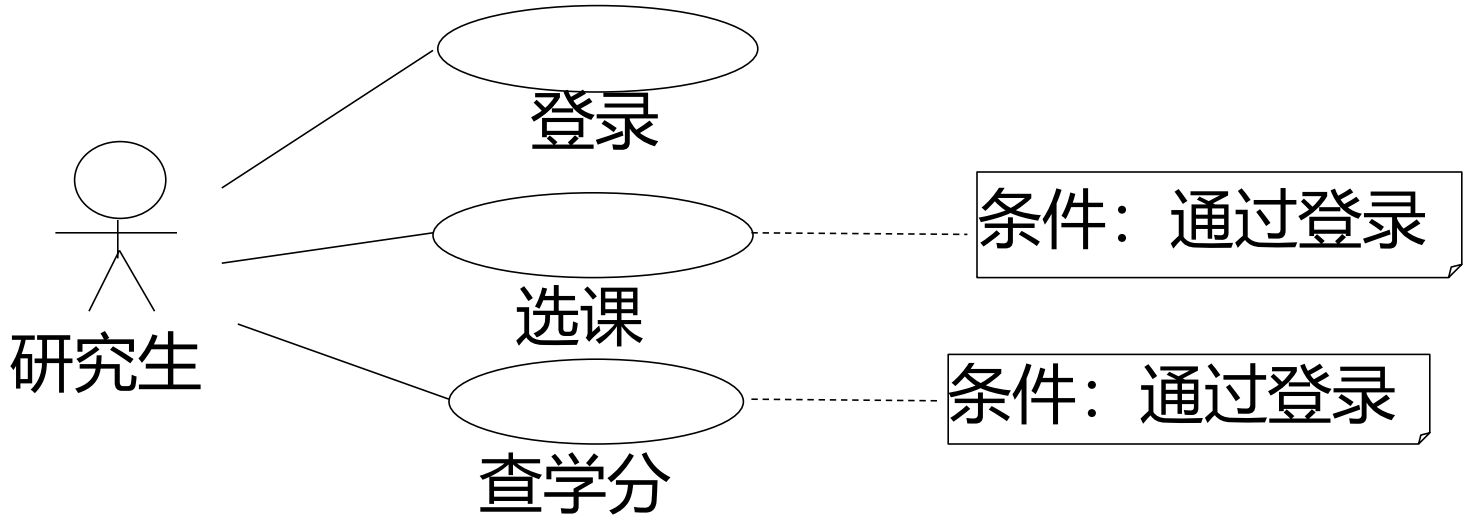


若增加功能，必须修改登录的条件和判断逻辑。

软件工程-软件需求分析与设计

表示四：

研究生有三个独立的功能，它们之间没有直接的关系，修改或添加功能，对其它的功能没有直接影响。



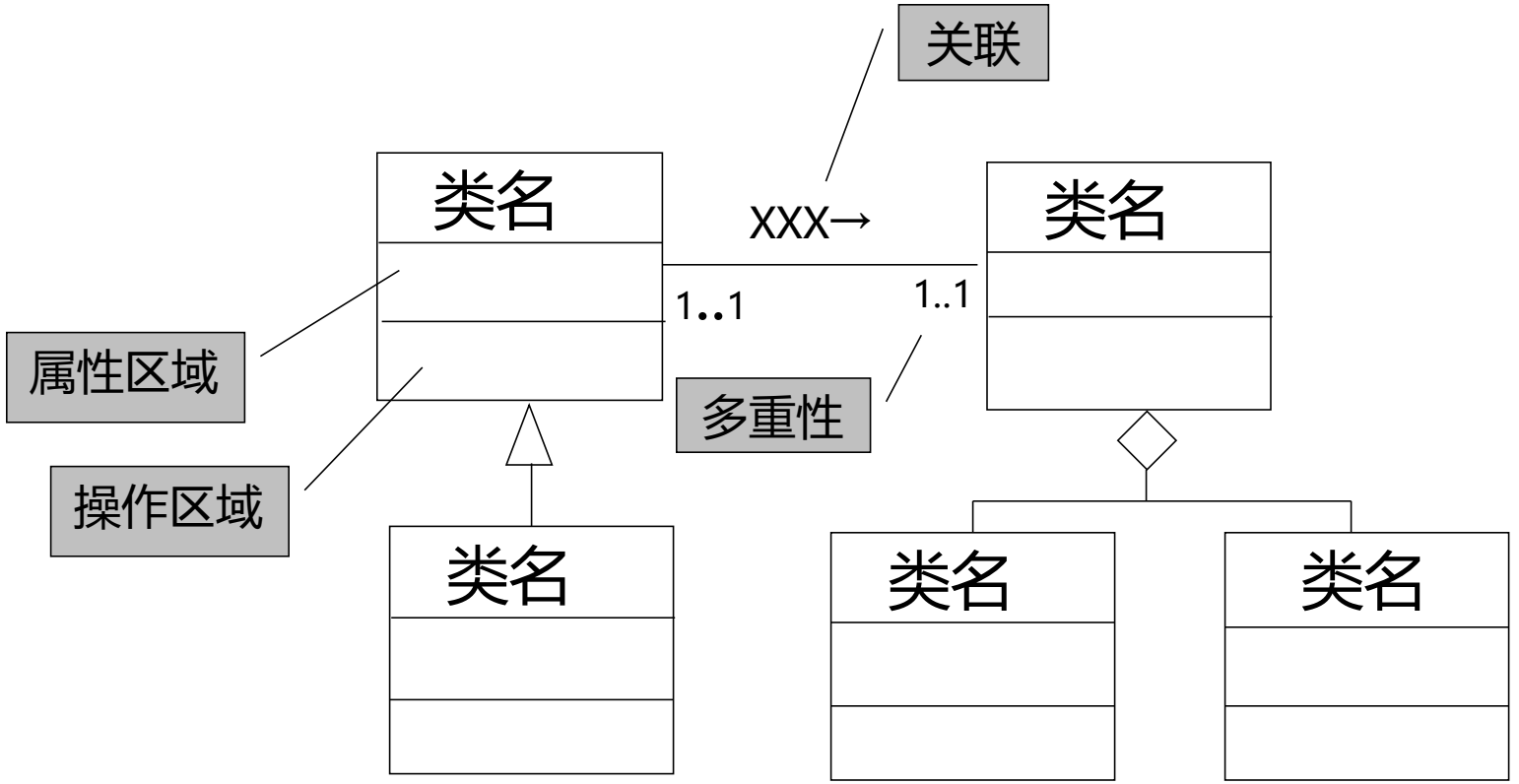
静态建模

类图和对象建模

- 类图是系统建模过程中最重要的部分，也是花费精力最大的活动。
- 类图描述系统中各个对象之间存在的关系，表达系统的静态结构，也叫做“对象建模”。

软件工程-软件需求分析与设计

- 类图的基本图元素：



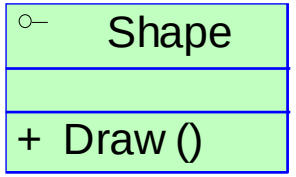
软件工程-软件需求分析与设计

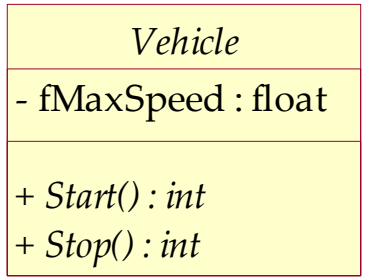
●类图中的事物及解释

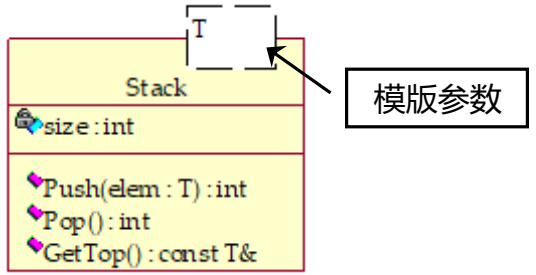
- 接口：一组操作的集合，只有操作的声明而没有实现；
- 抽象类：不能被实例化的类，一般至少包含一个抽象操作；
- 模版类：一种参数化的类，在编译时把模版参数绑定到不同的数据类型，从而产生不同的类；



Shape







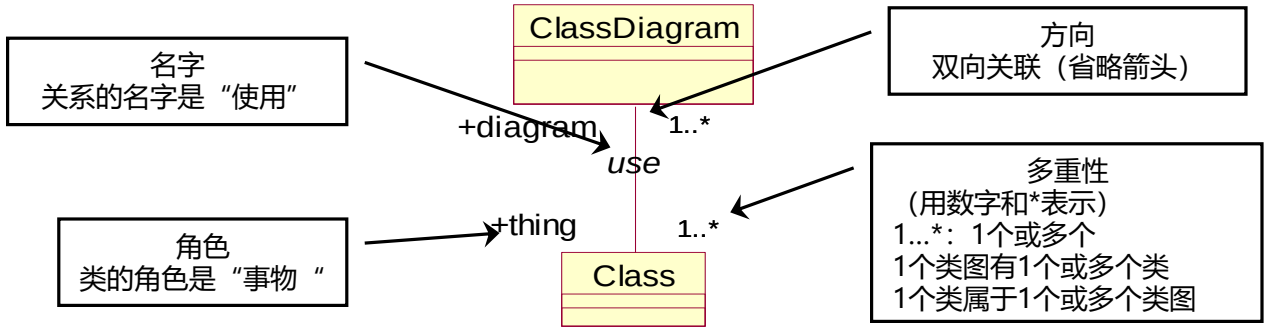
模版类

软件工程-软件需求分析与设计

● 类图中的关系及解释

➢ 关联关系--描述了类的结构之间的关系。具有方向、名字、角色和多重性等信息。一般的关 联关系语义较弱。也有两种语义较强，分别是聚合与组合。

UML表示法



实例

聚合关系

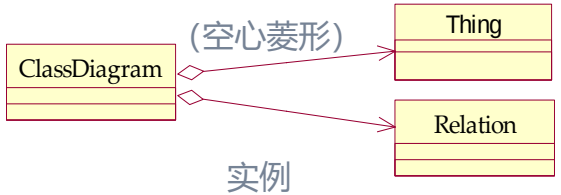
➢ 特殊关联关系，指明一个聚集（整体）和组成部分之间的关系

UML表示法

组合关系

➢ 语义更强的聚合，部分和整体具有相同的生命周期

UML表示法



实例

类图包含有事物和关系，类图不存在了，事物和关系还可用于其它的类图

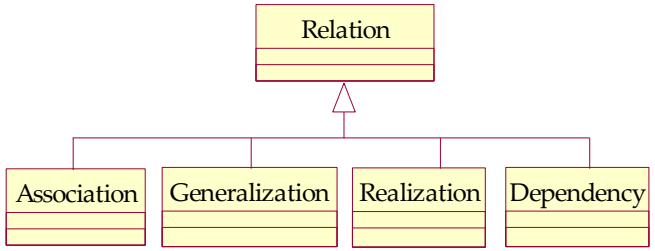


实例

类与关联关系之间有组合关系，类不存在了，则相应的关联关系也不存在

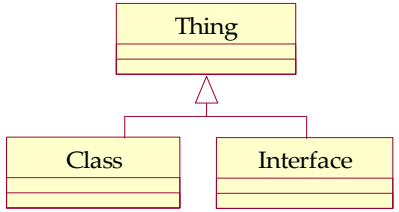
软件工程-软件需求分析与设计

- 泛化关系--在面向对象中一般称为继承关系，存在于父类与子类、父接口与子接口之间。



关联、泛化、实现、依赖都是一种关系

UML表示法

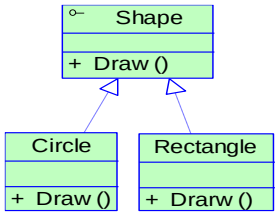


类、接口都是一种事物

- 实现关系

※对应于类和接口之间的关系

UML表示法

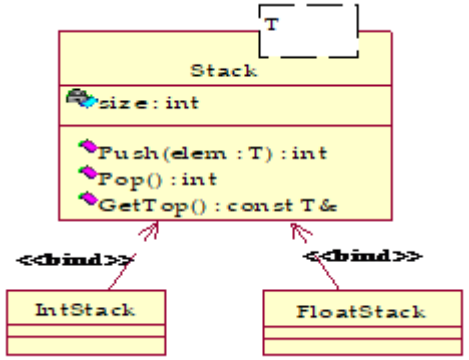


类Circle、Rectangle实现了接口Shape的操作

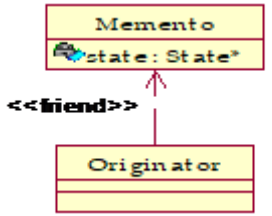
- 依赖关系

※描述了一个类的变化对依赖于它的类产生影响的情况。有多种表现形式，例如绑定(bind)、友元(friend)等

UML表示法



模板类Stack<T>定义了栈相关的操作；IntStack将参数T与实际类型int绑定，使得所有操作都针对int类型的数据



类Memento和类Originator建立了友元依赖关系，以便Originator使用Memento的私有变量state

软件工程-软件需求分析与设计

- **类定义**

类是对具有相同属性、操作、关系和语义的对象集合的描述。

- **对象定义**

对象是具有明确语义边界并封装了状态和行为的实体，即它是系统中用来描述客观事物的一个实体，是构成系统的一个基本单位，由一组属性和作用在这组属性上的一组操作构成。

软件工程-软件需求分析与设计

- **识别对象和筛选的策略**

对象建模的基础：

在用例图完成捕获需求后，将问题域和系统责任作为基础，分析系统中的对象和类。

识别对象

方法1：

利用需求得到的问题陈述，从中挑选名词或代词，以及名词短语来识别对象和类。

软件工程-软件需求分析与设计

方法2：

直接考虑现实问题中的对象，对应为系统中的对象。

方法3：

从系统边界外发现与系统进行交互的参与者，寻找系统处理对外接口的对象类。

方法4：

对照系统责任所要求的每项功能，确定能完成这些功能对象。

类属性及识别筛选策略

类的实例属性和类属性

实例属性定义：类的构成元素，用于描述类所对应的事物的一个性质。

- 实例属性与问题域和系统责任紧密相关；它并不是对象具有的全部属性；
- 每个实例属性有唯一的名字，其值属于给定的类型；
- 实例属性具有可见性；

软件工程-软件需求分析与设计

- **类属性定义：**类属性是描述一个类的所有对象的共同性质的一个数据项，对于该类的任何对象，它的属性值是相同的。

类属性，对一个类的全部对象只有一份共同的数据空间；在C++中可以用**static静态成员**说明符标出。

实例属性和类属性的区别和作用：

- 实例属性----用于不同性质的同种个体。
- 类属性-------用于相同性质的同种个体。

软件工程-软件需求分析与设计

- **类操作及识别调整策略**

操作定义 操作是类的构成元素，是类的对象被要求执行的服务。

- 内部操作和外部操作：

内部操作---只供对象内部的其他操作使用。

外部操作---响应其他对象请求时提供的服务。

- 系统行为和自身行为：

系统行为---系统施加于对象的行为。

例如：对象创建、复制、存储、删除

自身行为---简单算法行为：简单读写属性的值。

复杂算法行为：完成对象固有的行为算法。

- 分析模型应该以描述复杂算法操作为主

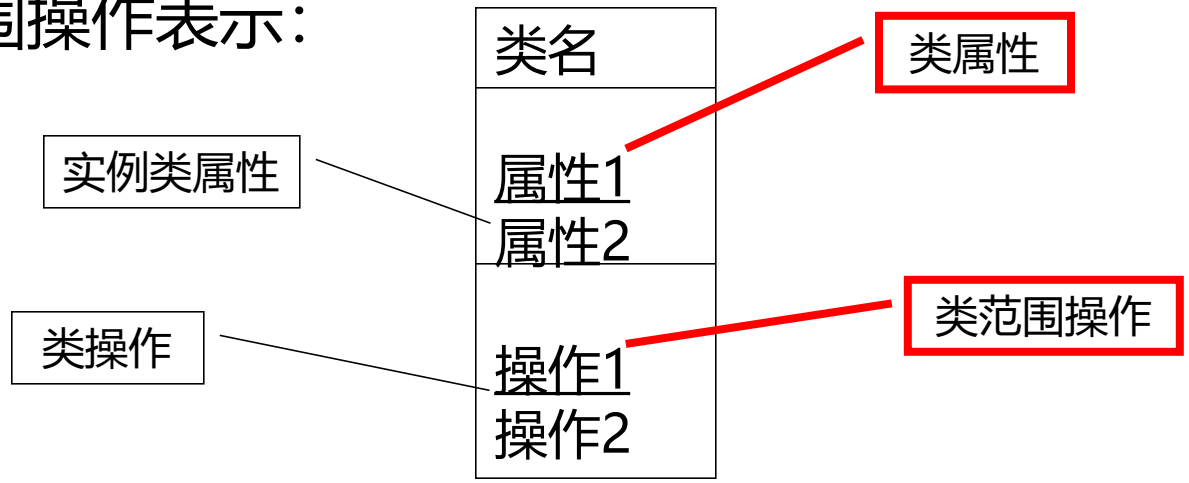
软件工程-软件需求分析与设计

- **类范围操作定义**

仅用于操纵类属性的操作叫做类范围操作，其余的操作叫做实例范围的操作。

类属性，对一个类的全部对象只有一份共同的数据空间；类范围操作，仅对这样的共同空间数据进行处理。

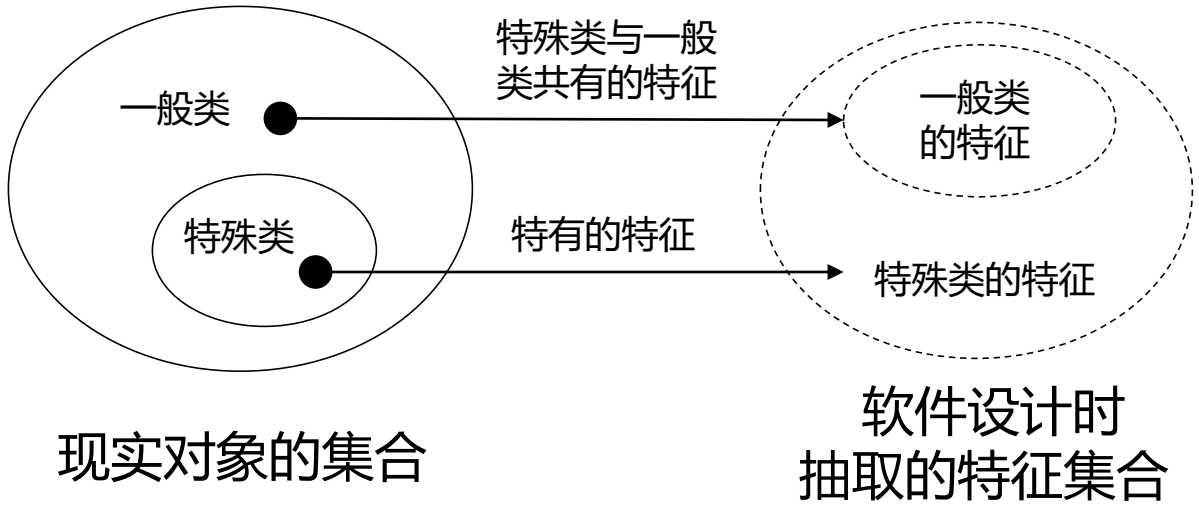
类属性和类范围操作表示：



类的继承关系

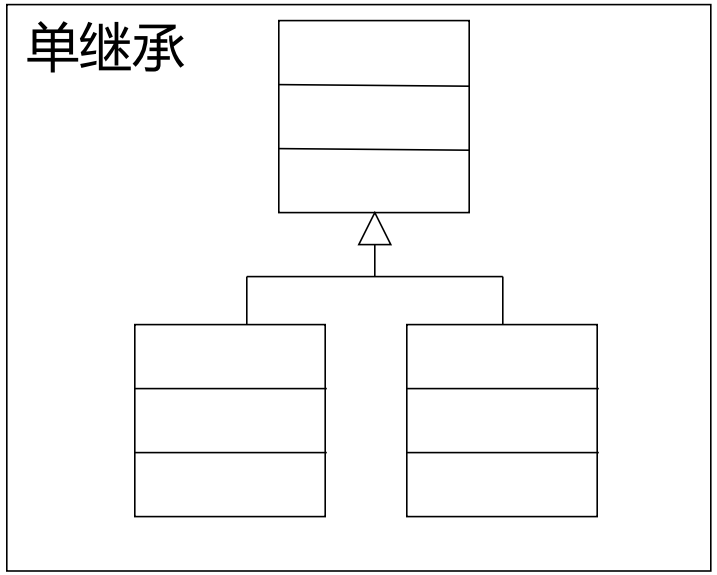
继承关系定义　如果类A具有类B的全部属性和全部操作，而且还具有自己特有的一些属性或操作，则A叫做B的特殊类，B叫做A的一般类，A与B之间的关系称为继承关系。

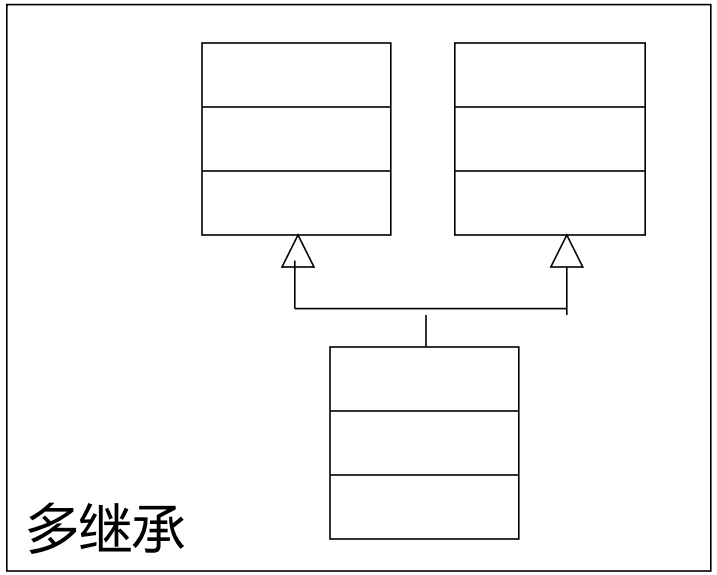
一般类与特殊类

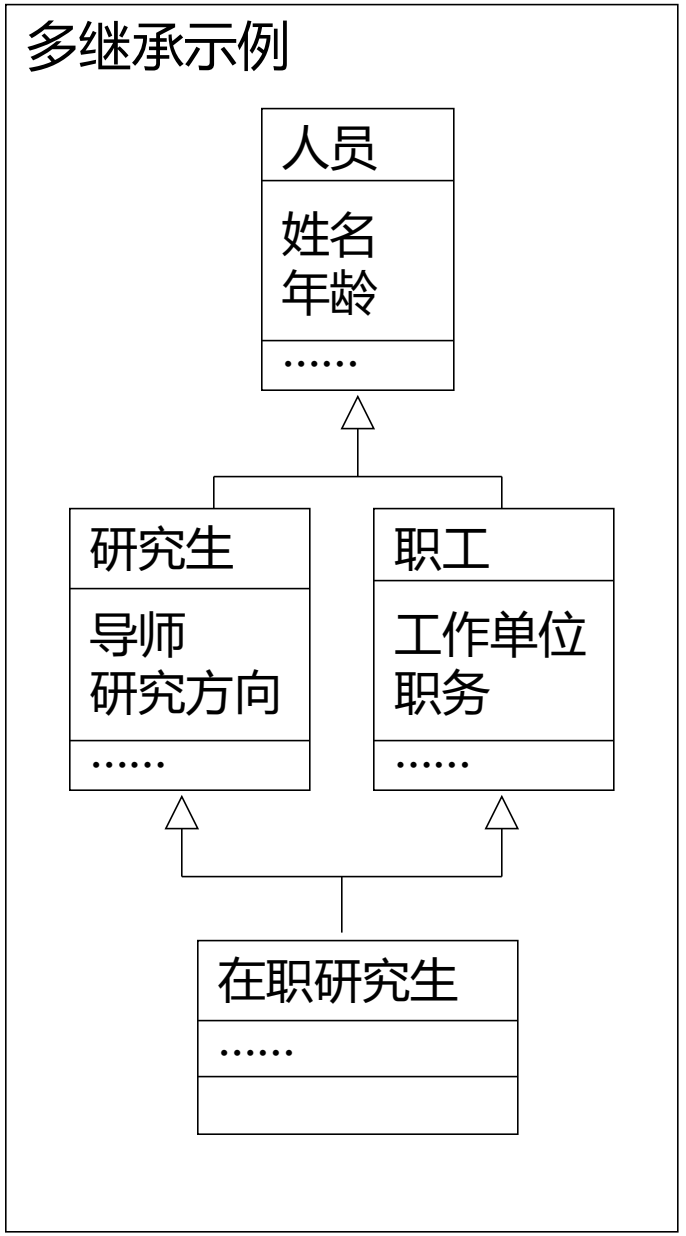


软件工程-软件需求分析与设计

- 继承关系表示







软件工程-软件需求分析与设计

- **建立类的关联**

1）类的关联
2）关联的重数
3）对象链
4）关联角色
5 ）关联类
6）N元关联
7）关联的限定符
8）聚合关联
9）类的依赖关系表示

1）类的关联

关联的基本概念：

关联-----表示对象类之间的静态关系。

静态关系----表示对象之间固有的联系。

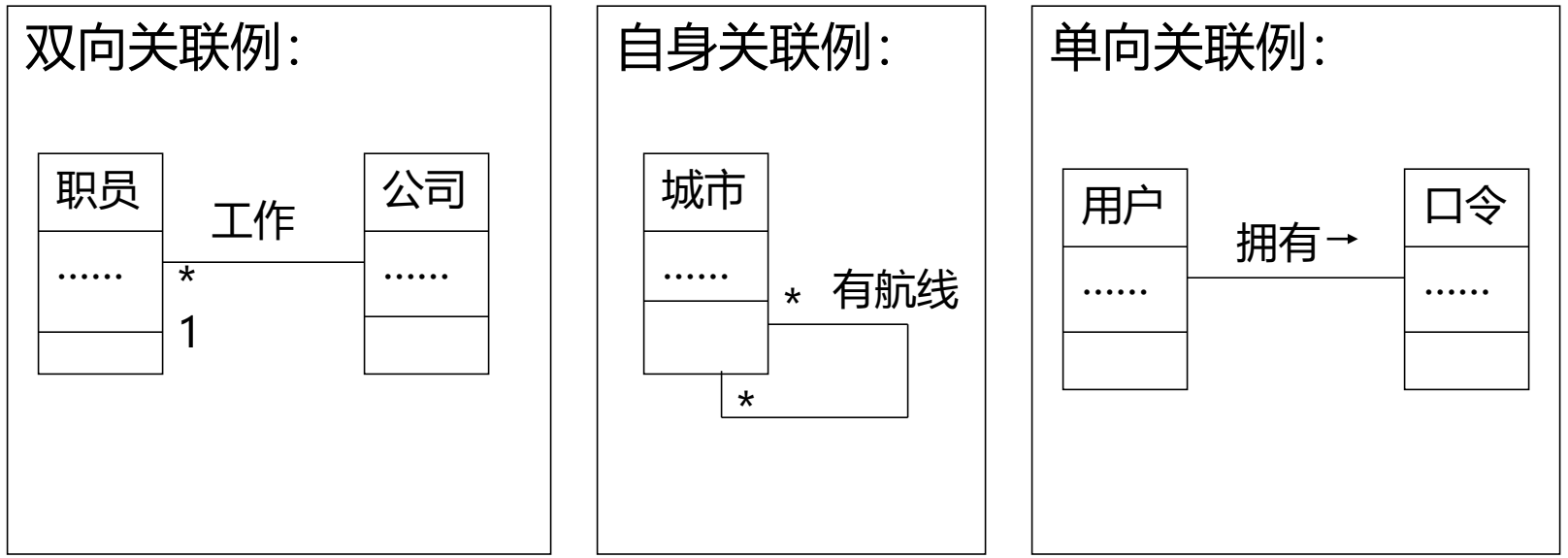
要点：

• 两个类之间的关联，实质上是通过属性来表示对象之间的联系，即一个类的对象属性值是另一个类的对象实例（用指针实现）。

• 静态关联与系统责任相关，如果这些关系是系统责任目标的必要信息，则需要表达它们。

软件工程-软件需求分析与设计

关联定义　如果一个类的对象与另一个类的对象之间有语义连接关系，则这两个类之间的语义关联就是关联。



关联一般是双向的，若有单向限制用箭头表示。

2）关联的重数

重数定义　关联端上的重数，表示一端的对象需要另一端对象的个数。

重数的表示：A端的一个对象需要B端Mb个对象

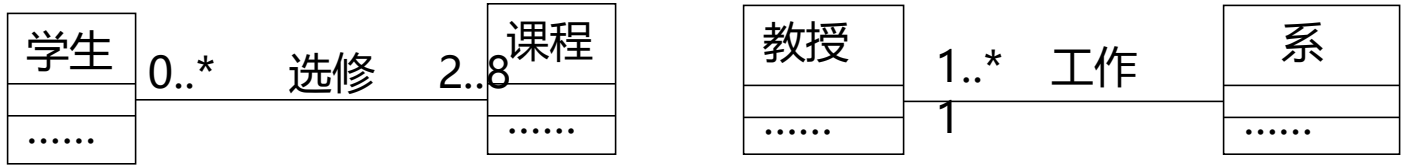
B端的一个对象需要A端Ma个对象



重数值的表示：

值	表　示
n .. m	表示与类的n至m个对象关联

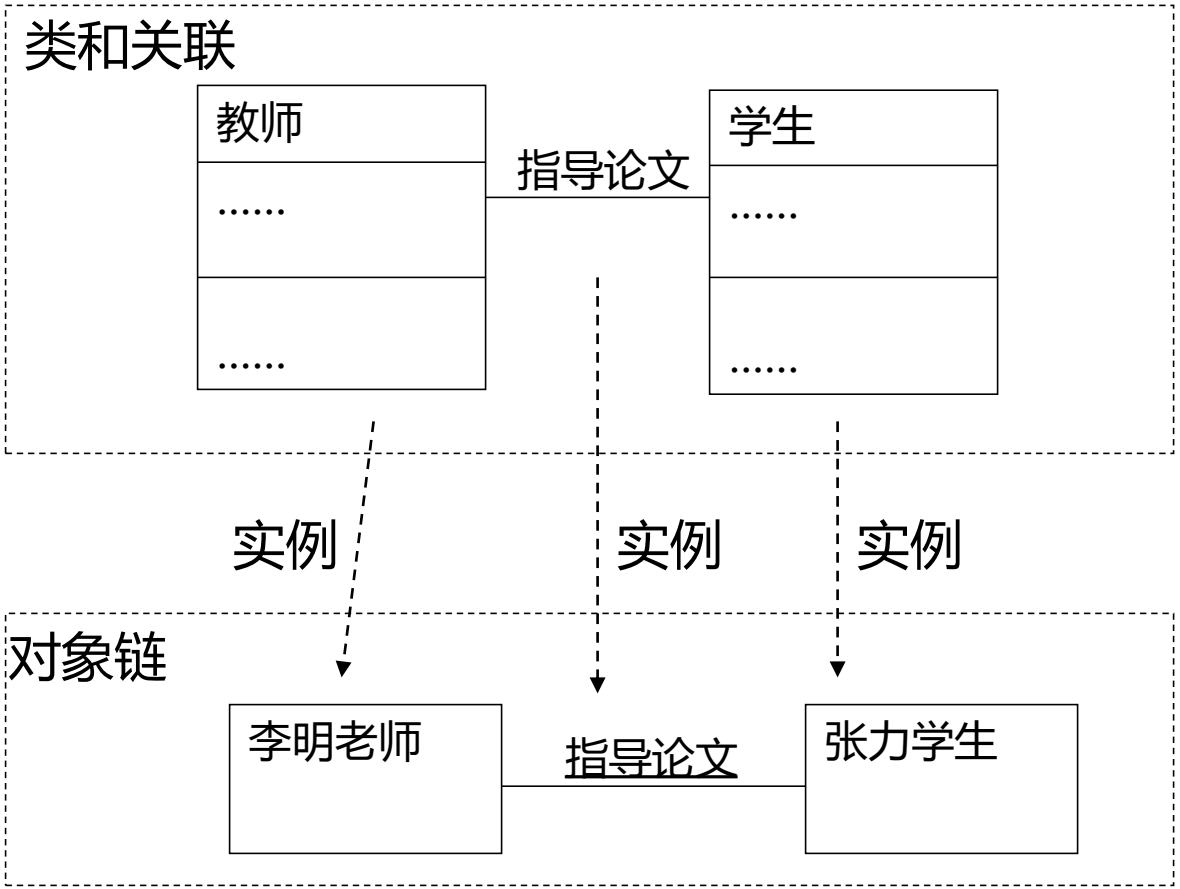
例：



软件工程-软件需求分析与设计

3）对象链

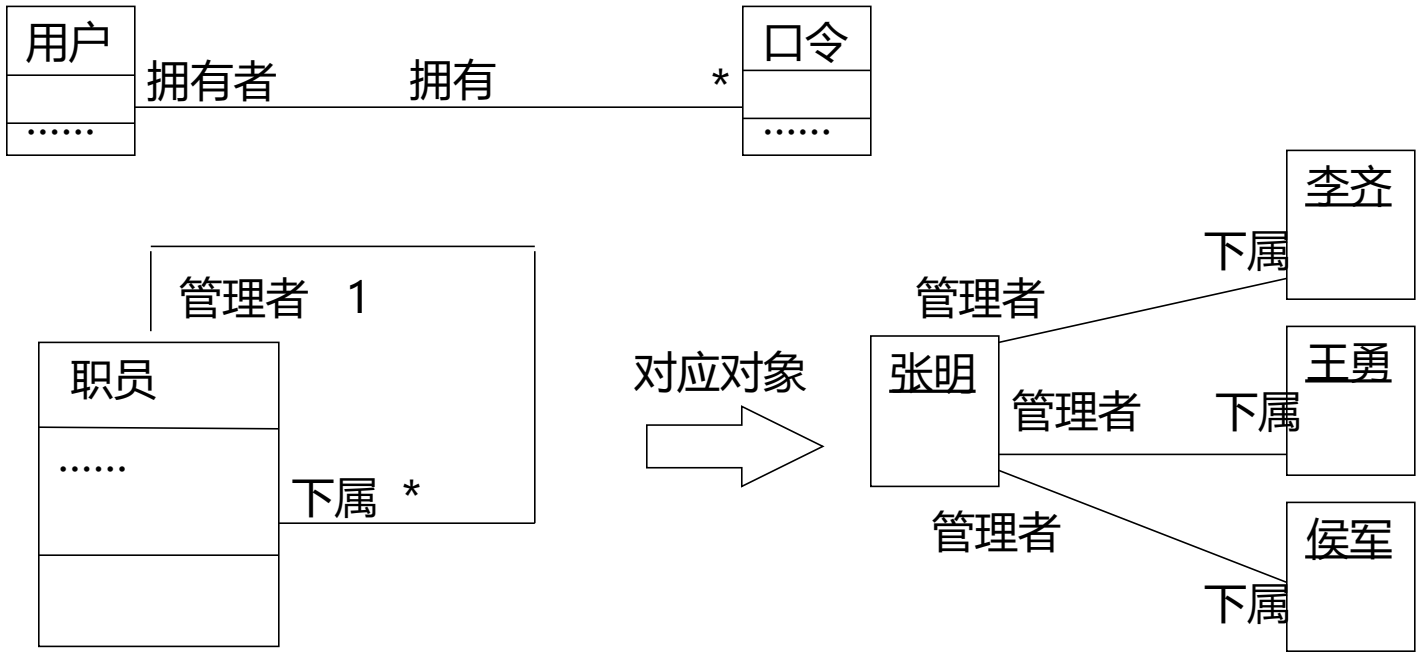
链定义　链是关联的实例，是对象间的语义连接。



4）关联角色

关联角色----表示需要明确的一个角色属性。在关联的端点，可以表示相连接类所扮演的角色，关联角色的名字称为“关联角色名”。

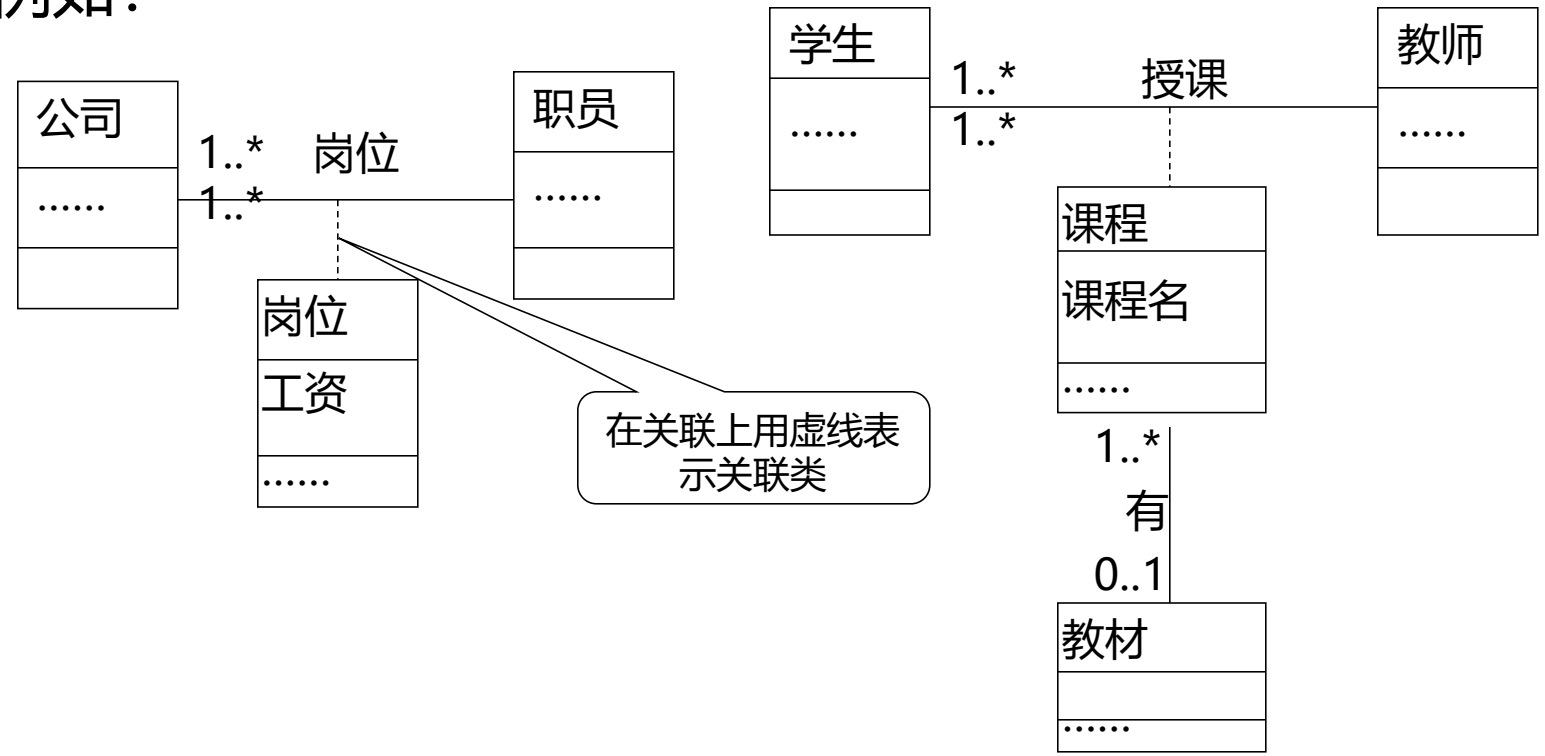
带有角色名的关联：



5） 关联类

关联类是一种建模元素，表示关联本身也是一个类。关联类兼有关联和类的双重特征，可把它看作具有类性质的关联，也可看作是具有关联性质的类。

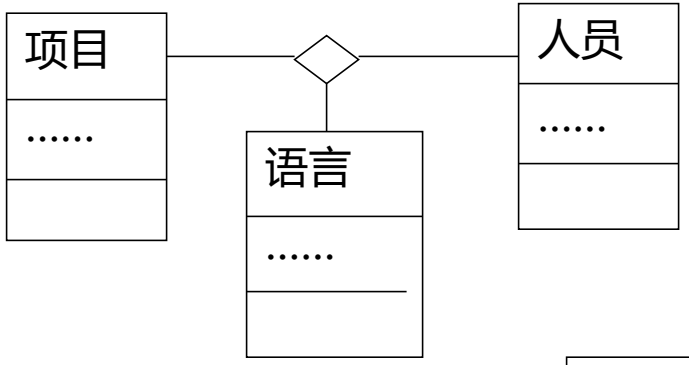
例如：



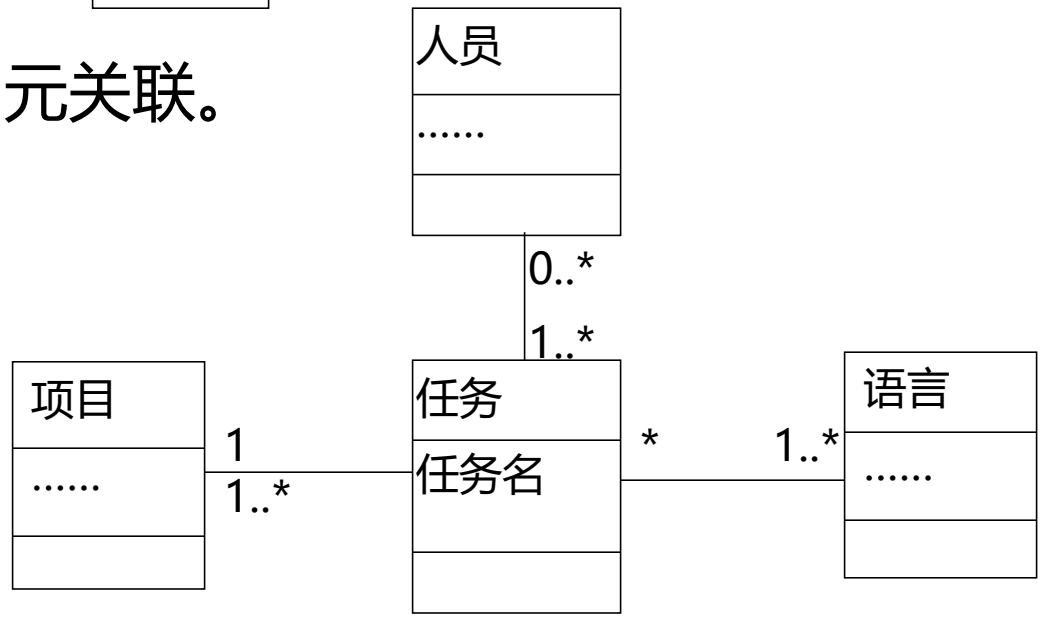
6） N元关联

N元关联是三个或多个类之间的关联。

N元关联



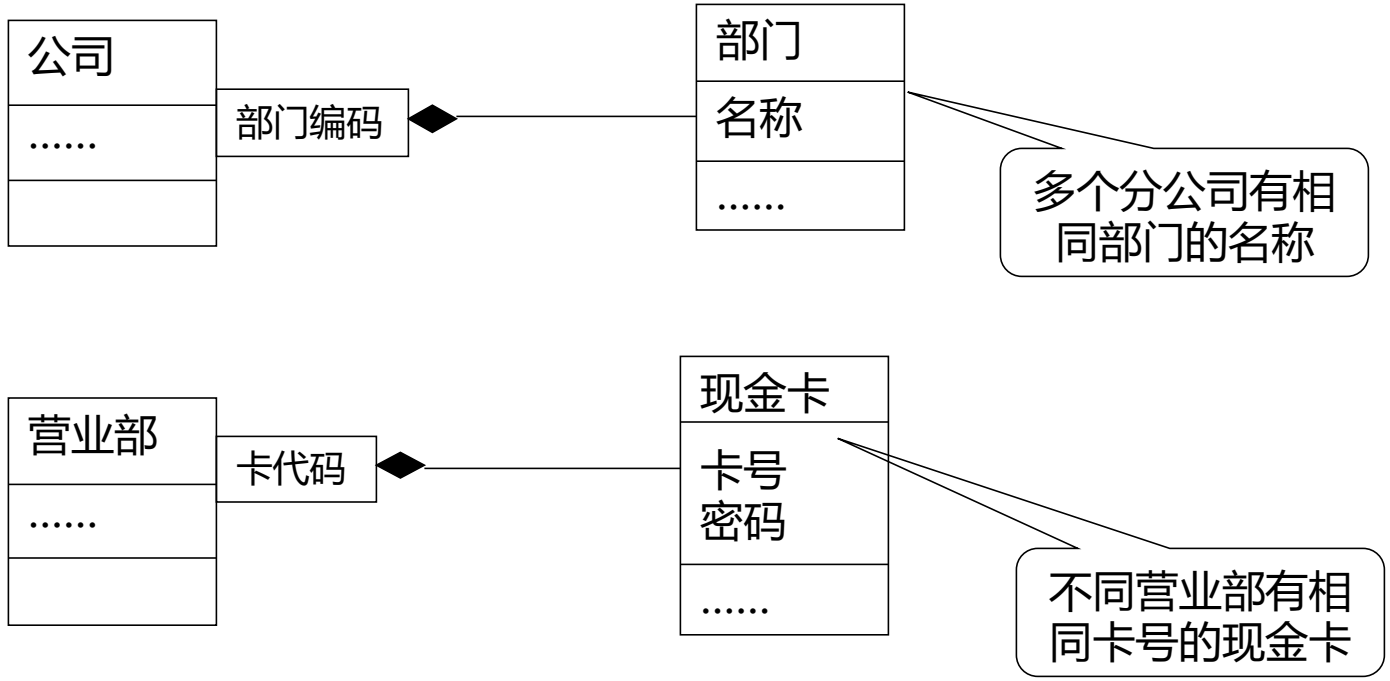
增设类可以表示N元关联。



7） 关联限定

关联限定----表示作为查找另一端对象的特征属性，在特定的约束下提供快捷的搜索路径。

例:



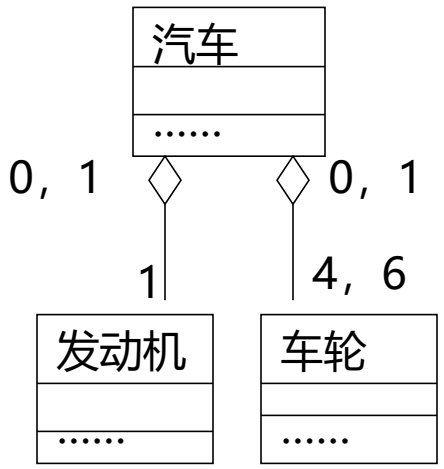
8） 聚合关联(Aggregation)

聚合关链表示一种特殊的关联。

聚合定义	聚合表示整体类和部分类之间的“整体—部分”关系。

聚合----部分对象和整体对象可以独立生存。

例:

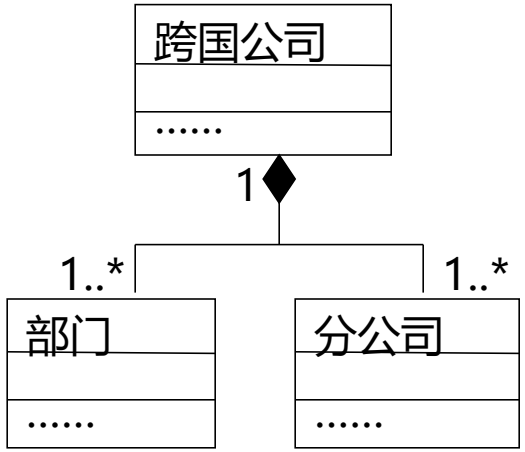


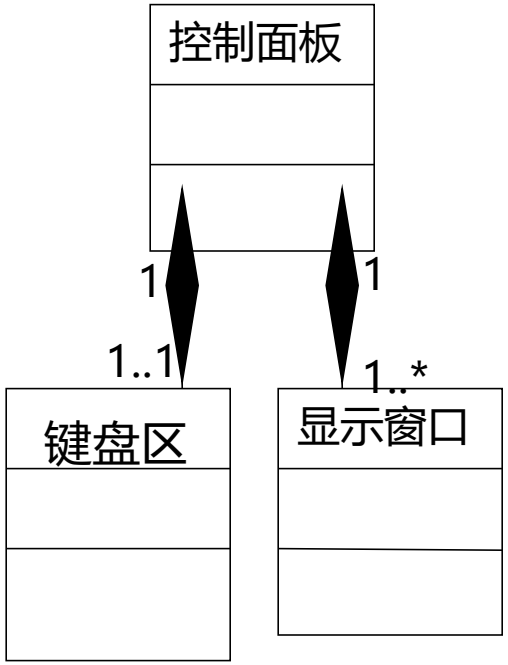
组合关联(Composition)

组合定义　组合是聚合的一种形式，其中部分和整体之间具有很强的“属于”关系，且它们的生存期是一致的。

组合----部分对象和整体对象生存期一致

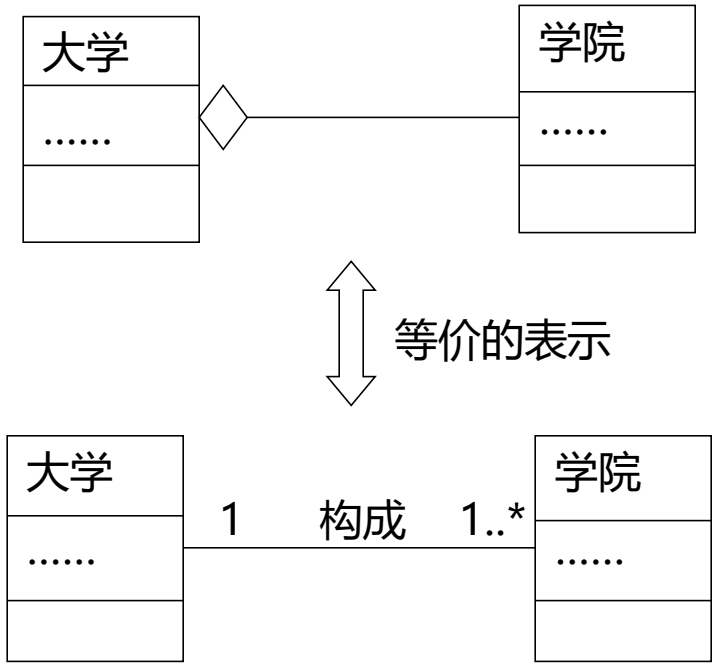
例:





软件工程-软件需求分析与设计

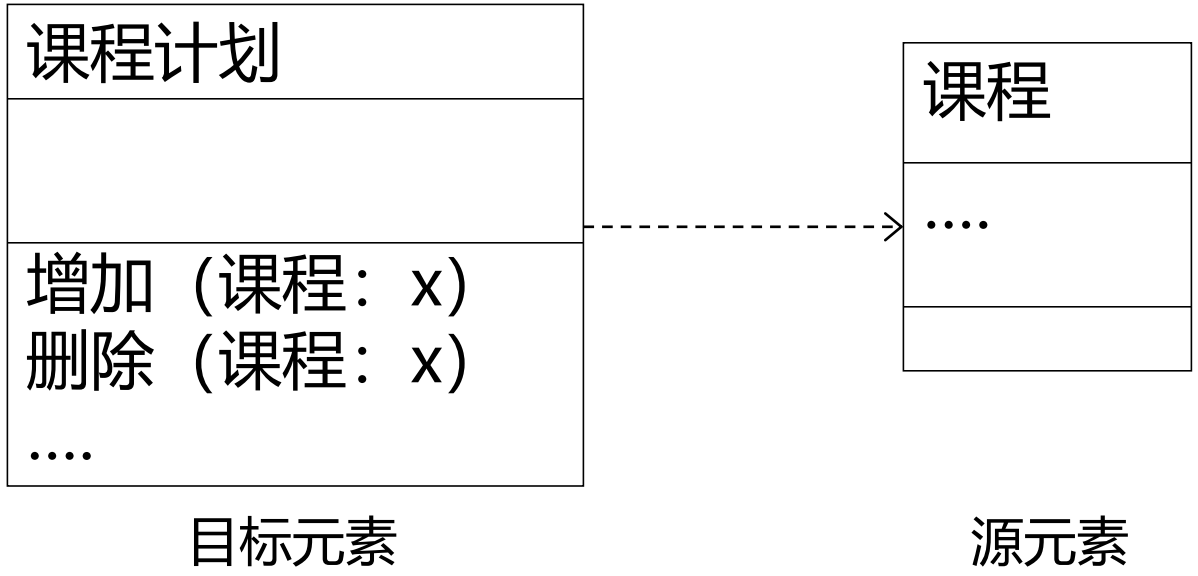
聚合也是一种关联，所以，可以表示成一般的关联的形式。



9） 类的依赖关系表示

- 依赖关系定义　模型元素(或模型元素集合)之间的一种语义关系，目标元素的改变需要根据源元素的改变而变化。

例：课程计划依赖课程的变化而变化



● 接口类和信号

接口类----把类的公共可见性操作组织在一起，提供的服务集合。

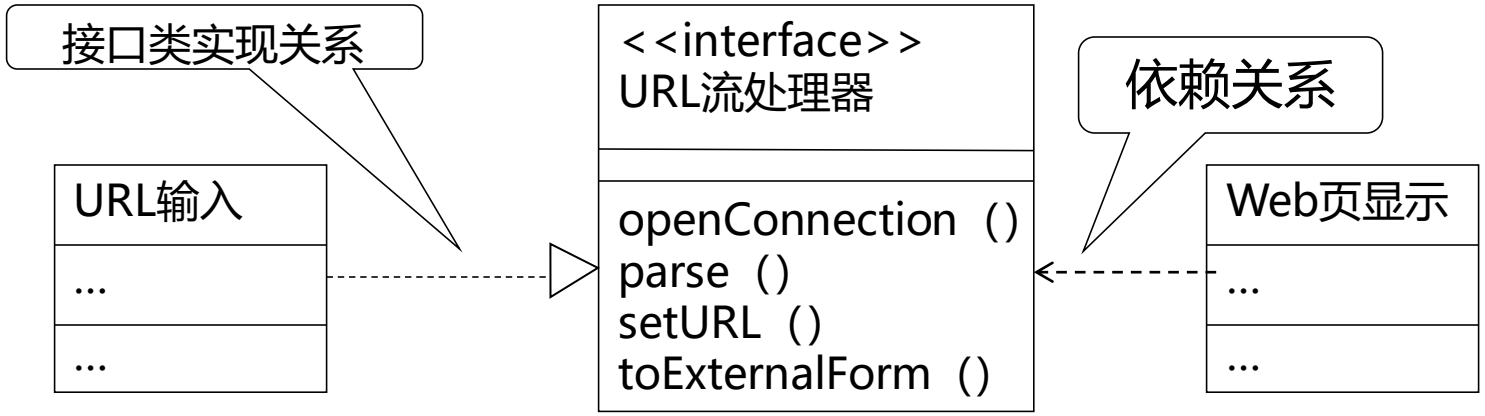
- 接口类作为类之间交互操作的契约。
- 接口类两端的类可独立变更，但操作契约不变。
- 多个类可使用一个接口类呈现整体服务。

接口类的表示：

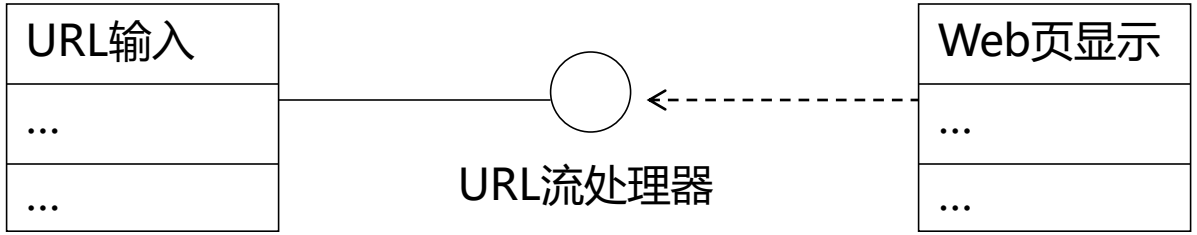


软件工程-软件需求分析与设计

- **接口类的表示例：**



简化形式的类接口表示：

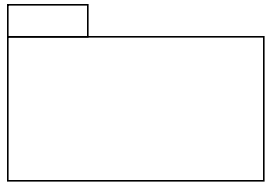


● 包图 (Packages)

包定义：包是对模型成分分组的机制

要点：

- 把模型成分组织成为包；
- 模型成分包括类或用例；
- 包有唯一的命名，可以被独立引用。



包的表示：

目的：

- 用包组织系统成分，使系统形成整体组织结构。
- 包作为独立系统成分，可被整体利用。

● 包的组织和划分

划分包的策略：

1）把在语义上接近并需要一起变化的成分组织成包；

2）可以组织合并包，形成包的嵌套结构，每个包的内层成分最多5-7个；

3）组织包形成树层次结构；

4）标识包中模型成分的可见性；

5）标识包之间的依赖关系。

动态模型

交互图 Interaction Diagram

描述对象间以及对象与参与者间动态交互过程的次序，通常对应一个用例。

包括：
- 顺序图---强调消息交互的时序；
- 协作图---强调对象收发消息的组织结构；

状态图State Diagram

描述一个对象在其生存期中的状态序列，表现引起状态变换的事件和状态变化的动作。

活动图 Activity Diagram

描述系统的工作流程和并发行为。

● 顺序图概念与表示

顺序图 Sequence Diagram 作用：

• 在给定的语境中，通过对象之间的消息通信，展现对象的行为；用以发现对象的操作。

• 顺序图可协助发现主动对象；

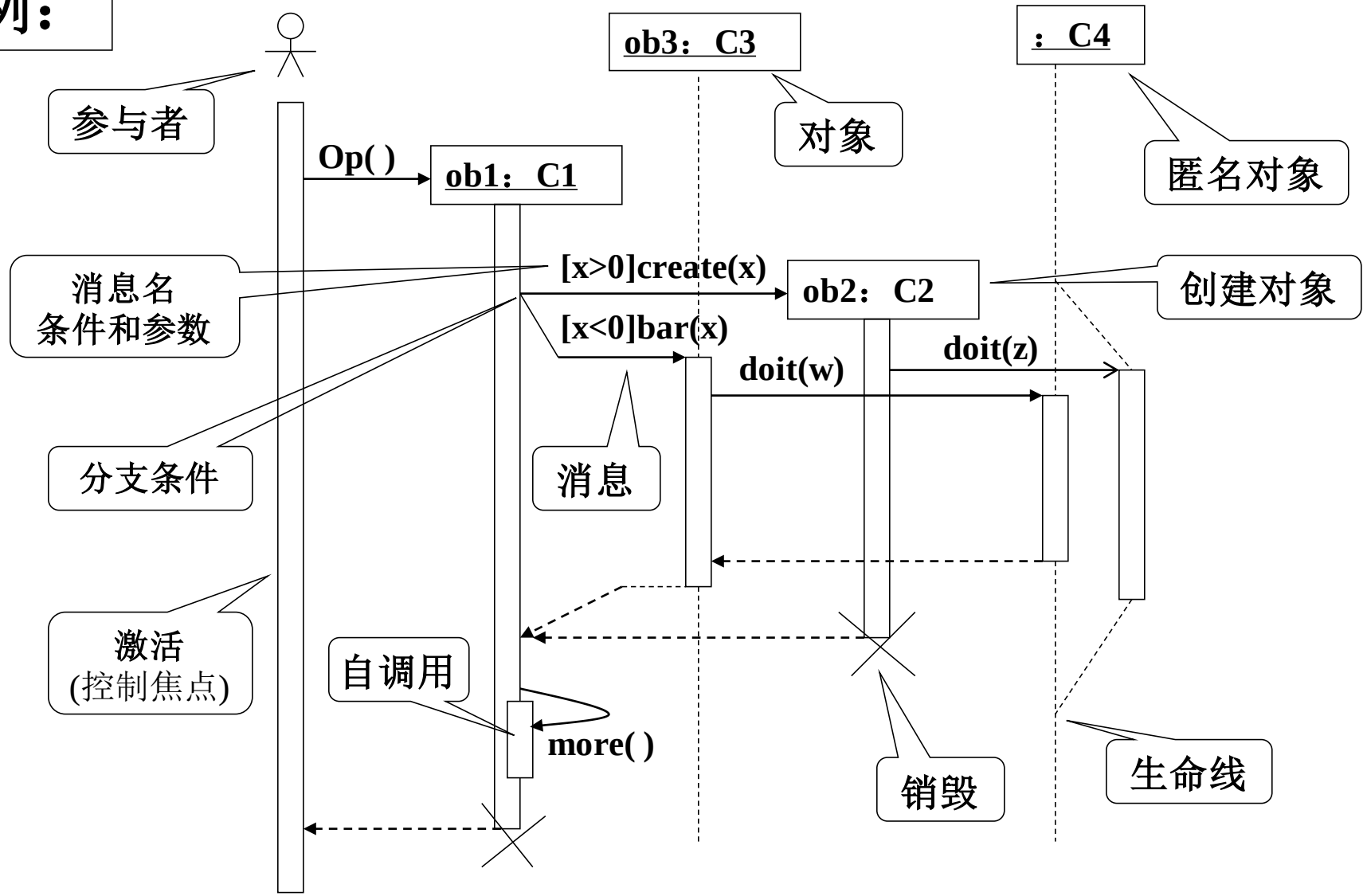
表示：对象(与参与者)之间的交互；

元素：一组协作的对象(与参与者)和之间发送的消息；

强调：消息之间的顺序；

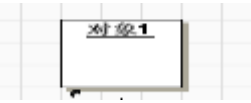
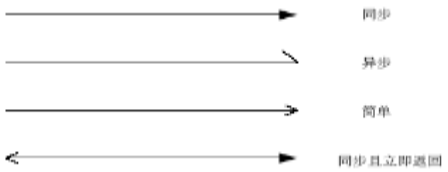
软件工程-软件需求分析与设计

顺序图示例：



软件工程-软件需求分析与设计

- **顺序图示说明：**

事物名称	解释	图
参与者	与系统、子系统或类发生交互作用的外部用户(参见用例图定义)。	
对象	顺序图的横轴上是与序列有关的对象。对象的表示方法是：矩形框中写有对象或类名，且名字下面有下划线。	
生命线	坐标轴纵向的虚线表示对象在序列中的执行情况(即发送和接收的消息，对象的活动)这条虚线称为对象的“生命线”。	
消息符号	消息用从一个对象的生命线到另一个对象生命线的箭头表示。箭头以时间顺序在图中从上到下排列。	

软件工程-软件需求分析与设计

● 顺序图示说明：

- 对象---类创建的对象实体名
- 匿名对象---只表示类名
- 创建对象---通过消息发送创建的对象
- 生命线---表示对象存在的时间段，
- 激活（控制焦点）---表示对象行为的操作历程，即活动的持续时间
- 销毁---对象被结束生命（杀死）
- 自调用---对象的操作递归调用自己，或本对象的其他操作
- 消息--- 包括：

  实箭线“ ⟶ ”同步过程，需要等待回应，会有嵌套控制

  枝状箭线“ ⟶ ” 异步通信，发出消息后不必等待，继续执行虚的枝状箭线“ - - - -> ” 显式表示回应返回（一般的控制流省略返回消息，表明每个请求都有返回）

● 顺序图的建立

过程和步骤：

1）根据具体用例中的对象或参与者的交互语境，设置交互；

2）在顺序图上部列出所选的一组对象或参与者；

3）为每个对象和参与者设置生命线。

4）用消息箭线显式地标出交互中将被创建和撤消的对象；

5）在对象之间，标出消息进行传递的序列；

6）在对象生命线上，按对象操作的次序，排列各操作的激活区间，若两个对象的操作执行属于同一控制线程，则接收者操作的执行应在发送者发出消息之后开始，并在发送者结束之前结束；

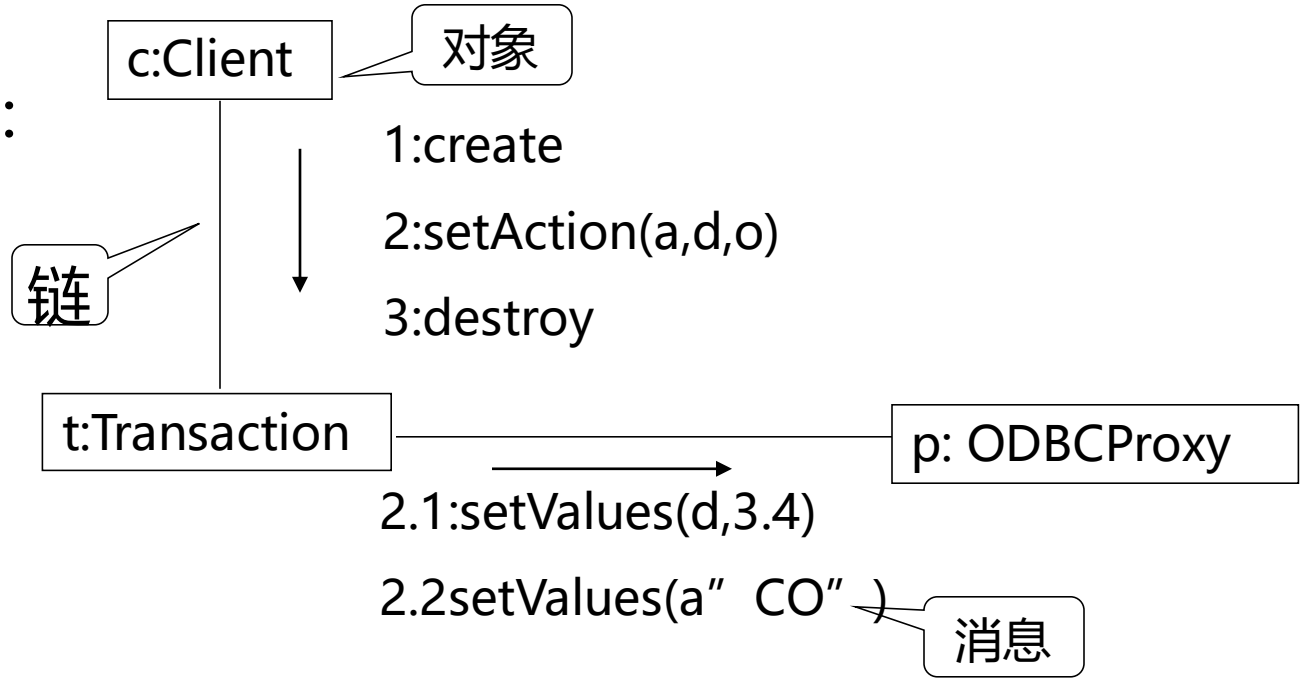
7）描述对象执行的操作功能、时间、范围约束；

8）可显式地表示消息的迭代或分支。

● 协作图概念与表示

协作图Collaboration Diagram表示协同完成某行为对象之间的交互，强调对象的消息结构，而忽略时间顺序。

协作图示例：

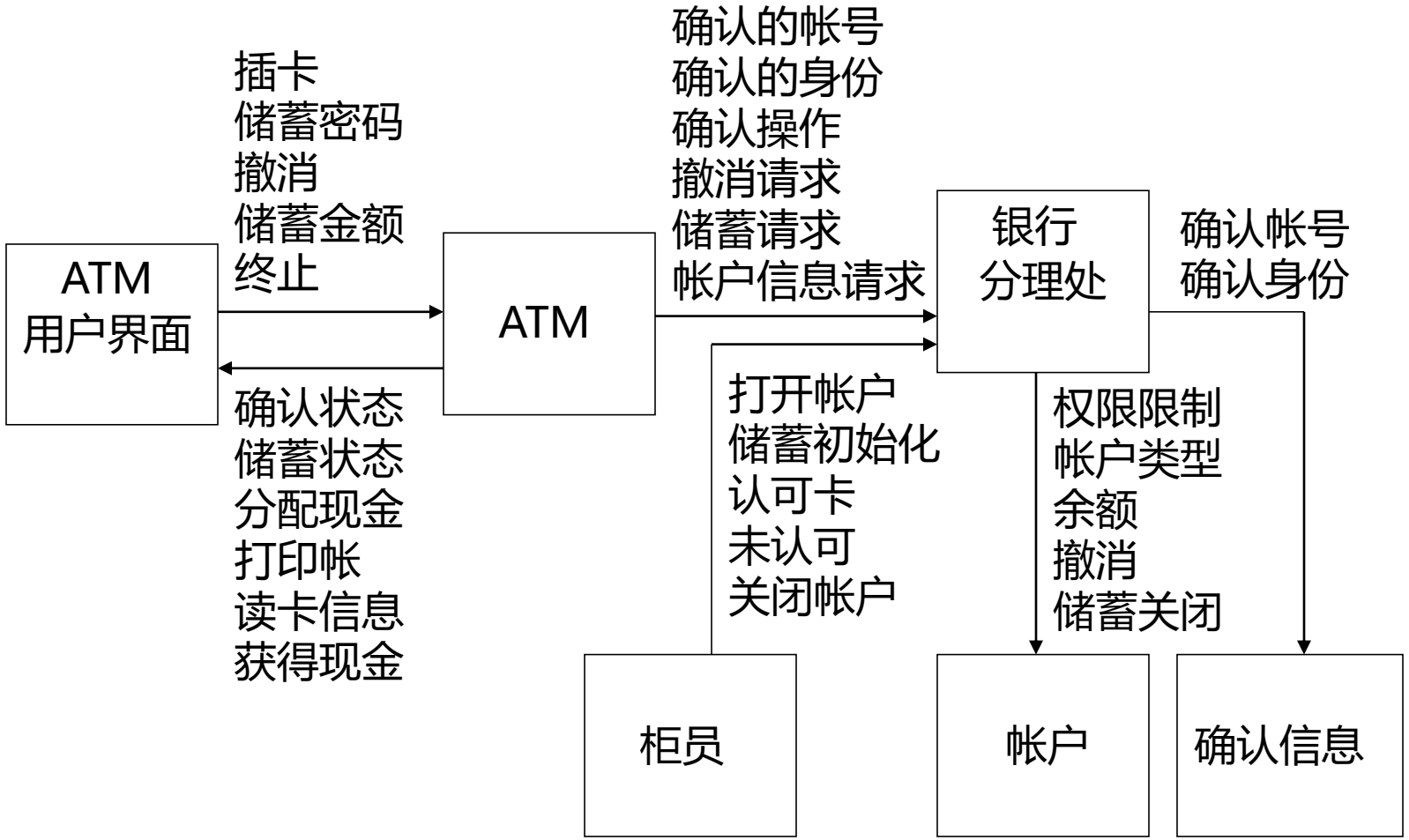


软件工程-软件需求分析与设计

- **协作图的用途**

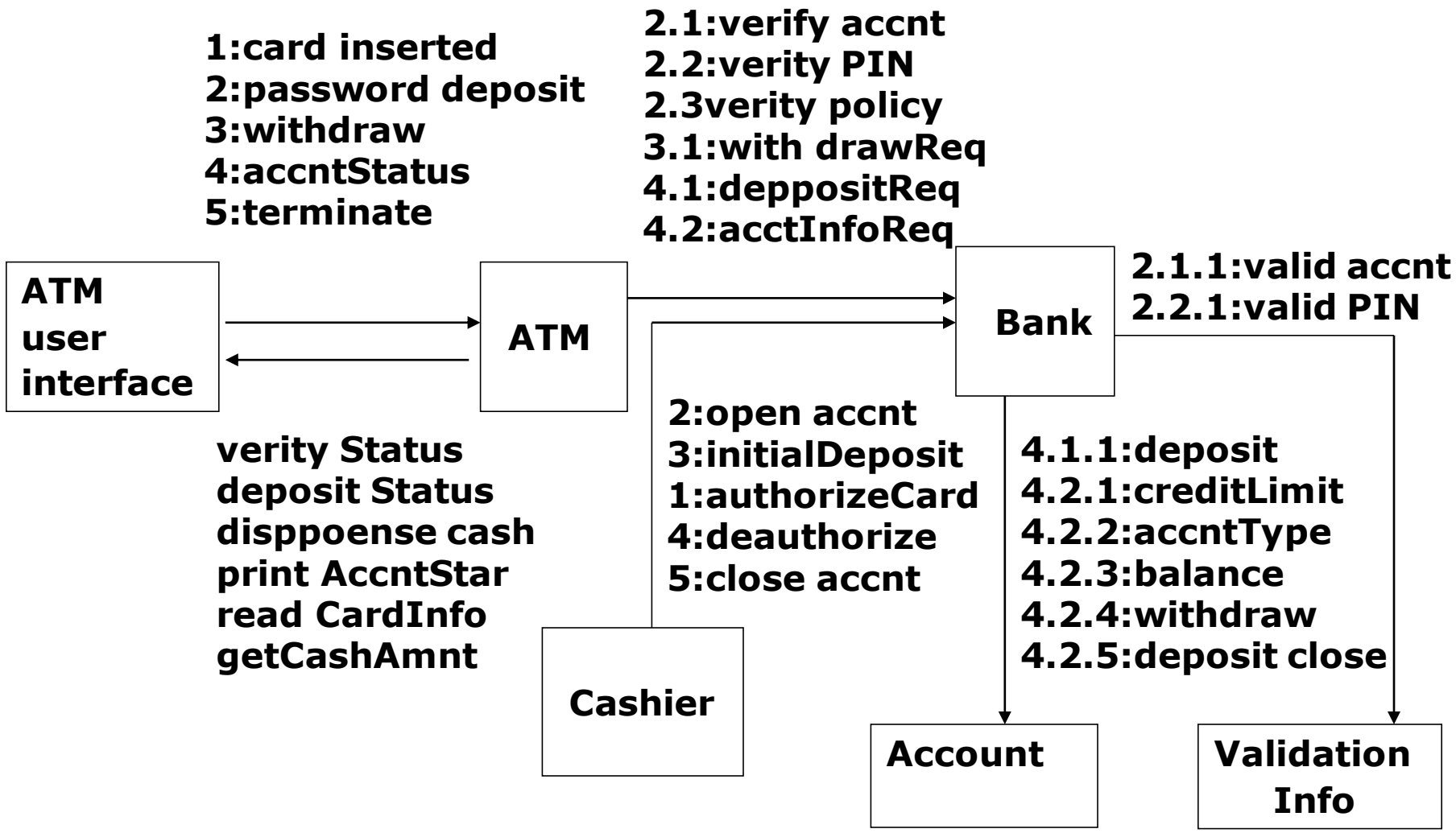
可用于分析对象行为的控制线程，规划系统分布及测试用例

例：用协作图规划储蓄应用的行为：



软件工程-软件需求分析与设计

例：用协作图规划储蓄应用的行为：

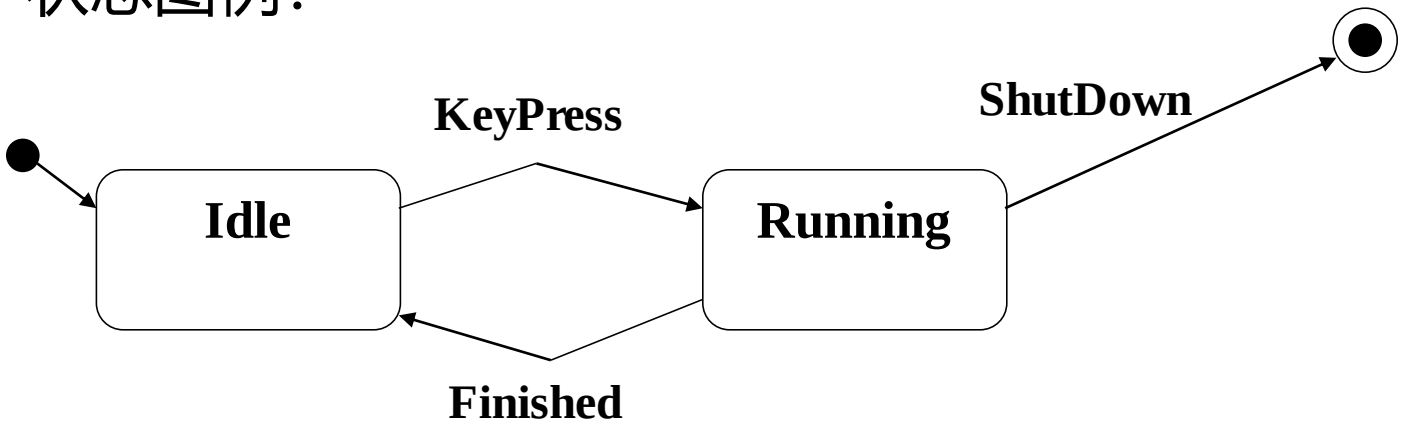


● 状态图概念与表示

状态图 State Transition：

表示对象事物的行为，描述了一个对象在其生命期内响应事件所经历的状态序列以及对这些事件所做出的反应。

状态图例：



软件工程-软件需求分析与设计

- **状态图遵循事物转换的规律：**

1）事物在生命期中都经历不同的状态；

2）事物在特定时刻总处于某确定的状态；

3）事物的状态变化总是由事件引起的；

4）事物状态的转化是即时的；

5）事件发生时事物需要采取相应的动作；

6）事物的状态转换总会遵照一定的规律的。

软件工程-软件需求分析与设计

● 状态图的作用：

分析对象上的所有状态，确定状态值（行为属性）的范围，

以及对应的操作，包括:

- 对象具有的稳定状态
- 保持稳定状态的动作行为
- 状态转换的触发事件
- 状态改变时发生的动作
- 对象的创建和消亡（必要时）

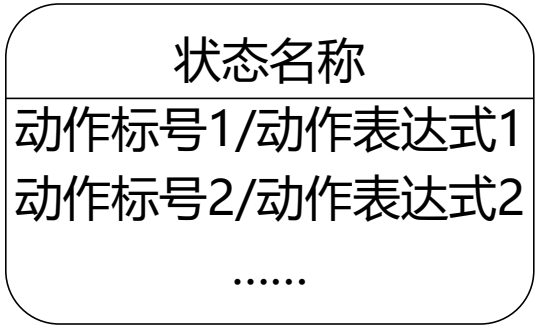
要点：针对有明确生命阶段和复杂行为的对象，才有必要建立状态图，用以分析行为属性和值域范围。

软件工程-软件需求分析与设计

- 状态：状态是对象生命期的一个阶段，在该阶段中，该对象满足一些特定的条件、从事特定的动作或等待某个（些）事件。

- 动作：动作是在状态内或在状态转化时所做的操作，是原子的和即时的。

状态表示：



伪状态表示：

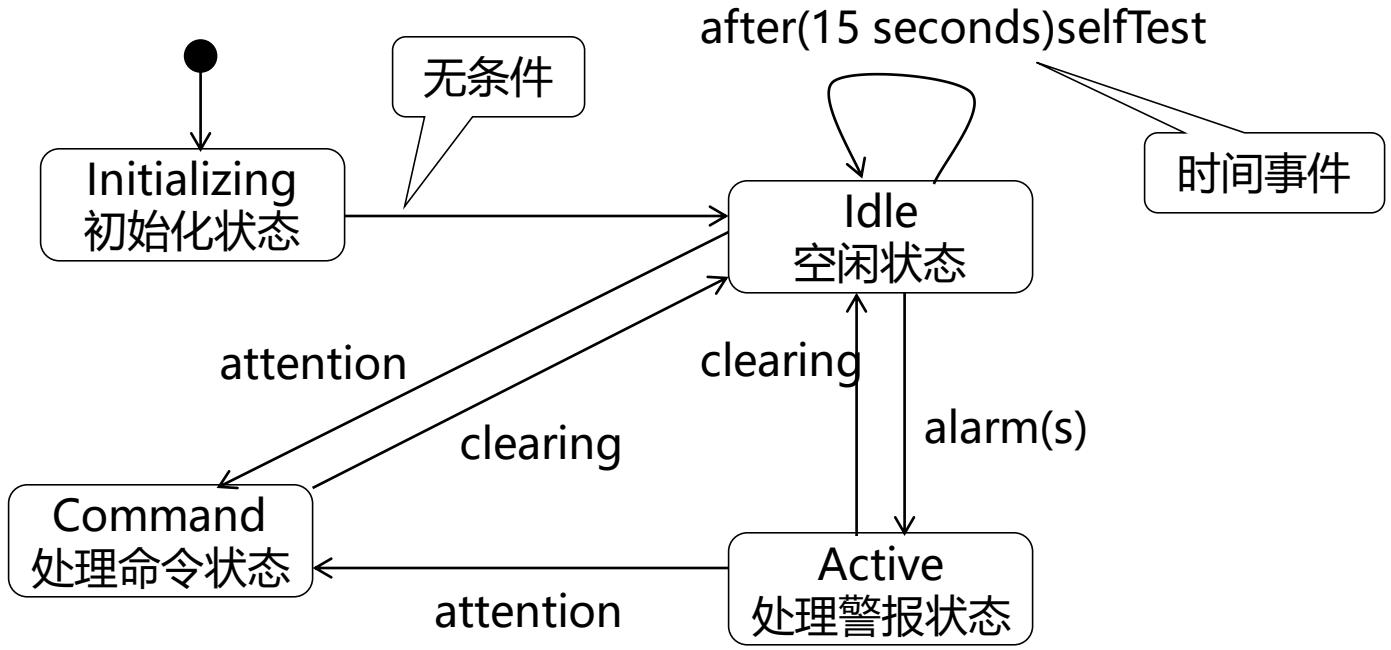


初始状态　　终止状态

软件工程-软件需求分析与设计

状态图例：

监视传感器类的状态图：



活动图概念与表示

活动图Activity Diagram：

描述系统的工作流程和并发行为，与状态图相比它强调流程的控制而不是状态在事件下的变化。

活动图的作用：

- 描述业务过程，特别能较好地表示并发流程，帮助理解涉及多个用例的工作流程。
- 描述具体操作算法，与程序框图有相同的作用。

➢ 活动图的概念接近结构化方法的流程图思想。

软件工程-软件需求分析与设计

- **活动图元素：**

包括：

1）活动---流程中的任务执行单元

2）泳道---活动的区域划分

3）分支---活动转向的分支

4）分叉--- 并发控制流的分支

5）汇合---分叉后的合并

6）对象流---活动图中的控制流

软件工程-软件需求分析与设计

- 活动图中的“活动” Activity

包括：动作状态 Action State

活动状态 Activity State



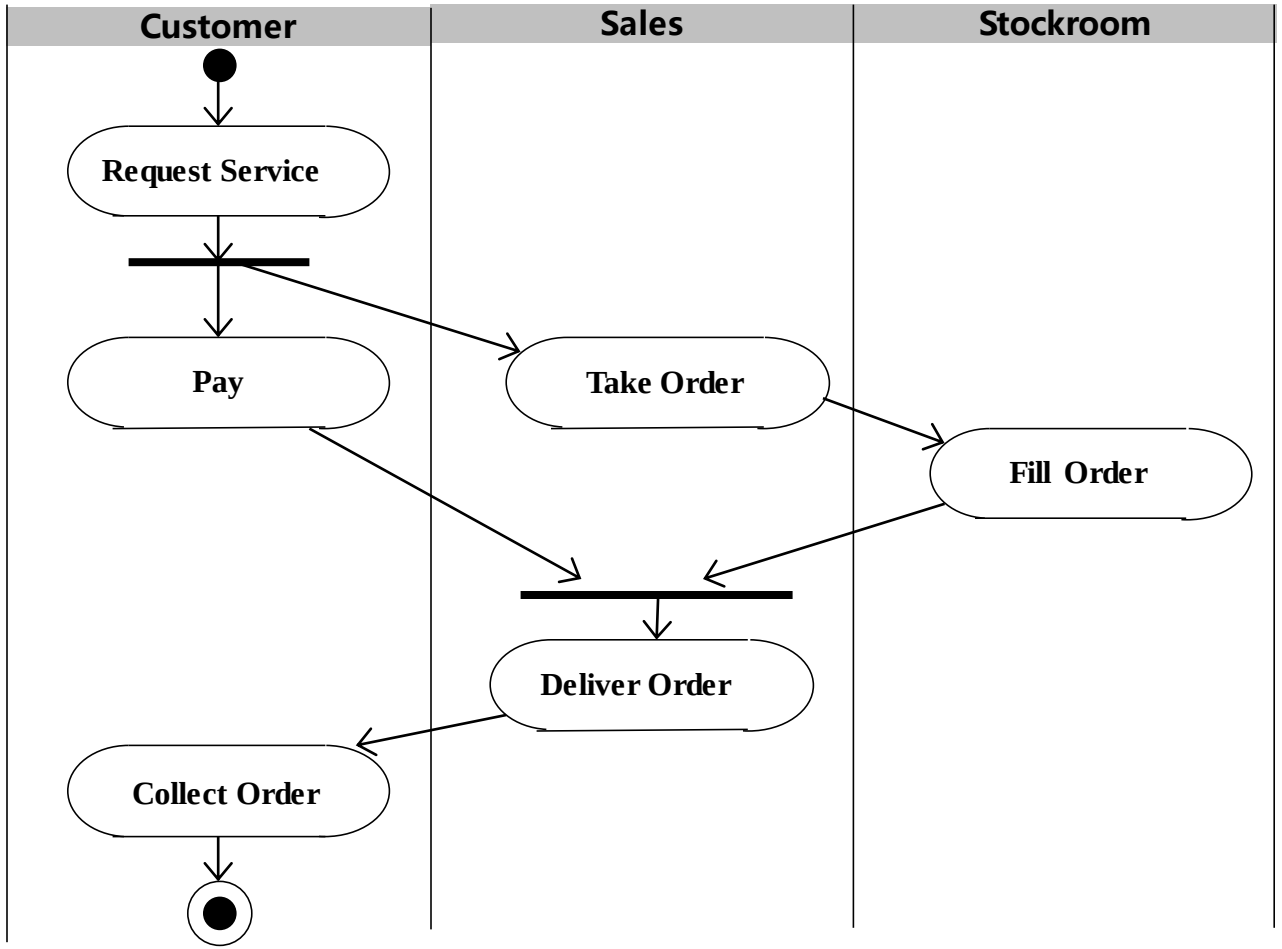
- 动作状态，没有内部转移和内部活动，是原子的最小执行单元，作用是表示执行进入动作后转向的状态，执行时间可以被忽略。
- 活动状态，可分解的、非原子的，并有一定持续时间的执行单元。

软件工程-软件需求分析与设计

- 活动图中的“泳道” swimlane

根据每项活动的职责，划分所有活动的责任区域

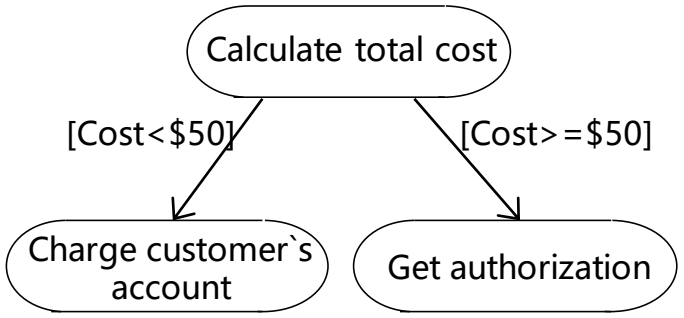
例：



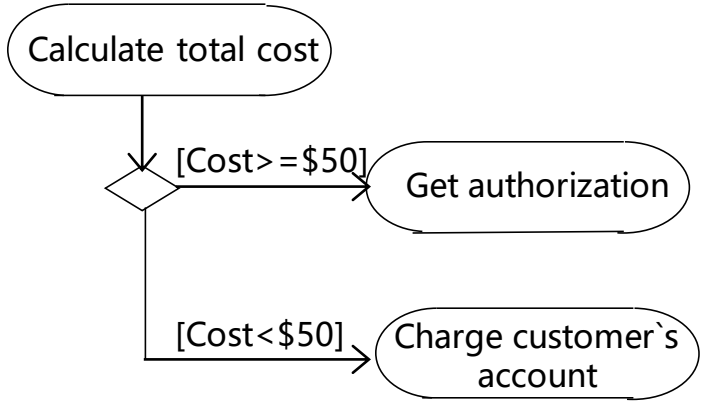
● 活动图中的“分支” branch

在活动图中同一出触发事件，可根据警戒条件转向不同的活动，有两种表示方法：

例：



表示法1

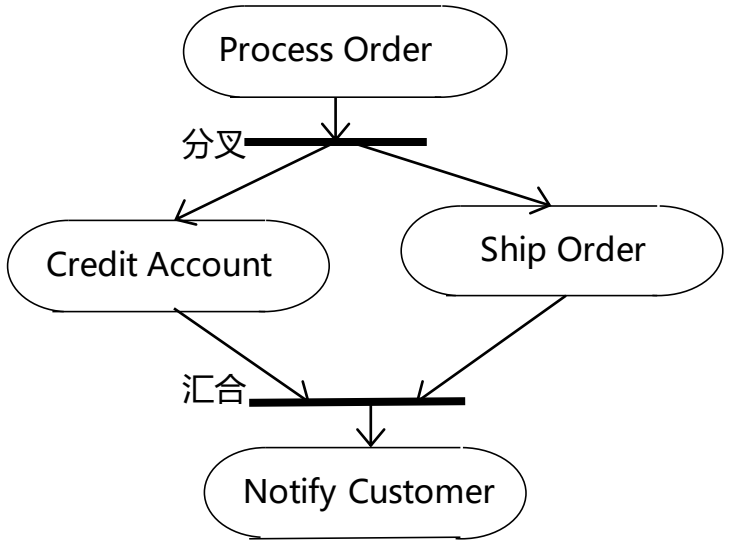


表示法2

● 活动图中的“分叉和汇合” fork and join

在活动图中同一控制流被两个或多个控制流替代且并发，其转换点为分叉；汇合则与此过程相反。

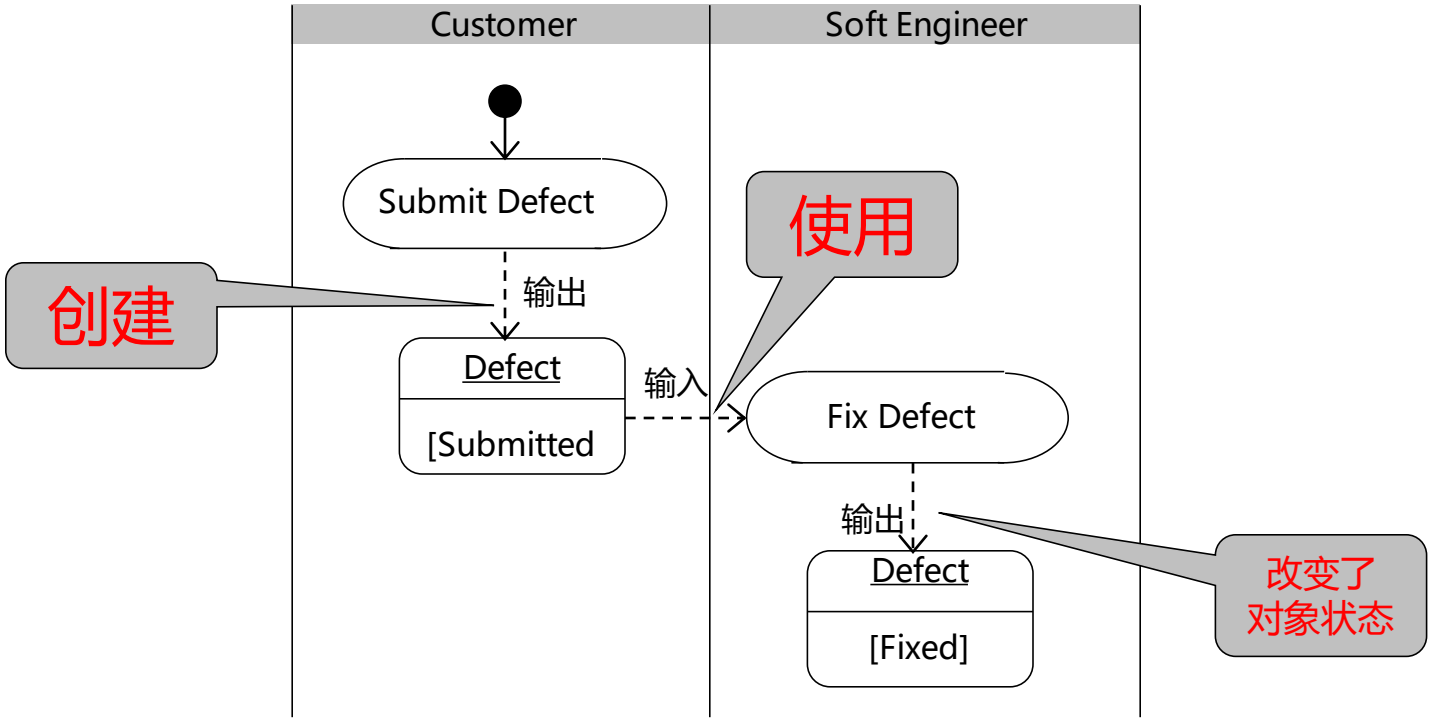
例：



软件工程-软件需求分析与设计

- **活动图中的“对象流”**

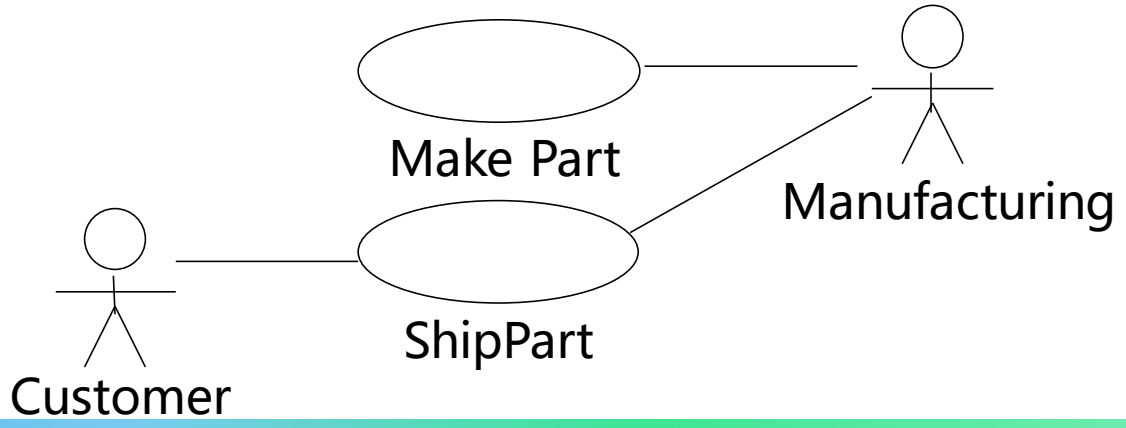
对象作为活动的输入或输出，对象流可表示对象与活动之间的行为，对象流属于控制流。



活动图应用

- 用活动图对工作流建模
  - 工作流建模：系统的业务过程的描述
  - 用例图的局限性：用例图是以系统的各独立功能为单位描述的，并不涉及到整体业务过程，在进行系统分析时，有时需要对业务过程进行必要的描述。

例：产品制造和发货过程在用例图中无法表示



实现建模

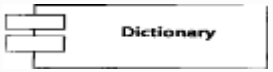
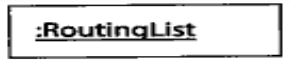
- 构件和部署基本概念
- 构件图
- 部署图

软件工程-软件需求分析与设计

构件图概要

构件图用于静态建模，是表示构件类型的组织以及各种构件之间依赖关系的图。
构件图通过对构件间依赖关系的描述来估计对系统构件的修改给系统可能带来的影响。

构件图中的事物及解释

事物名称	含义	图例
构件	指系统中可替换的物理部分，构件名字(如图中的Dictionary)标在矩形中，提供了一组接口的实现。	
接口	外部可访问到的服务(如图中的Spell-check)。	
构件实例	节点实例上的构件的一个实例，冒号后是该构件实例的名字(如图中的RoutingList)。	

可替换的物理部分包括软件代码、脚本或命令行文件，也可以表示运行时的对象，文档，数据库等。
节点(node)是运行时的物理对象，代表一个计算机资源。具体请参见教程“部署图(deployment diagram)”部分。

构件图中的关系及解释

关系名称	含义	图例
实现关系	构件向外提供的服务。	
依赖关系	构件依赖外部提供的服务(由构件到接口)。	

软件工程-软件需求分析与设计

● 构件和部署基本概念

◆ 系统设计工具

构件图Component Diagram 与部署图Deployment Diagram 是在系统设计时，用来表示系统软件成分以及之间关系结构的工具。

◆ 物理事物建模

分析构件及其间的关系，并对它们在运行节点上的成分给与描述，也叫“物理事物建模”。

软件工程-软件需求分析与设计

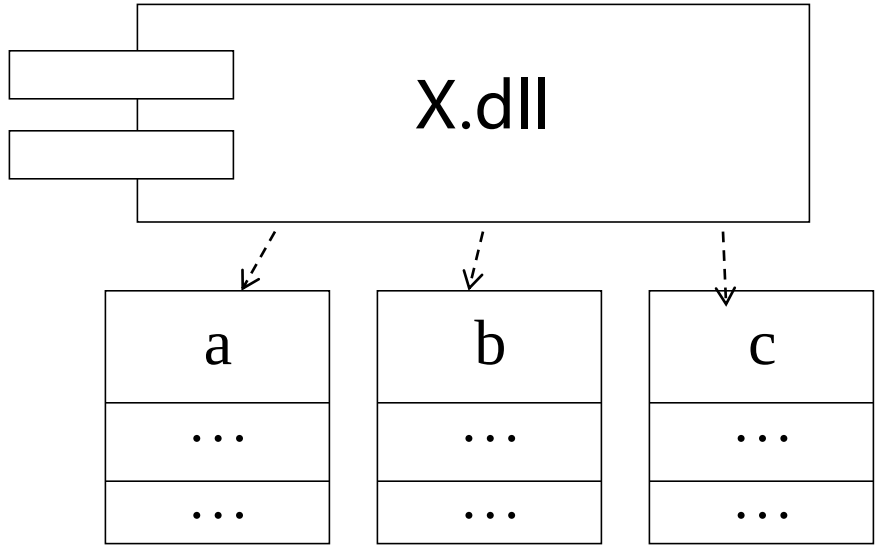
- **构件图** Component Diagram

➢ UML中的构件：

提供单个或组接口，物理上可替换的软件实现单元



➢ 构件与类之间的关系：



软件工程-软件需求分析与设计

- **构件与类差别：**

  - 类表示逻辑抽象，是逻辑模块；
  - 构件表示机器空间中的物理模块，是逻辑元素及协作关系的物理实现；
  - 类有属性和操作；
  - 构件仅通过接口向外提供可请求的操作。

部署图 Deployment Diagram

- 部署图

表示系统在一个或多个物理节点上运行的拓扑结构。物理节点是可部署构件的处理器或设备。

- 节点

是具有独立存储空间，运行时存在，并代表一项计算资源的物理元素和执行能力。

节点表示法：



软件工程-软件需求分析与设计

- 节点和构件

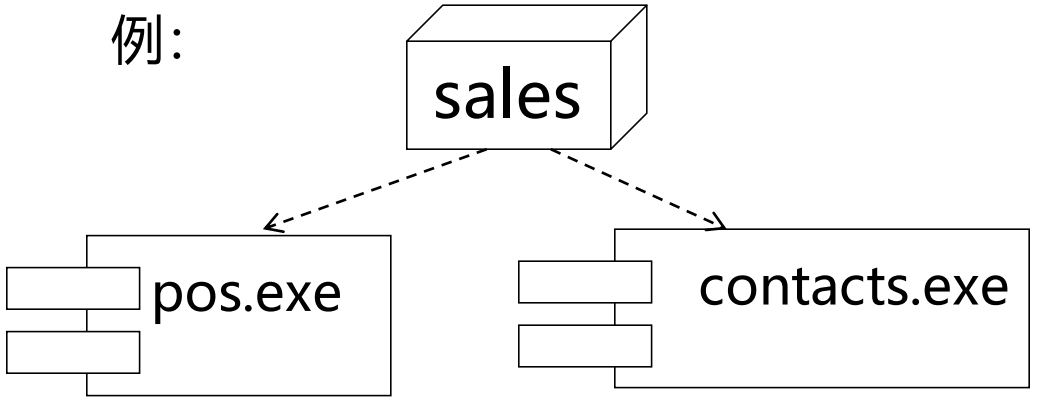
①构件是系统执行的事物，节点是执行构件的事物。

②构件代表逻辑元素的物理打包，节点可表示构件的物理部署。

- 节点和构件的关系：

节点上可以有一个或多个构件，一个构件也可以部署在一个或多个节点上。

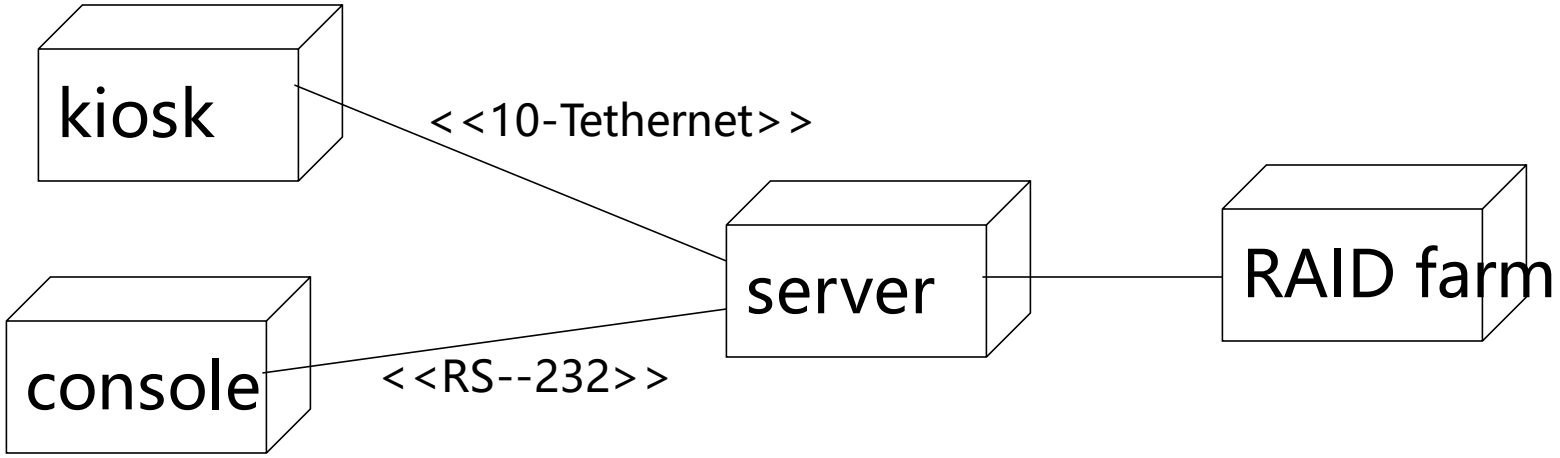
例：



软件工程-软件需求分析与设计

- 节点之间的关联关系：用来表示节点之间的物理连接。

节点之间的连接示例：



软件设计

软件工程-软件需求分析与设计

概述软件设计是软件开发的关键步骤，直接影响软件质量。软件设计阶段要解决“如何做”的问题。

一、软件设计阶段的任务与目标

设计任务：将需求阶段获得的需求说明（模型）转换为计算机中可实现的系统。设计阶段主要任务是:

软件体系结构设计

数据结构设计

界面设计

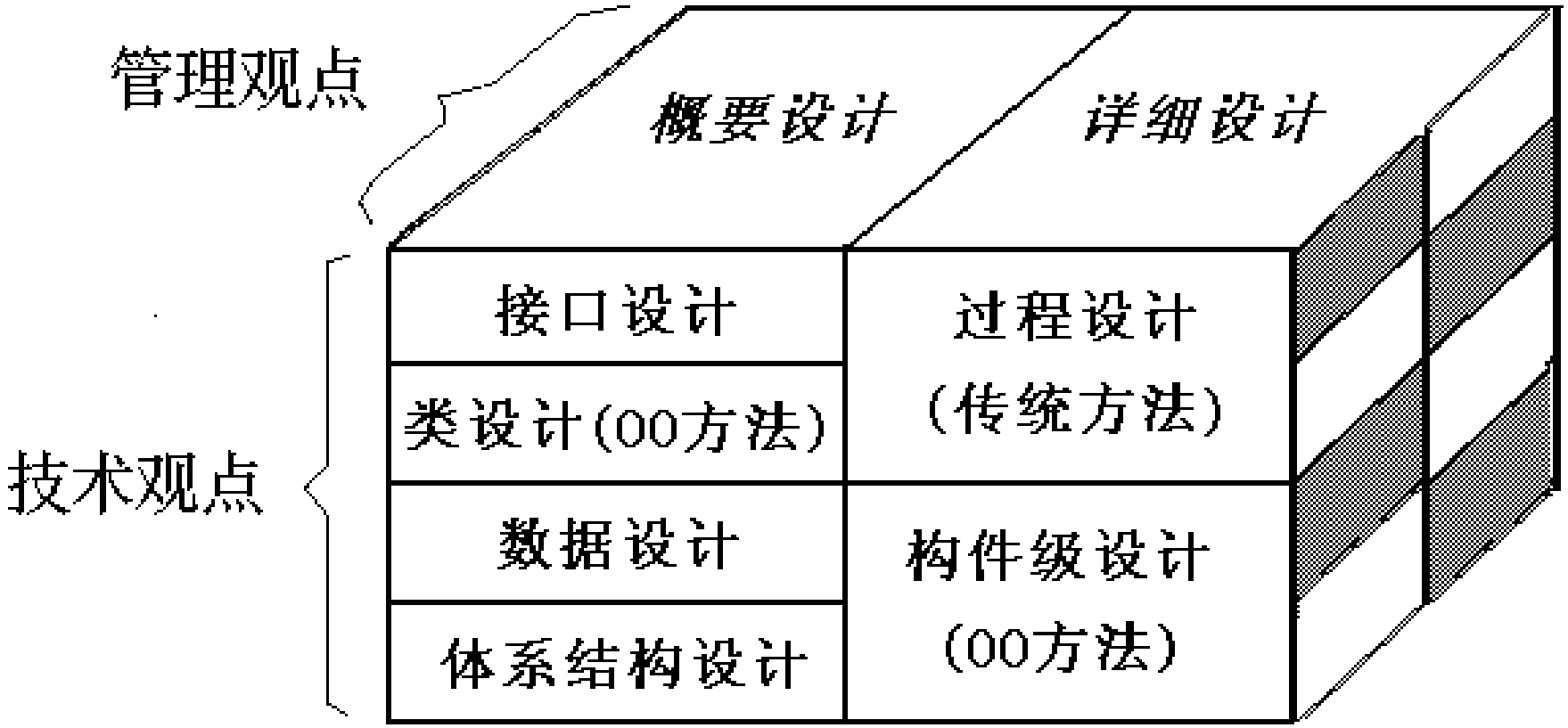
过程设计

软件工程-软件需求分析与设计

设计阶段：

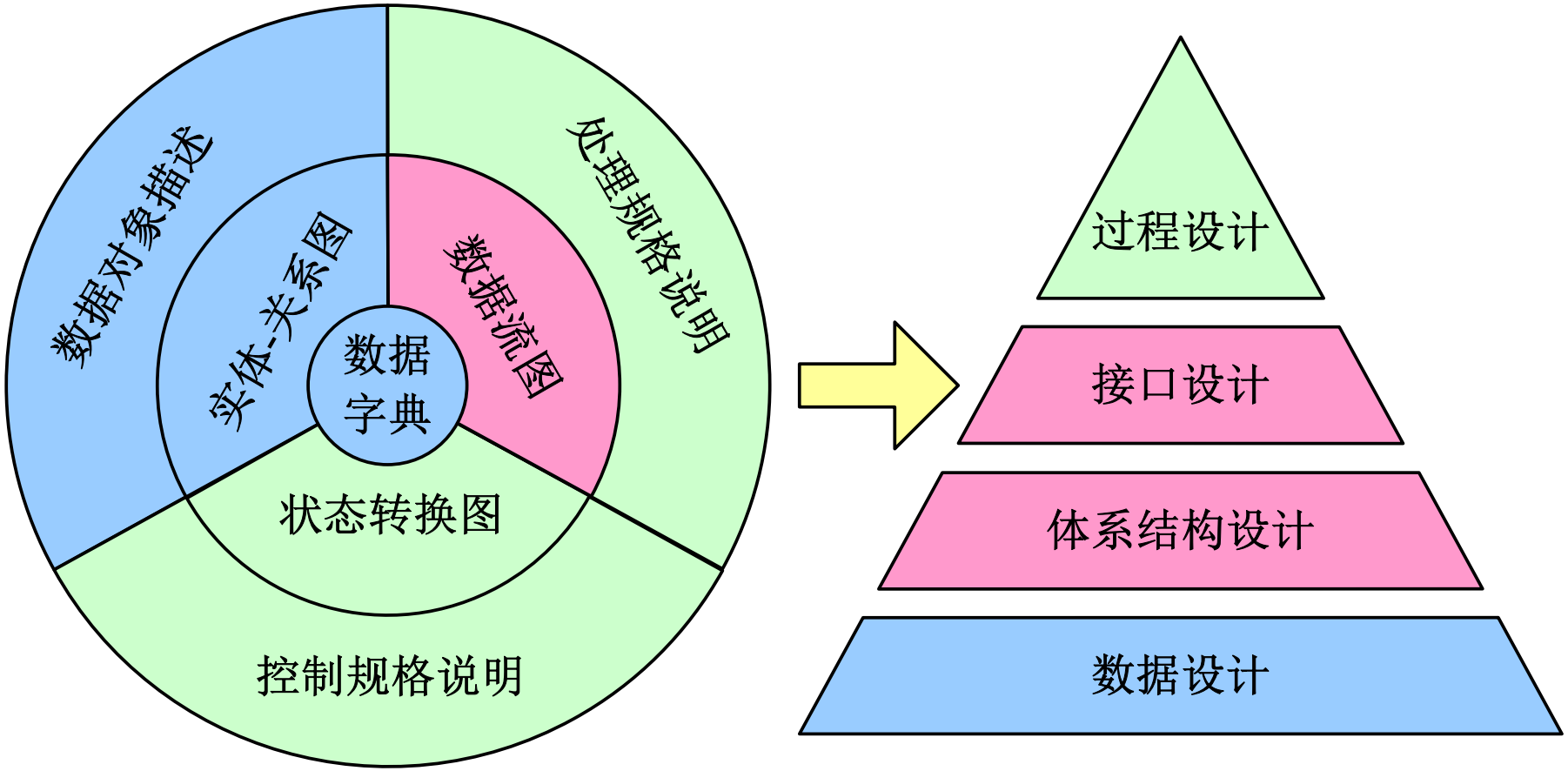
- 从工程管理的角度，可以将软件设计分为**概要设计**阶段和**详细设计**阶段。
- 从技术的角度，传统的结构化方法将软件设计划分为体系结构设计、数据设计、接口设计和过程设计4部分。
- 面向对象方法则将软件设计划分为体系结构设计、类设计/数据设计、接口设计和构件级设计4部分。

软件工程-软件需求分析与设计



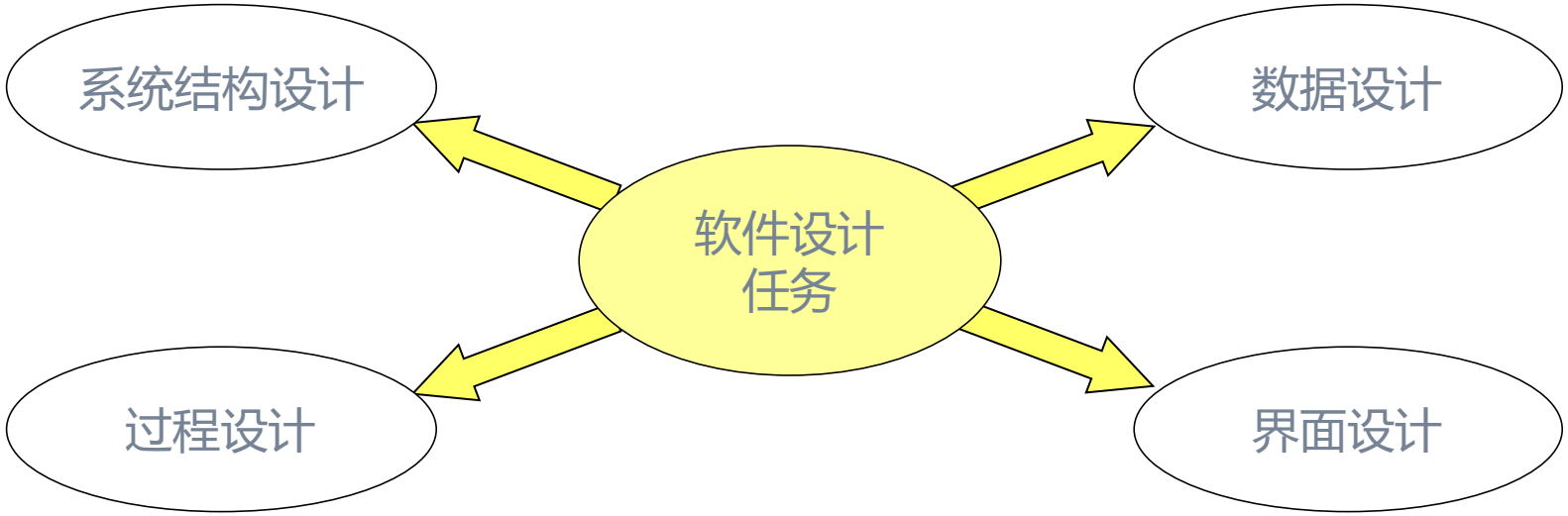
软件工程-软件需求分析与设计

- 结构化设计和结构化分析的关系：



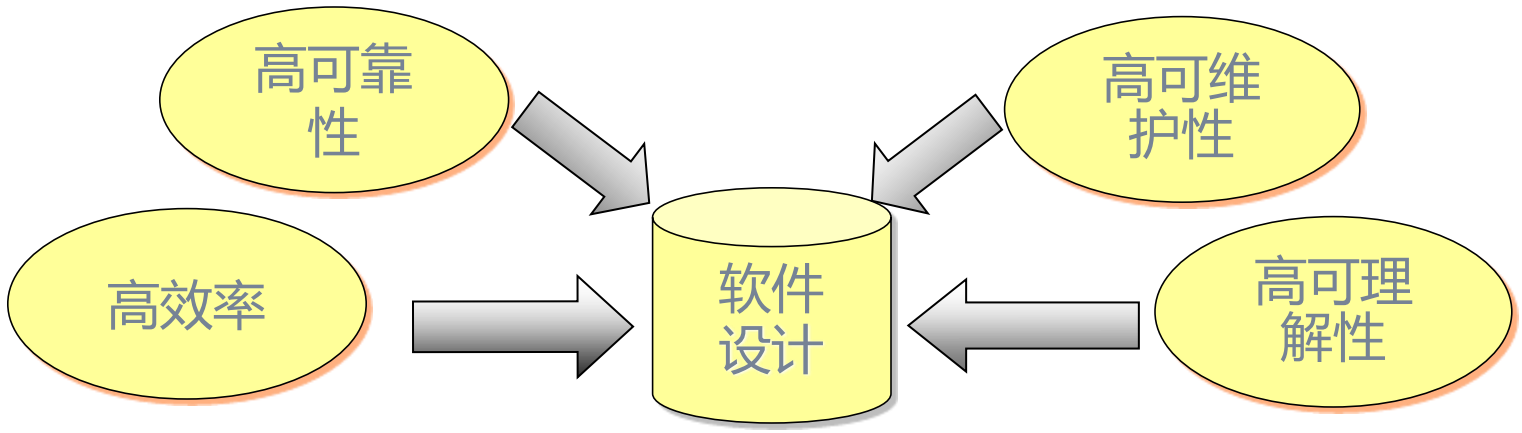
软件工程-软件需求分析与设计

- 软件设计任务涉及多方面，可分为“**总体设计**”和“**详细设计**”。



软件工程-软件需求分析与设计

- 软件**设计的目标** 构造一个“**高内聚低耦合**”的软件模型。



软件工程-软件需求分析与设计

软件设计的划分：

(1)总体设计 (概要设计)

确定软件的结构以及各组成成分(子系统或模块)之间的相互关系。

(2)技术设计 (详细设计)

确定模块内部的算法和数据结构，产生描述各模块程序过程的详细文档。

软件工程-软件需求分析与设计

- **总体设计过程：**首先寻找实现目标系统的各种不同的方案；然后分析员从这些供选择的方案中选取若干个合理的方案，从中选出一个最佳方案向用户和使用部门负责人推荐；分析员应该进一步为这个最佳方案设计软件结构，进行必要的数据库设计，确定测试要求并且制定测试计划。
- **必要性：**总体设计可以站在全局高度上，花较少成本，从较抽象的层次上分析对比多种可能的系统实现方案和软件结构，从中选出最佳方案和最合理的软件结构，从而用较低成本开发出较高质量的软件系统。

软件工程-软件需求分析与设计

典型的总体设计过程包括下述9个步骤：

1. 设想供选择的方案

•根据需求分析阶段得出的数据流图考虑各种可能的实现方案，力求从中选出最佳方案。

2. 选取合理的方案

•从前一步得到的一系列供选择的方案中选取若干个合理的方案。对每个合理的方案分析员都应该准备下列4份资料：

- 系统流程图；组成系统的物理元素清单；
- 成本/效益分析；实现这个系统的进度计划。

软件工程-软件需求分析与设计

3. 推荐最佳方案

•分析员应该综合分析对比各种合理方案的利弊，推荐一个最佳的方案，并且为推荐的方案制定详细的实现计划。

4. 功能分解

•首先进行结构设计，然后进行过程设计。

•结构设计确定程序由哪些模块组成，以及这些模块之间的关系；过程设计确定每个模块的处理过程。

•结构设计是总体设计阶段的任务，过程设计是详细设计阶段的任务。

软件工程-软件需求分析与设计

5. 设计软件结构

•通常程序中的一个模块完成一个适当的子功能。应该把模块组织成良好的层次系统。软件结构可以用层次图或结构图来描绘。

•如果数据流图已经细化到适当的层次，则可以直接从数据流图映射出软件结构，这就是面向数据流的设计方法。

6. 设计数据库

•对于需要使用数据库的那些应用系统，软件工程师应该在需求分析阶段所确定的系统数据需求的基础上，进一步设计数据库。

软件工程-软件需求分析与设计

7. 制定测试计划

•在软件开发的早期阶段考虑测试问题，能促使软件设计人员在设计时注意提高软件的可测试性。

8. 书写文档

•应该用正式的文档记录总体设计的结果，在这个阶段应该完成的文档通常有下述几种： (1) 系统说明； (2) 用户手册； (3) 测试计划； (4) 详细的实现计划； (5) 数据库设计结果。

9. 审查和复审

•最后应该对总体设计的结果进行严格的技术审查和管理复审。

软件工程-软件需求分析与设计

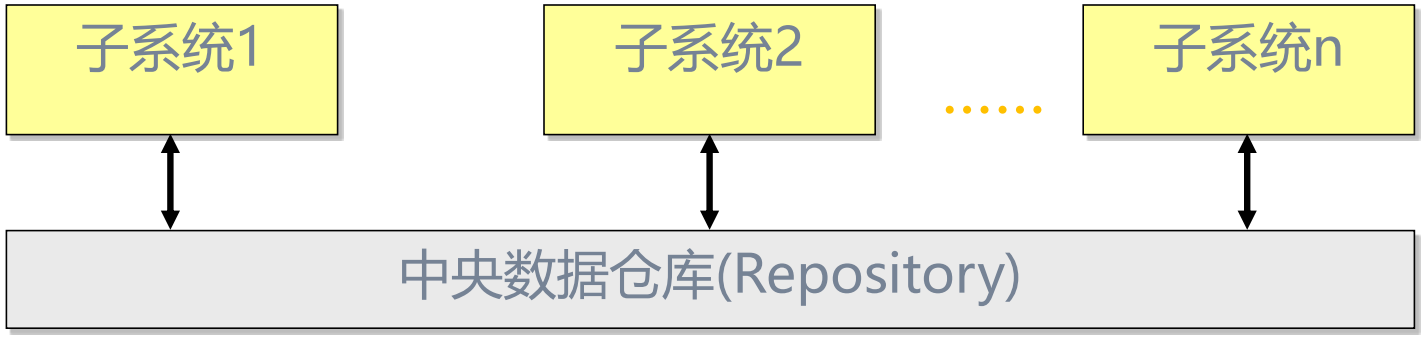
- 软件体系结构确定了系统的组织结构和拓扑结构，显示了系统需求和构成系统的元素之间的对应关系，提供了一些设计决策的基本原理。
- 体系结构的设计过程的主要活动：
  - 1. 系统分解—将系统分解为若干相互作用的子系统。
  - 2. 控制建模—建立系统各部分间控制关系的一般模型。
  - 3. 模块分解—将子系统进一步划分为模块。

软件工程-软件需求分析与设计

- 体系结构设计是软件设计的第一个阶段，该阶段侧重于系统“**宏观**”结构的设计，而不关心模块的内部算法。

一、仓库模型（The repository model）

也称“容器模型”，是一种集中式的模型。各子系统可以直接访问中央数据仓库存储的共享数据。子系统之间紧密耦合。



仓库结构

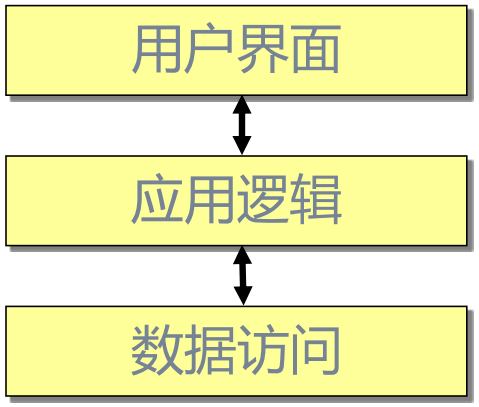
软件工程-软件需求分析与设计

二、 客户机/服务器模型（Client/Server Architectural Model）

C/S结构是一种分布式模型，采用发请求、得结果的模式：

- 客户机 向服务器发出请求(数据请求、网页请求、文件传输请求等等)，
- 服务器 响应请求，进行相应的操作，将结果回传给客户机，客户机再将格式化后的结果呈现给用户。

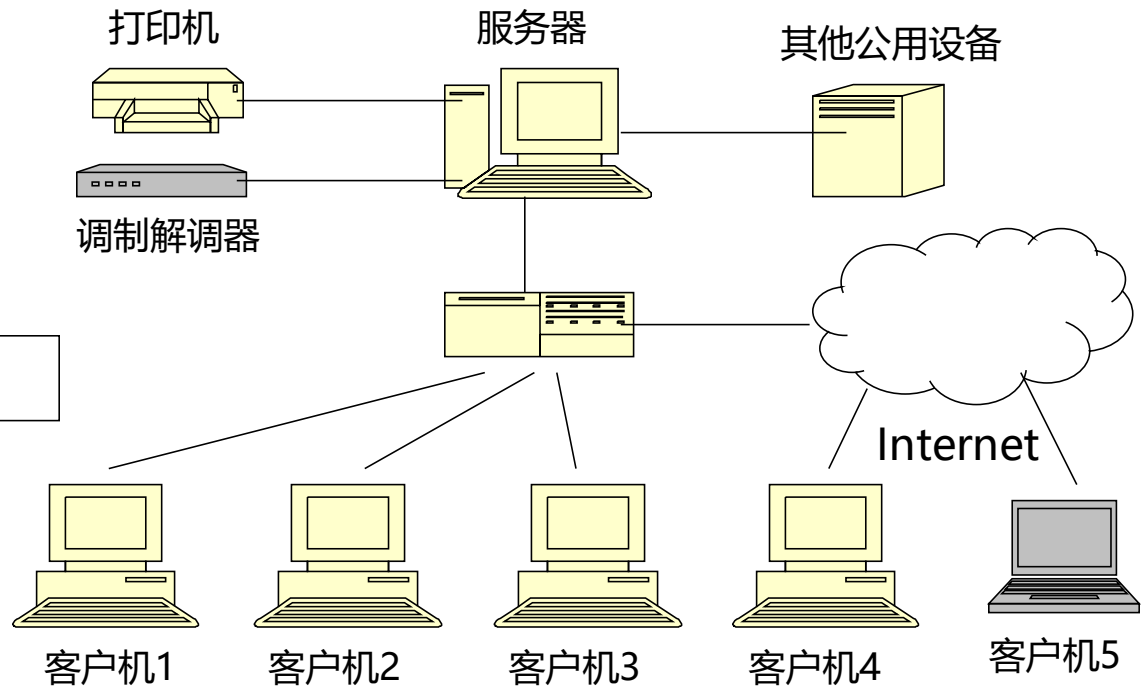
C/S结构的应用都由三个相对独立的逻辑部分组成：



三种逻辑之间的关系

软件工程-软件需求分析与设计

C/S体系结构硬件示意图



- 客户机是具有较强性能的微机系统，在自身操作系统的控制下，执行着运行在其内部的应用系统，并且向服务器发送消息，以完成文件存取或数据库访问等服务。
- C/S结构可实现分布在多个营业点的大型分布式业务系统。

软件工程-软件需求分析与设计



- 浏览器/服务器(Browser/Server)

随着Internet技术的发展，大量的应用需要在广域网络上运行，而C/S体系结构不适合广域网应用需求：

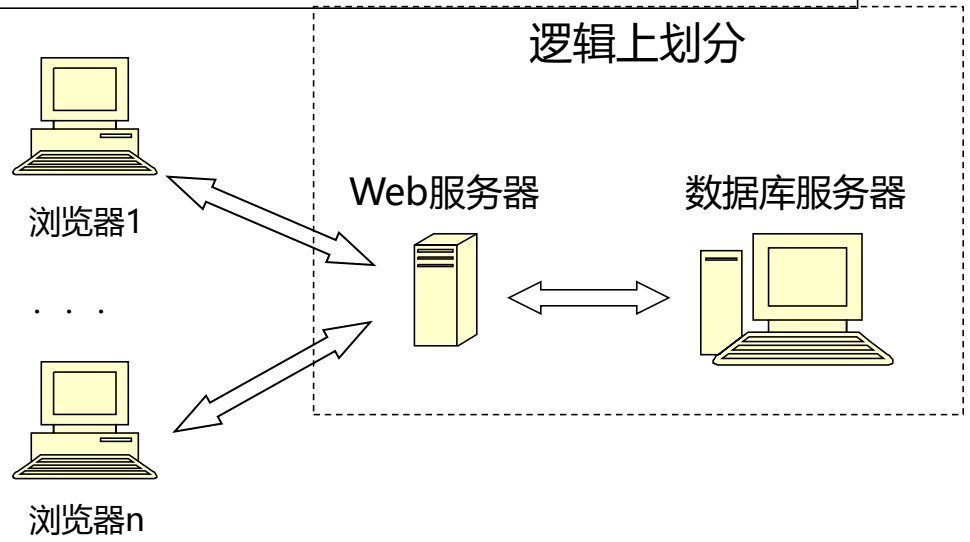
●以特定服务器局域网为中心的频繁的交互操作方式，不能适应多节点传输的广域网应用。

●客户应用系统的操作直接对应数据库信息格式，广域网络的应用系统不能要求必须针对具体的数据库信息格式。

B/S体系结构是在成熟的WWW(World Wide Web)技术基础上，并且，继承了C/S结构的基本形式和内容，从两层结构扩展为三层体系结构。

软件工程-软件需求分析与设计

B/S体系结构逻辑示意



客户浏览器：

浏览器程序

（客户端）

Web服务器：

应用界面处理

应用业务处理

数据库信息转换

数据库服务器：

数据读写

数据检索和更新

软件工程-软件需求分析与设计

B/S体系结构存在的问题

●操作交互处理和业务处理逻辑紧密相关，不能单独修改或复用。

●业务处理的输出信息是以全体共识的方式直接发往客户端的，所以外界可直接了解系统的数据结构，安全性差。

●Web服务器权力过于集中，处理业务加上处理传输，使负载过大，系统处理中回避风险的能力降低。

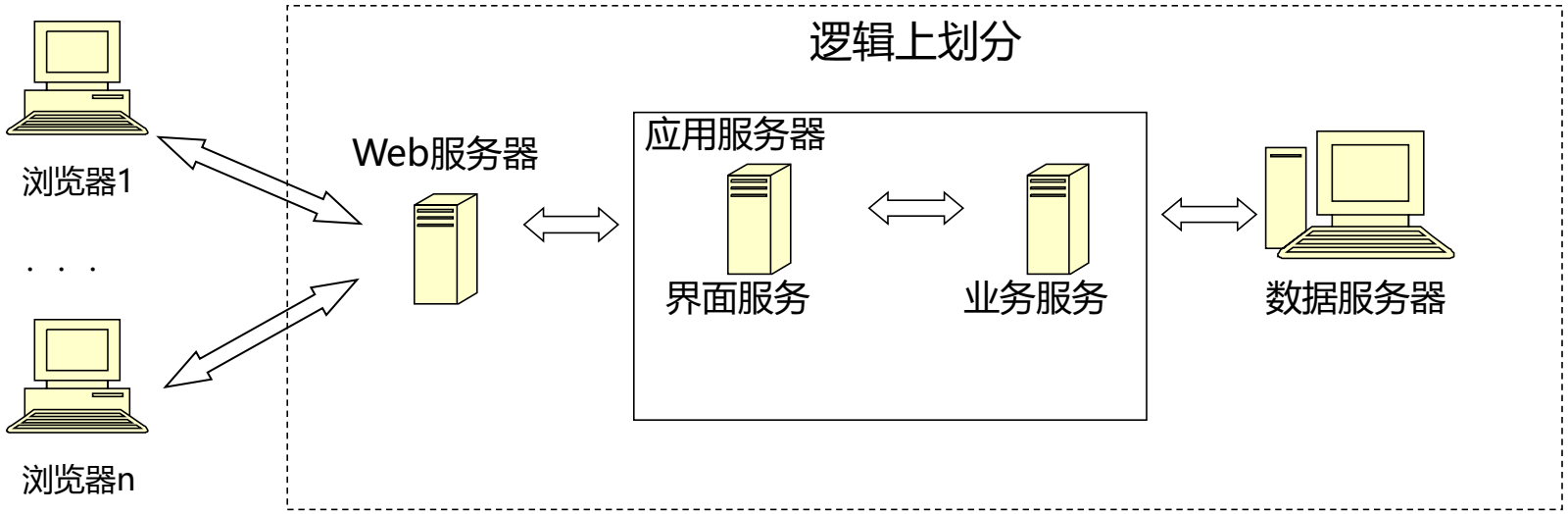
B/S多层体系结构

将Web服务器分为两层或多层的结构。

Web服务器：	应用服务器：	数据服务器：
界面规范化 数据校验	业务处理 数据访问	数据访问

软件工程-软件需求分析与设计

B/S多层体系逻辑结构



三、分布式对象结构(Distributed Objects Architecture)

在C/S模型中，客户和服务器在服务/请求上的差别，在一定程度上限制了系统的灵活性和可扩展性。

采用分布式对象结构：

- “对象(Object)”——提供服务的系统组件(System component)。
- 每个对象在逻辑上是平等的，它们可以互相为对方提供所需的服务。
- 提供服务的对象就是服务器，而提出服务请求的对象就是客户。

CORBA EJB COM/DCOM

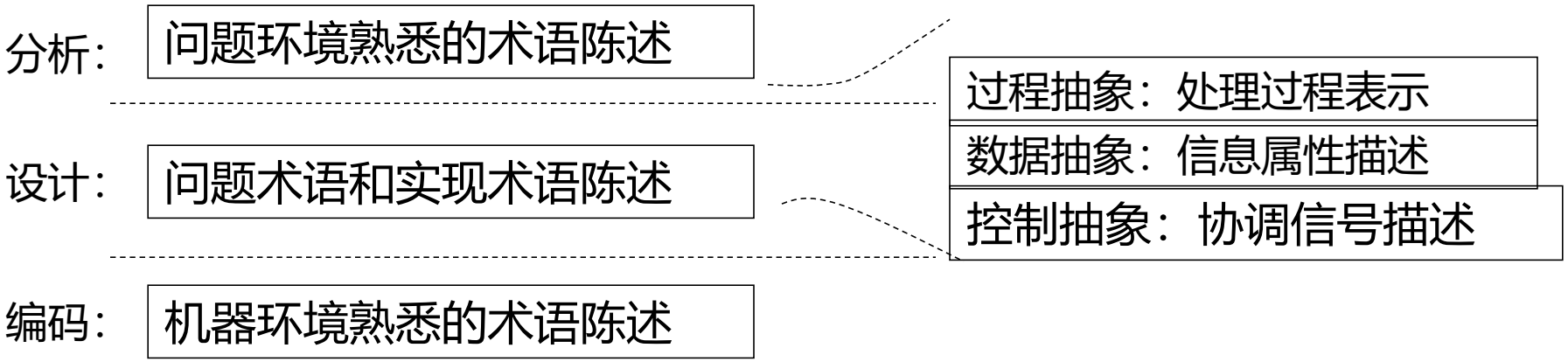
软件工程-软件需求分析与设计

1）抽象：

集中于问题的一般性的概念，而忽略无关的细节。

2）逐步求精：

进一步详细描述的过程，与抽象可达到互补的作用。



软件工程-软件需求分析与设计

- 抽象：现实世界中一定事物、状态或过程之间总存在着某些相似的方面（共性）。把这些相似的方面集中和概括起来，暂时忽略它们之间的差异，这就是抽象。
- 抽象就是抽出事物本质特性而暂时不考虑细节。
- “抽象是人类处理复杂问题的基本方法之一。”——Grady Boach

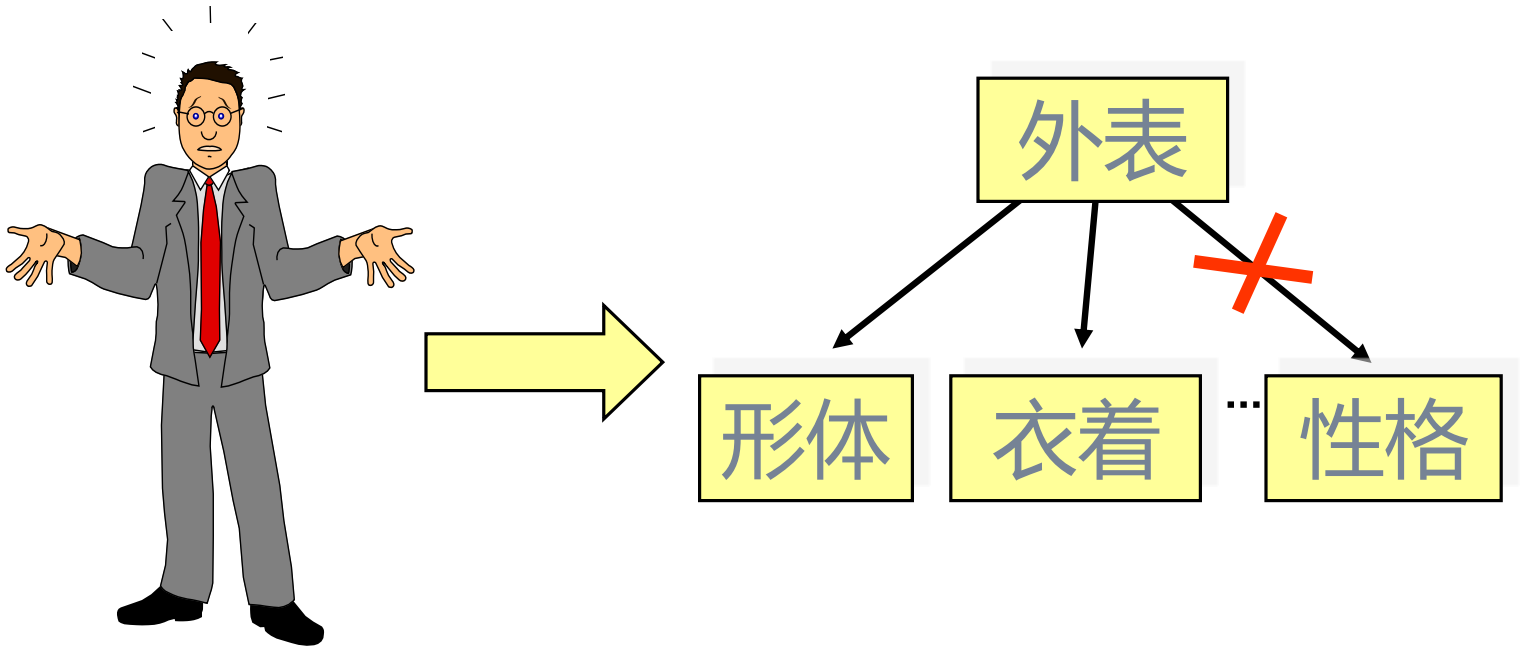
软件工程-软件需求分析与设计

●一般抽象过程：

- 处理复杂系统的惟一有效的方法是用层次的方式构造和分析它。
- 一个复杂的动态系统首先可以用一些高级的抽象概念构造和理解，这些高级概念又可以用一些较低级的概念构造和理解，如此进行下去，直至最低层次的具体元素。
- 例：过程抽象、数据抽象

开(行为抽象） + 门(数据抽象)

抽象



软件工程-软件需求分析与设计

软件工程抽象过程：

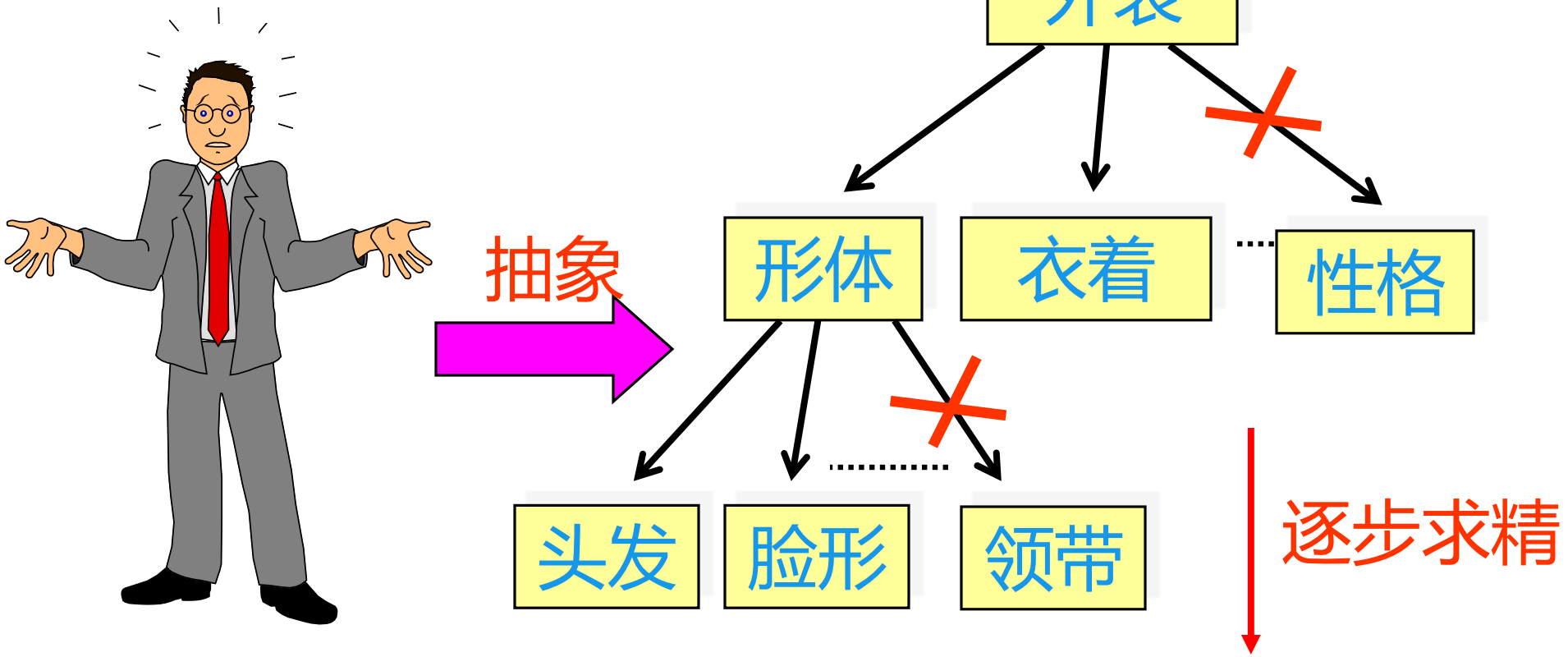
- 软件工程过程的每一步都是对软件解法的抽象层次的一次精化。
- 在可行性研究阶段，软件作为系统的一个完整部件；
- 在需求分析期间，软件解法是使用在问题环境内熟悉的方式描述的；
- 当由总体设计向详细设计过渡时，抽象的程度也就随之减少了；
- 最后，当源程序写出来以后，也就达到了抽象的最低层。

软件工程-软件需求分析与设计

- **逐步求精：**为了能集中精力解决主要问题而尽量推迟对问题细节的考虑。逐步求精是人类解决复杂问题时采用的基本方法，也是许多软件工程技术的基础。
- Miller法则：一个人在任何时候都只能把注意力集中在（7±2）个知识块上。

软件工程-软件需求分析与设计

逐步求精的例子



信息隐藏(Information Hiding)

- 信息隐藏的含义：有效的模块化可以通过定义一组独立模块来实现，这些模块相互之间只交流软件功能必需的信息。

- 隐藏实现信息局部化：
  - 把关系密切的软件元素物理地放得彼此靠近。
  - 优点：可维护性好、可靠性好、可理解性好。

- 信息隐藏的目的:
  - 提高模块的独立性，减少修改或维护时的影响面。

- 信息隐藏：应该这样设计和确定模块，使得一个模块内包含的信息（过程和数据）对于不需要这些信息的模块来说，是不能访问的。

  局部化：局部化的概念和信息隐藏概念是密切相关的。所谓局部化是指把一些关系密切的软件元素物理地放得彼此靠近。显然，局部化有助于实现信息隐藏。

软件工程-软件需求分析与设计

- **信息隐藏**和**局部化**的作用：

  “隐藏”意味着有效的模块化可以通过定义一组独立的模块而实现，这些独立的模块彼此间仅仅交换那些为了完成系统功能而必须交换的信息。

  使用信息隐藏原理作为模块化系统设计的标准就会带来极大好处。因为绝大多数数据和过程对于软件的其他部分而言是隐藏的，在修改期间由于疏忽而引入的错误就很少可能传播到软件的其他部分。

软件工程-软件需求分析与设计

模块化(Modularity)： 好的软件设计的一个基本准则。

高层模块 整体上把握问题,隐藏细节

复杂问题 → 分解 → 较小问题

较小问题 → 分解

可减小解题所需的总的工作

模块独立性含义：

- 模块完成独立的功能；
- 符合信息隐蔽和信息局部化原则；
- 模块间关连和依赖程度尽量小。

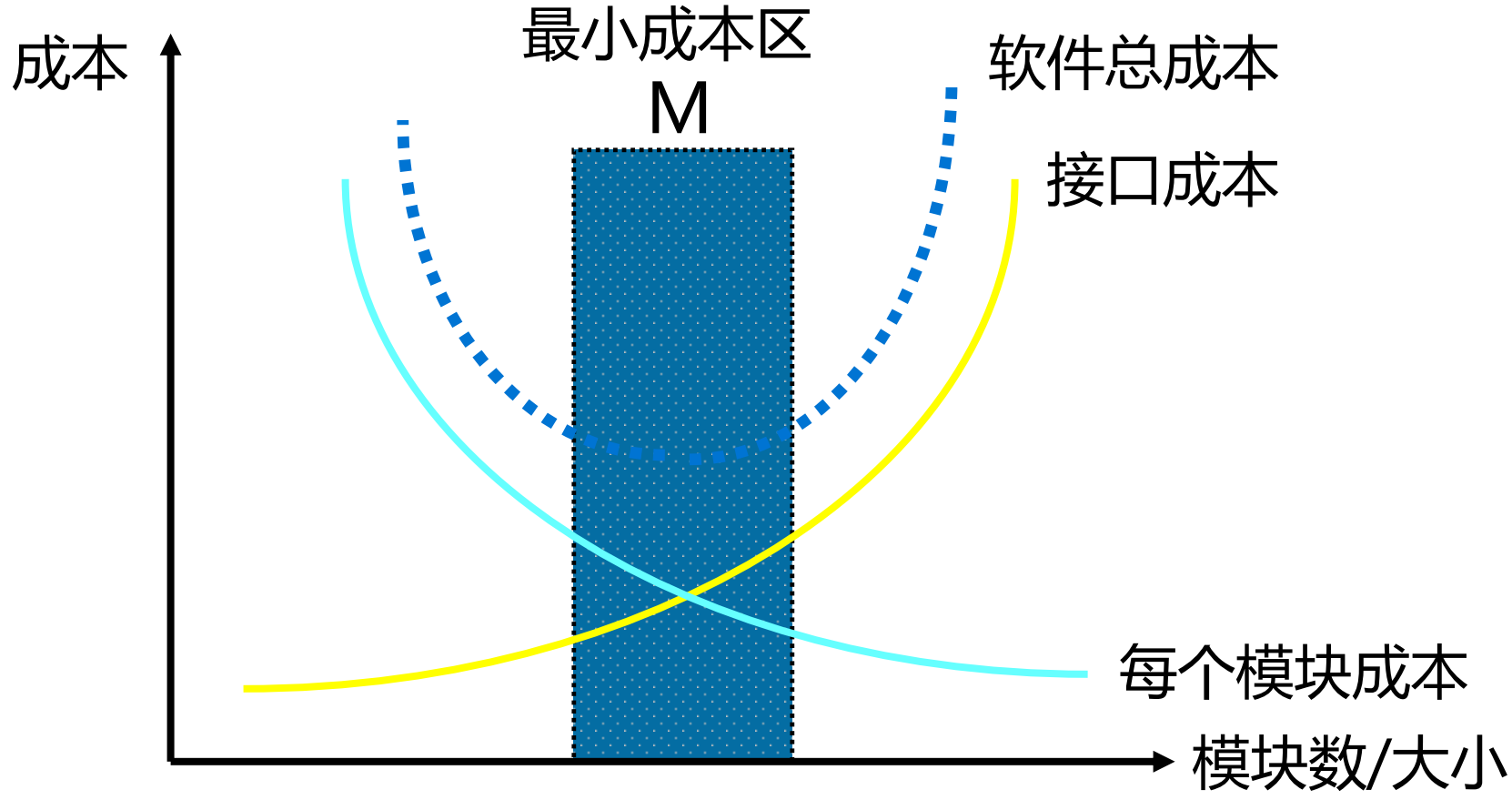
软件工程-软件需求分析与设计

- **模块**：是由边界元素限定的相邻程序元素的序列，而且有一个总体标识符代表它。
- **模块化**：就是把程序划分成独立命名且可独立访问的模块，每个模块完成一个子功能，把这些模块集成起来构成一个整体，可以完成指定的功能满足用户的需求。

模块化的作用：

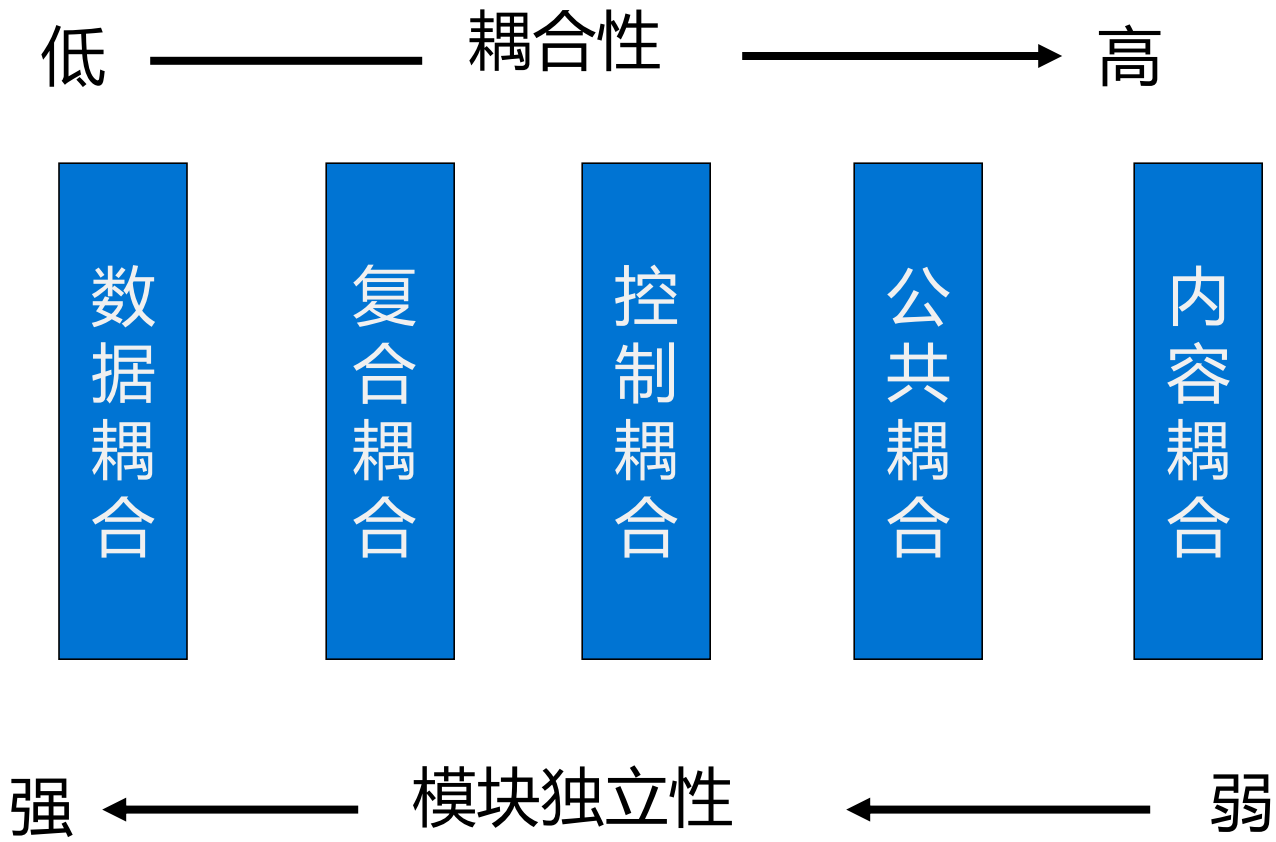
- 采用模块化原理可以使软件结构清晰，不仅容易设计也容易阅读和理解。
- 模块化使软件容易测试和调试，因而有助于提高软件的可靠性。
- 模块化能够提高软件的可修改性。
- 模块化也有助于软件开发工程的组织管理。

模块数目、大小与软件成本关系



软件工程-软件需求分析与设计

- 耦合类型与关系

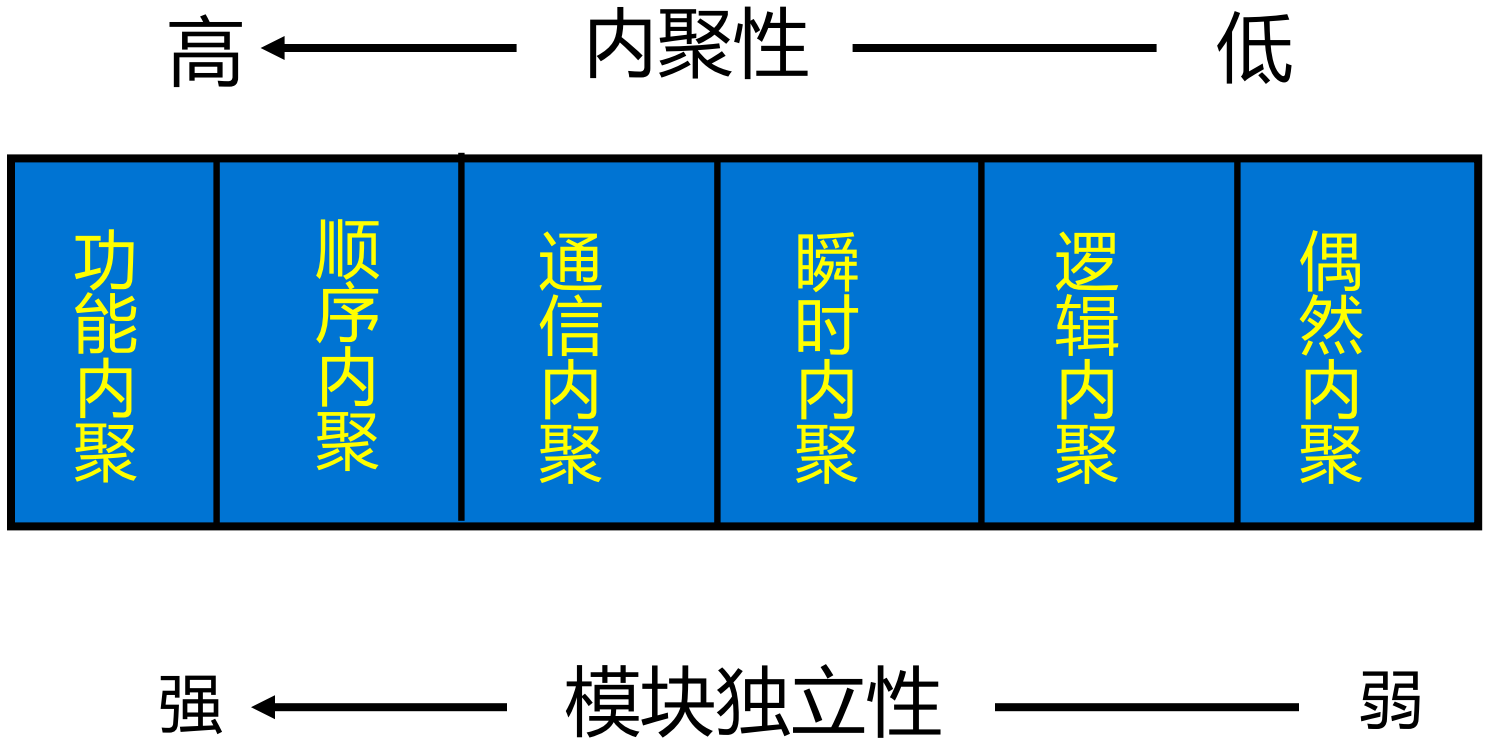


软件工程-软件需求分析与设计

- 模块独立性的度量--内聚性
- 内聚性:一个模块内部各成分之间相互关联的强度
- 设计目标：高内聚

软件工程-软件需求分析与设计

- 内聚类型与关系



软件工程-软件需求分析与设计

- 内聚与耦合关系：

内聚与耦合密切相关，同其它模块强耦合的模块意味者弱内聚，强内聚模块意味着与其它模块间松散耦合.

设计目标：力争**强内聚**、**弱耦合**

- 内聚、耦合是密切相关的，模块内的高内聚往往意味着模块间的松耦合。内聚和耦合都是进行模块化设计的有力工具，但是实践表明内聚更重要，应该把更多注意力集中到提高模块的内聚程度上。

软件工程-软件需求分析与设计

软件设计文档评审

目的：尽早发现设计中的错误、缺陷，经过修改完善，形成SCM设计阶段的基线；

评审内容：

- 软件总体结构；
- 数据结构；
- 人机界面；
- 接口；
- 出错处理功能；

软件工程-编码

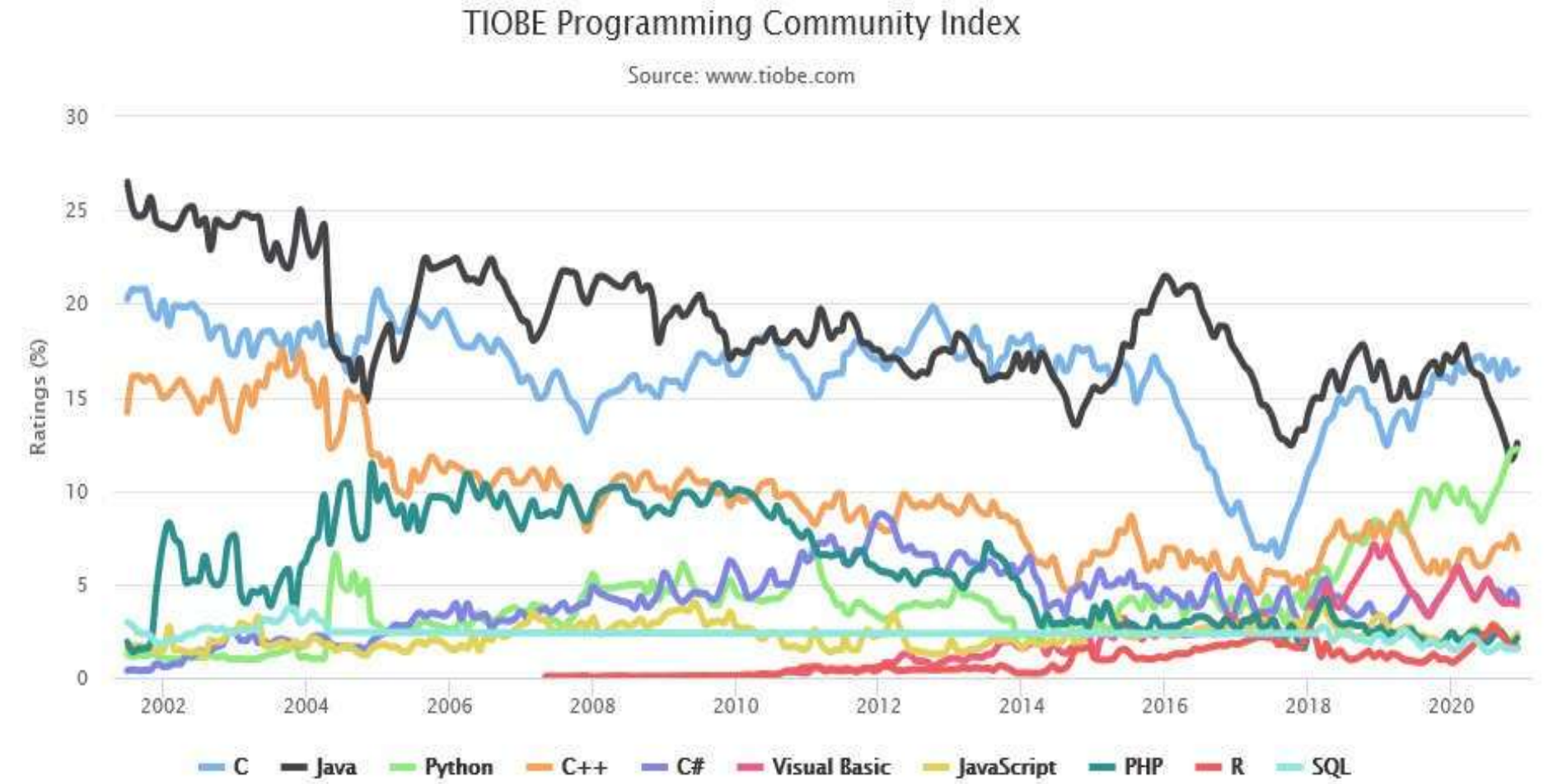
- 程序设计语言
- 程序设计风格
- 代码复用
- 代码审查

软件工程-编码

2021年10月“TIOBE世界编程语言 排行榜” 统计，前20种常用编程语言 的使用概率进行排名，可以了解目前常 用编程语言的使用情况，并展现全球范围内编程语言的应用趋势。

Oct 2021	Oct 2020	Change	Programming Language	Ratings	Change
1	3	↑	Python	11.27%	-0.00%
2	1	↓	C	11.16%	-5.79%
3	2	↓	Java	10.46%	-2.11%
4	4		C++	7.50%	+0.57%
5	5		C#	5.26%	+1.10%
6	6		Visual Basic	5.24%	+1.27%
7	7		JavaScript	2.19%	+0.05%
8	10	↑	SQL	2.17%	+0.61%
9	8	↓	PHP	2.10%	+0.01%
10	17	⇈	Assembly language	2.06%	+0.99%
11	19	⇈	Classic Visual Basic	1.83%	+1.06%
12	14	↑	Go	1.28%	+0.13%
13	15	↑	MATLAB	1.20%	+0.08%
14	9	⇊	R	1.20%	-0.79%
15	12	↓	Groovy	1.18%	-0.05%

软件工程-编码


TIOBE Programming Community Index
Source: www.tiobe.com
Ratings (%)
30
25
20
15
10
5
0
2002
2004
2006
2008
2010
2012
2014
2016
2018
2020
C
Java
Python
C++
C#
Visual Basic
JavaScript
PHP
R
SQL

软件工程-编码

不同的程序语言机制，对设计的支持不尽相同，目前被广泛采用的是结构化程序设计语言和面向对象语言。

结构化程序设计语言机制需考虑到：

- 数据结构的表示
- 算法的模块化分解
- 控制结构

软件工程-编码

面向对象程序设计语言机制需考虑到：

- 类：考虑局部化设计原则。
- 继承性：继承性是使得类自动具有其它类的属性（数据结构）和方法（功能）的机制。
- 多态性：多态性是指相同的模块接口定义，却有完全不同的实现过程。

软件工程-编码

程序设计风格，力图从编码原则的角度来探讨提高程序的可读性、改善程序质量的方法和途径。

- 代码文件
- 数据说明
- 语句结构的处理
- 输入输出设计准则

软件工程-编码

程序设计风格

```
/*************************************************
 * 实验：定义并实现字符串类CMyString，完成常见的字符串操作。
 *************************************************/
#include <iostream>
using namespace std;

// 定义字符串类
class CMyString {
public:
        CMyString(const char* = NULL);              // 可构造无参的字符串
        CMyString(const CMyString&);
        ~CMyString();
        int Find(char, int = 0);                    // 从指定位置开始查找字符
        int Find(const CMyString&);                 // 查找字符串
        int Length();                               // 获取字符串长度
        CMyString Mid(int, int);                    // 获取子串
        char operator[ ](int index);                // 获取指定位置的字符
        CMyString& operator=(const CMyString&);
        CMyString& operator+=(const CMyString&); // 重载加运算符对应的 += 运算符
        friend CMyString operator+(const CMyString&, const CMyString&);
        friend ostream& operator<<(ostream&, const CMyString&);
protected:
        char* m_pString;    // 保存的字符串
        int   m_iSize;      // 字符串长度
};
```

代码文件头部，包括程序运行所需的外部文件。注意文件路径。

类的定义及简要说明

类的成员函数接口定义及简要说明

类的成员属性定义及简要说明

软件工程-编码

只编写够用的注释，重质量而非重数量



- 好的注释解释为什么，而不是怎么样；
- 不要在注释中重复代码；
- 写完注释之后，需要在代码的上下文中回顾一下；
- 当修改代码时，切记维护代码周围的注释；
- 注释不要产生歧义，更不能误导。
- 注释不是目的，最好的、可理解的程序就是代码文件自身。
- 要编写可阅读的代码，代码文件自身就简单易懂。

软件工程-编码

代码评审，也称为代码复查，是指在软件开发过 程中，通过阅读源代码和相关设计文件，对源代码编 码风格、编码标准以及代码质量等活动进行系统性检 查的过程。



代码评审主要分为正式评审和轻量级评审。

软件工程-编码

代码评审——正式评审

正式评审是针对代码编写完成之后召开的评审会议。评审会议由评审　小组组织完成，成员包括组长、评审员、质量过程管理员和程序员，评审　秉承沟通、协作、互助、学习的态度。



评审内容包括：

- 按照软件需求设计代码输入、输出的文档；
- 代码功能描述和性能描述；
- 代码编写标准、接口定义规范；
- 源代码。

```
/*
 * 实现：获取指定起始位置和长度的子串
 * 参数：startPos：指定子串的起始位置；Length：子串的长度
 * 返回：返回获取子串的对象
 */
CMyString CMyString::Mid(int startPos, int Length) {
        // 确保参数的有效性
        if (startPos < 0 || startPos >= m_iSize || Length <= 0) return CMyString("");
        if (Length > m_iSize - startPos) Length = m_iSize - startPos;
        char* pTemp = new char[Length + 1];  // 保存获取的子串
        int i = 0;
        for (i = 0; i < Length; i++)
                pTemp[i] = m_pString[startPos++];
        pTemp[i] = '\0';

        // 不要直接写成：return MyString(pTemp );
        // 否则将无法释放pTemp的内存空间，造成内存泄漏。
        CMyString my(pTemp );
        delete[] pTemp ;
        return my;
}
```

软件工程-编码

代码评审——轻量级评审



轻量级评审主要采取非正式的**代码走查**方式，不需要组织正式 会议，因此它成本小、灵活性强。

轻量级评审方式包括：

- 程序员向评审者提供需评审的代码；
- 程序员可以借助代码自动评审工具；
- 程序员可以与评审员同步进行软件设计和开发；
- 定期召开小型的评审会，共同探讨代码编写过程中遇到的各类问题。

软件工程-编码



代码评审——轻量级中的9项实践方式

1、一次评审少于 200 ~ 400 行的代码。

2、目标为每小时低于 300 ~ 500 LOC 的检查速率。

3、花足够的时间进行正确缓慢的评审，但不要超过60 ~ 90 分钟。

4、确定开发者在评审开始之前就已注释了源代码。

5、为代码评审和获取制度建立可量化的目标，以利于改进流程。

6、使用检查列表。

7、确认缺陷确实得到修复。

8、培养良好的代码评审文化氛围，维护开发团队的团结协作。

9、采用轻量级，能用工具支持的代码评审，例如CodeStriker、IDE。

软件工程-编码

程序员的基本素质

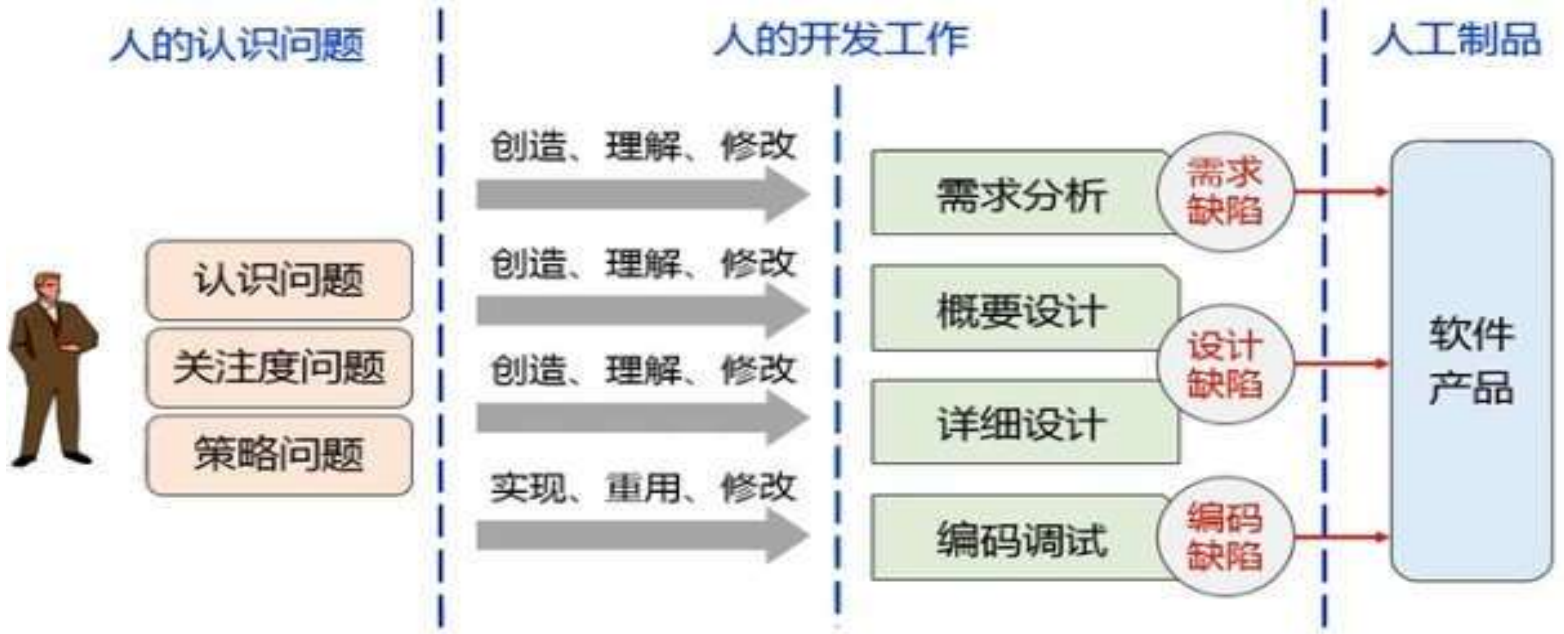
- 团队精神和协作能力;
- 文档的自我管理;
- 规范化、标准化的代码编写习惯;
- 需求理解能力;
- 复用性，模块化思维能力;
- 测试习惯;
- 学习和总结的能力。

软件工程-测试

- 软件测试基础
- 白盒测试
- 黑盒测试
- 白盒测试和黑盒测试的比较
- 测试用例的有效性

软件工程-测试

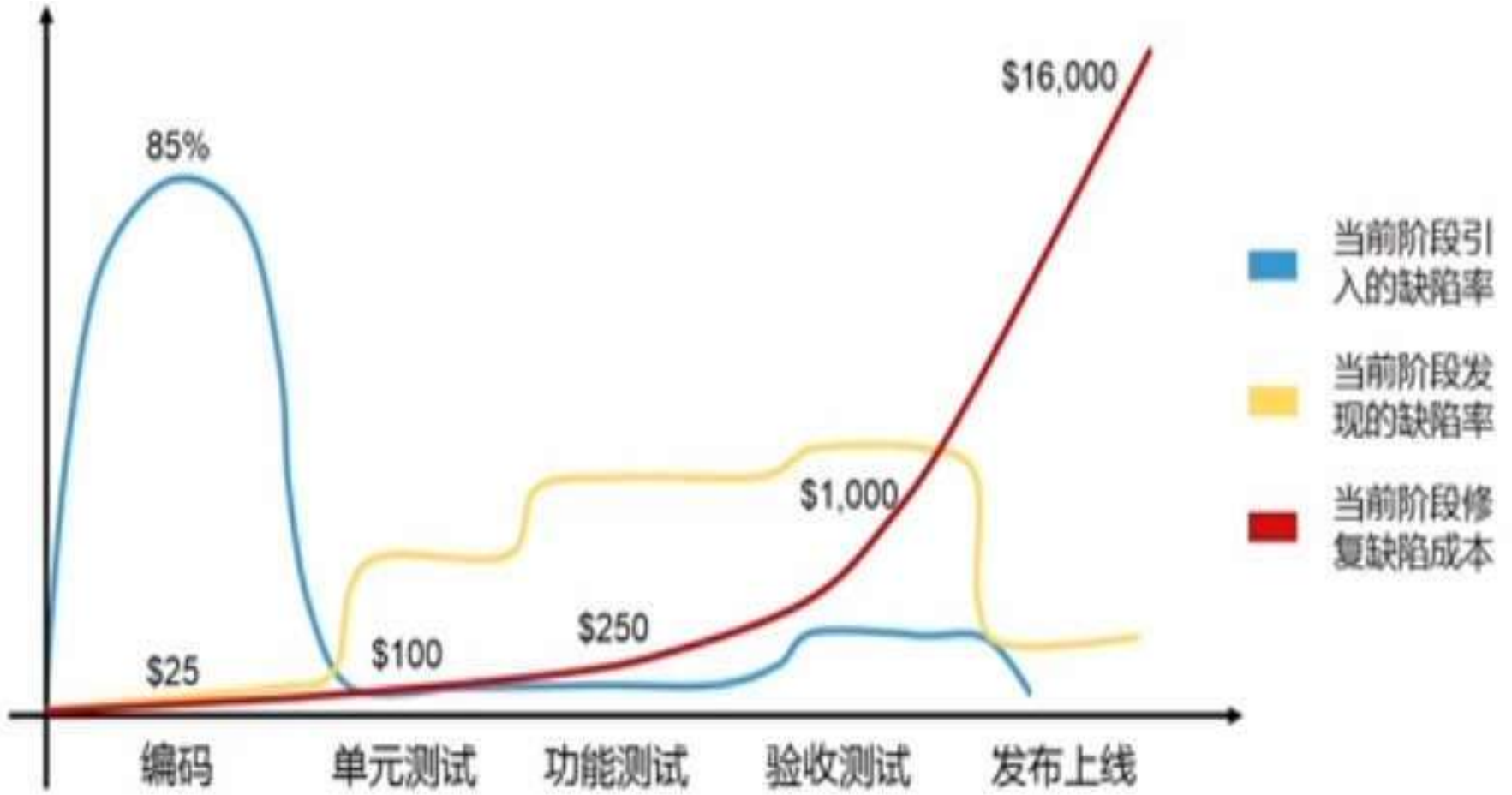
为什么要进行软件测试?



- 软件测试是在软件开发过程中保证软件质量、提高软件可靠性的最主要的手段之一。
- 它是软件系统在正式交付用户使用前，对系统分析、设计、代码等开发工作的最后检查和复审。

软件工程-测试

软件修复成本与软件开发各过程的变化



软件工程-测试

- 对于软件测试的定义，有如下不同的描述：
- IEEE（1983）：使用人工或自动运行测试系统的过程，其目的在于检验系统是否满足用户需求，或找出预期结果与实际运行结果间的差别，发现程序错误。
- Glen Myers：软件测试为了发现错误而执行程序的过程。
- 从软件质量和可靠性角度理解，软件测试是为保证软件质量、提高软件可靠性的活动，它应 用测试理论和技术，发现程序中的错误和缺陷而实施的过程。

软件工程-测试

- 从定义中看出软件测试的四个特点：
- 动态：软件测试总要通过一组输入执行程序。由于系统会处于不同的状态，因此对于同样的 输入可能产生不同的响应。
- 有限：在测试中实际能够观察的执行数量是有限的。正是这种矛盾带来测试技术（测试充分 性评判准则）和管理（测试工作量估计）两个方面的测试问题。
- 选取：很多测试手段的本质区别就是如何选择有限的测试集（测试用例）
- 预期：必须能够确定所观察到的程序执行输出是否可接受，否则测试工作就是无用的。

软件工程-测试

- 软件测试过程主要包括测试对象和测试方法两部分内容。测试对象分为程序代码和文档。 文档分为技术文档和用户文档。
  - 对技术文档，检查设计方案是否符合需求；
  - 对技术文档，检查设计方案是否正确；
  - 对用户手册，提供的系统安装过程说明是否详尽；
  - 对用户手册，提供的系统操作方法、过程是否正确；
  - 对用户手册，提供的示例数据是否准确；
  - 对用户手册，提供的信息是否完整、详尽且无二义性

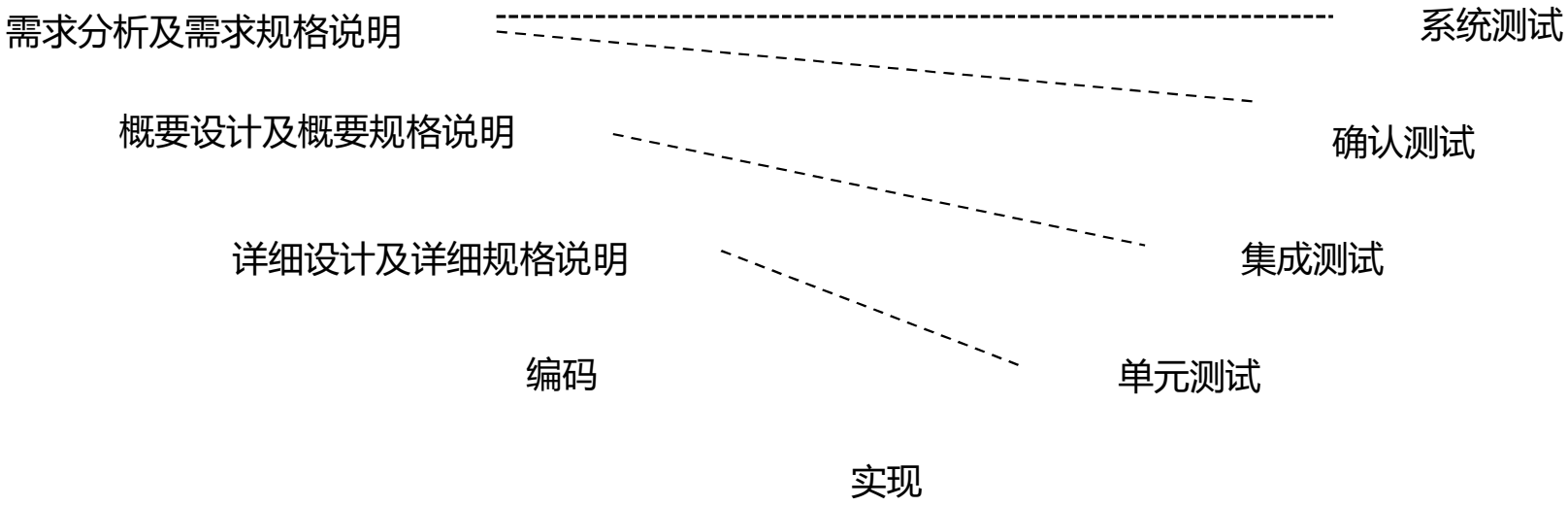
软件工程-测试

- 传统软件工程把测试作为独立阶段，直至代码结束才开始测试过程。但这时已经错过发现软件系统结构、业务逻辑、数据设计中存在严重问题的时机，等到代码结束再去查找，将更难以发现这些问题，对这些问题的 修改将付出更高的代价。



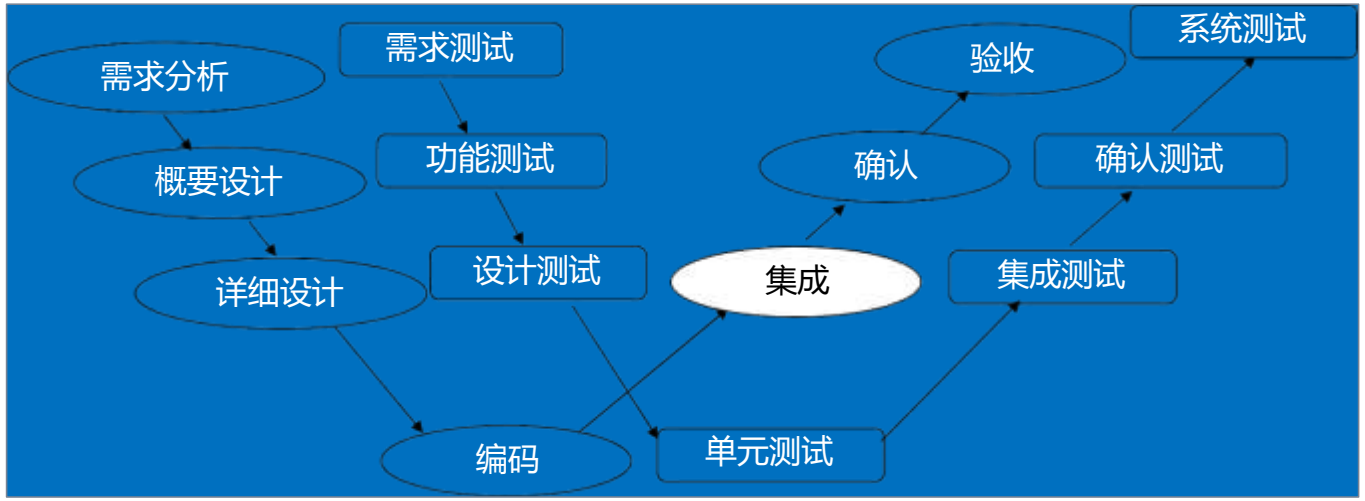
软件工程-测试

- 1. 软件测试过程模型——V模型
- V模型的重要价值在于，它定义了软件测试如何与软件工程各阶段相融合，它清楚地描述了各级别软件 测试与软件开发各阶段的对应关系。



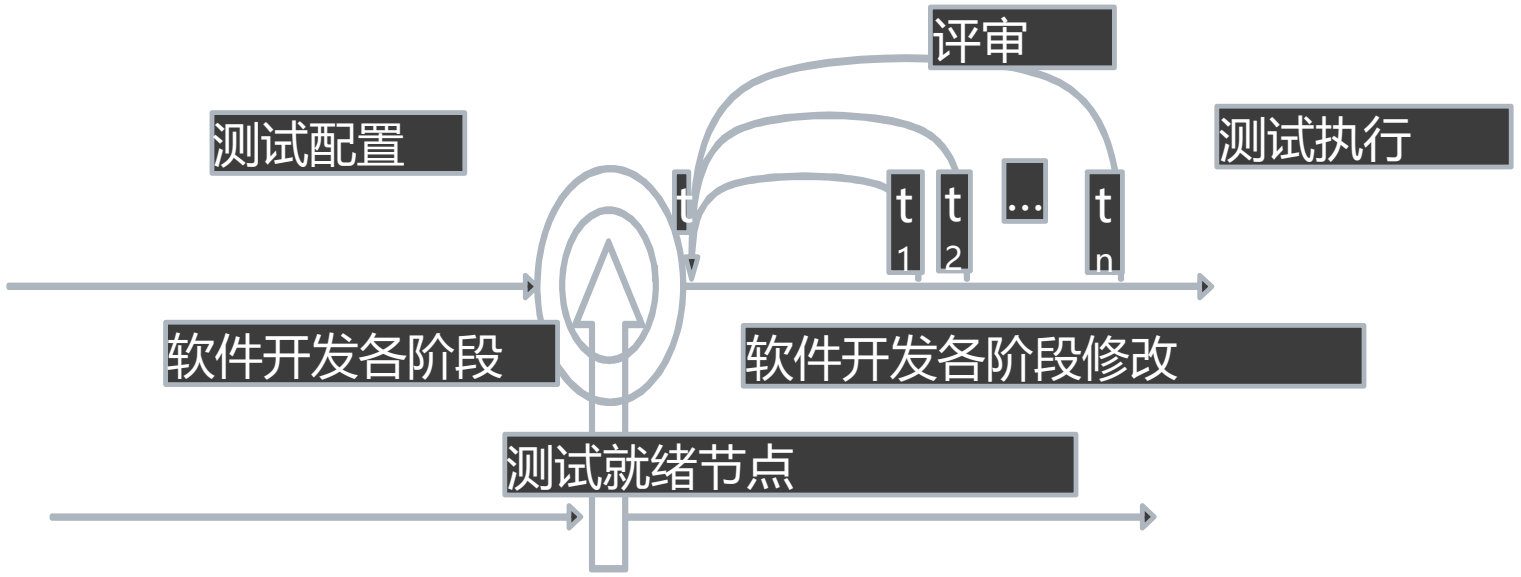
软件工程-测试

- 2.软件测试过程模型——W模型
- W模型的重要贡献在于，明确软件开发各阶段都要进行测试，而不仅仅是在编码结束后才开始。这样， 测试的对象不仅是代码，还可以是文档（需求规格说明、设计规格说明等）。



软件工程-测试

- 3. 软件测试过程模型——H模型
- 软件测试的H模型是对W模型在更高层次上的线性抽象。它明确表示，在任何一个开发流程， 只要有必要，并且测试配置已准备就绪，就能进行测试活动。



软件工程-测试

- 基本的软件测试技术
  - 静态测试与动态测试
  - 白盒测试与黑盒测试
  - 测试策略

软件工程-测试

- 软件测试技术分类（一）——静态测试
- 静态测试的测试对象包括源程序和文档。项目开发过程中产生大量的规格说明，对这些规 格说明的技术审查和管理复审，以及对文档的测试数据都属于静态测试。







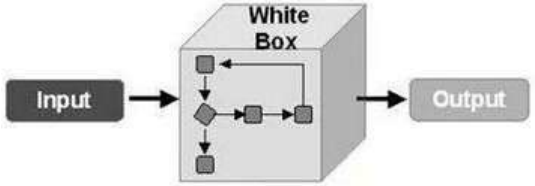
软件工程-测试

- 软件测试技术分类（一）——动态测试
- 动态测试的测试对象针对源程序。就源程序来讲，静态测试是指不运行程序就找出程序中存 在的错误，动态测试是通过运行程序而发现存在的错误和问题。



软件工程-测试

- 软件测试技术分类（二）——白盒测试
- 白盒测试又称为结构测试、基于覆盖的测试。它是针对模块内部逻辑 结构进行的测试。它面对程序内部的实现细节，分别对语句、条件、条件 组合、循环等控制结构、异常、错误处理等特殊流程设计测试用例。



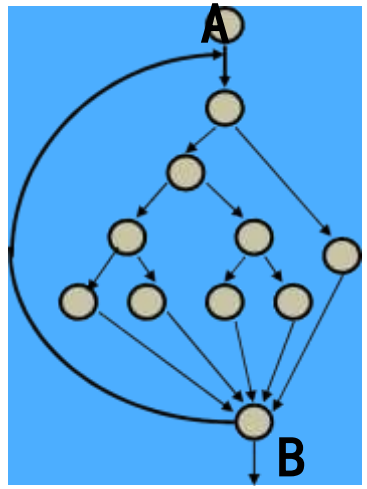
- 白盒测试中的穷尽测试

例：含5个分支,循环次数≤20,从A到B的可能路径：

51 + 52 + … + 519 + 520 ≈ 1.2×1014

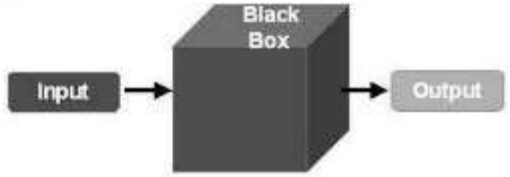
执行时间: 设测试一次需要2ms，

则穷尽测试需要约3万年



软件工程-测试

- 软件测试技术分类（二）——黑盒测试
- 黑盒测试是把模块作为一个整体进行测试。它不关心程序逻辑实现的具体细 节，而是关注模块的输入（接口）、输出（运行结果）。因而它测试的是模块功 能是否符合设计，运行时是否能（被）正确调用。
- 黑盒测试中的穷尽测试

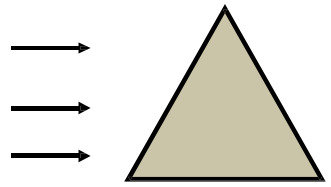


例:输入三条边长用黑盒测试中的穷尽测试，

可采用的测试用例数(设 字长32位)：

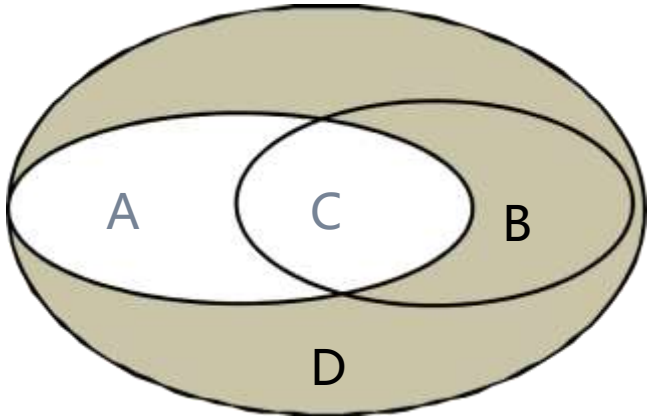
$$2^{32} \times 2^{32} \times 2^{32} \approx 2^{96}$$

执行时间: 设测试一次需1毫秒，共需约2^{61}年。



软件工程-测试

- 在实际测试过程中，无论是黑盒测试还是白盒测试都不能进行穷尽测试, 所以软件测试不可能发现 程序中存在的所有错误。为此，需精心组织测试方案、设计测试用例, 力争用尽可能少的测试用例，发 现尽可能多的错误。



黑盒测试与白盒测试能发现错误的关系

A 只能用黑盒测试发现的错误;

B 只能用白盒测试发现的错误;

C 两种方法都能发现的错误;

D 两种方法都不能发现的错误。

软件工程-测试

- 黑盒测试又称功能测试或行为测试，它主要根据设计说明中的功能设计来测试程序能否按预期实现。黑盒测试的目的是尽量发现系统功能中的错误。常见的系统功能错误有以下几类：
  - 功能不正确或不完整；
  - 界面或接口错误；
  - 数据结构错误；
  - 访问外部数据库错误；
  - 性能不满足需求；
  - 初始化或终止系统时的错误。

软件工程-测试

- 黑盒与不白盒测试的比较

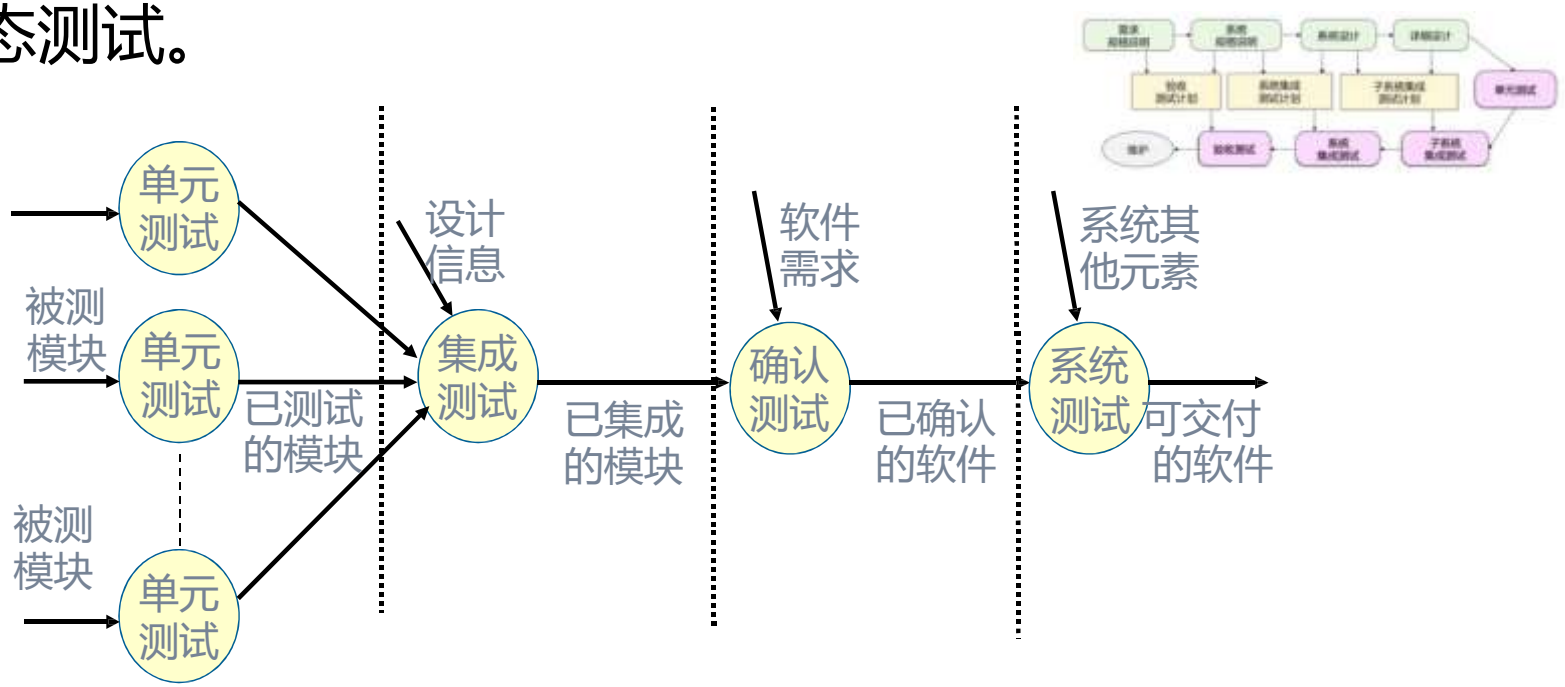
	黑盒测试	白盒测试
优点	▶适用于各测试阶段 ▶从产品功能角度测试 ▶容易入手生成测试数据	▶可以构成测试数据使特定程序部分得到测试 ▶有一定的充分性度量手段 ▶可获得较多工具支持（反向工程）
缺点	▶某些代码段得不到测试 ▶如果规格说明有误则无法发现 ▶不易进行充分性度量	▶不易生成测试数据 ▶无法对未实现规格说明的部分测试 ▶工作量大，通常只用于单元测试，有引用局限

软件工程-测试

- 软件测试阶段完成的主要任务有两类：
- 一类是局部模块的测试，它是整个测试阶段的基石。
- 另一类是软件系统全局结构的测试，它构成整个测试系统的大厦。
- 从局部到全局，是经过一系列测试过程转换而成，它们包括单元测试、集成测试、确认测试和系统测试，每个测试过程都有各自不同的测试策略。

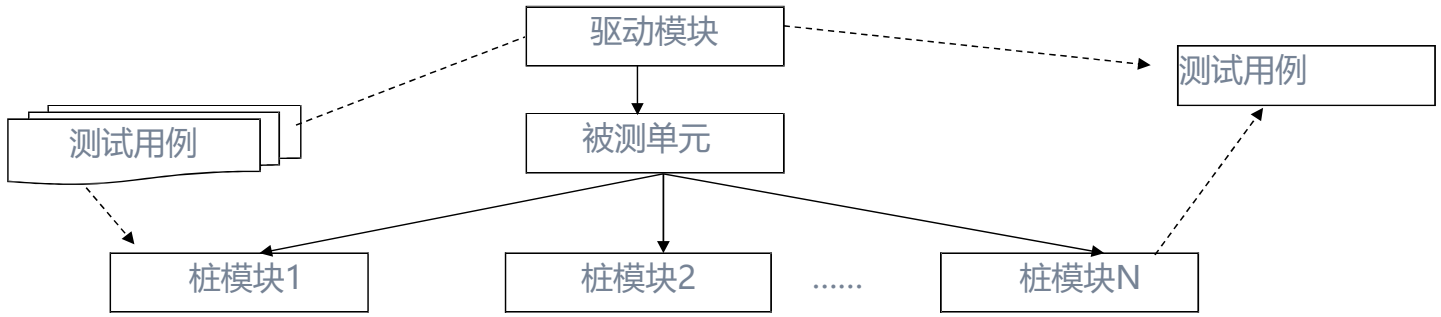
软件工程-测试

- 测试步骤及策略
- 所有测试过程都应采用综合测试策略；即先作静态分析，制订测试计划，再进行动态测试。



软件工程-测试

- 1. 单元测试——测试策略
  - 单元是软件测试过程中最小的测试单位，它有两个基本属性：
  - 单元是可以独立编译的最小组件（函数、接口、类）；
  - 单元是由个人开发的软件组件（菜单、界面）。



软件工程-测试

- 集成测试。集成测试是指在完成单元测试后，将各单元模块依据设计的软件系统结构，按照一定的集成测试 策略进行组装的过程。
- 集成测试在组装各模块时，应该注意下列因素：
  - 各模块间的数据传递是否会丢失；
  - 各模块间的数据传递是否按照期望进行传递；
  - 各模块组装后，是否能实现所期望的更复杂的功能；
  - 各模块组装后，是否出现对全局数据结构、公共数据操作的混乱，资源的竞争；
  - 各模块组装后，集成的误差是否会被快速放大，直到难以接受为止。

软件工程-测试

- 3. 确认测试。确认测试也称为验收测试，它是指在模拟用户实际操作的环境下（或开发环境下），运用黑盒测试法验证软件的有效性是否符合需求。

- 确认测试完成后，可能产生两种情形：

  - 用户的操作场景；
  - 用户的操作习惯；
  - 用户的操作流程；
  - ……

  - (1) 软件系统功能、性能、领域等要素满足用户需求规格说明，软件是有效的。
  - (2) 软件系统功能、性能、领域等要素存在不满足用户需求规格说明的某些方面，确认测试 文档指出存在的错误或问题、对比预期结果给出测试分析报告。

软件工程-测试

- 4. 系统测试。系统测试是指软件系统作为整个计算机系统的一部分，与计算机系统的硬件系统、数据、外部 其它软件、文档等要素相结合，在用户实际运行环境中进行的确认测试。
  - 功能测试　性能测试　压力测试　容量测试　安全测试　文档测试　恢复性测试　备份测试

软件工程-测试

- 系统测试——压力测试
- 压力测试也成为强度测试，是指系统在各种资源超负荷得情况下对运行状况进行测试。
- 压力测试的基本思路：计算机系统资源匮乏或有限的条件下运行的测试，是测试系统在使 用峰值时的性能。
- 压力测试的主要对象包括：内存、外部存储、网络负载、中断处理、缓冲区、事务队列、 在线用户量等。

软件工程-测试

- 系统测试——容量测试。容量测试是面向数据的测试，是指在系统运行时超额处理大数据量时的能力。在线系统：测试在某时临时缓冲区是否快溢出或填满，执行时间是否停机。要混合使用创建、更新、读和 删除不同的操作。
- 数据库系统：每个作业是在大量的业务数据下运行的，比如数据库中所有对象都需要处理的业务，复杂的 表检索。
- 文件交换：尤其长文件，长度长于通讯协议里典型最大值（1,2,4,8,...兆-或千兆字节）。比如邮件协议不支 持的长度。大长度通常是为了在通信中超时设定。
- 磁盘空间：试着去填充磁盘中任何有磁盘空间的地方。检查如果是已经不再有空余空间，甚至还有更多数 据要填入这个系统的情况下，会发生什么情况。有没有像溢出缓冲区这样的存储区？是否会有任何警告信号，是否有数据丢失?
- 文件系统：文件系统的文件的最大数目或者最大长度。

软件工程-测试

- 软件测试的目的是发现错误，测试完成之后的软件调试目的是为了定位和修改错误，以保证软件运行的正确性 和可靠性。
- 软件调试活动主要分为以下三部分内容：
- (1) 确定软件系统出现错误的准确位置，这需要一个过程，特别需要的是调试人员的经验和技巧。
- (2) 对发现错误的修改。修改错误也需要经验，如分析修改的程序是否涉及全局数据、是否涉及面向对象机制

软件工程-测试

- 测试报告文档
- 在软件测试各阶段完成之前，必须编写软件测试报告，并按照评审标准对软件测试报告进行评审。 编写测试报告的目的是发现并消除其中存在的遗漏、错误和不足，使得测试用例、测试预期结果等内 容符合标注及规范的要求。通过了评审的软件测试报告成为基线配置项，纳入项目管理的过程。
- 软件测试报告主要包括软件测试说明和软件测试报告两个部分 。

谢 谢!



